

C Language — 15 Module Course

1. Introduction to C Programming

- History of C
- Features
- Applications
- C program structure
- Compilation process
- First C program
- **Flowchart: C Program Execution**

2. Variables, Constants & Data Types

- Variables
- Constants
- `int`, `float`, `char`, `double`
- Type conversion
- **Flowchart: Variable & Data Flow**

3. Operators & Expressions

- Arithmetic
- Relational
- Logical
- Assignment
- Increment/Decrement
- Bitwise operators
- **Flowchart: Expression Evaluation**

4. Input, Output & Control Statements

- `printf()`
- `scanf()`
- `if`
- `if-else`
- Nested conditions
- **Flowchart: Decision Making**

5. Loops & Iteration

- `for`
- `while`
- `do-while`
- Nested loops
- `break`
- `continue`
- **Flowcharts: All Loop Types**

6. Functions

- Function declaration
- Definition
- Calling
- Parameters
- Return values

- Recursion
- **Flowchart: Function Execution**
- 7. **Arrays**
 - One-dimensional arrays
 - Two-dimensional arrays
 - Array traversal
 - Searching
 - Sorting
 - **Flowchart: Array Processing**
- 8. **Strings**
 - Character arrays
 - String input/output
 - String functions
 - String manipulation
 - **Flowchart: String Processing**
- 9. **Pointers**
 - Addresses
 - Pointer variables
 - Dereferencing
 - Pointer arithmetic
 - Pointers and arrays
 - **Flowchart: Pointer Memory Flow**
- 10. **Structures, Unions & Enumerations**
 - `struct`
 - Nested structures
 - Array of structures
 - `union`
 - `enum`
 - **Flowchart: Structure Data Processing**
- 11. **Dynamic Memory Allocation**
 - Stack vs Heap
 - `malloc()`
 - `calloc()`
 - `realloc()`
 - `free()`
 - **Flowchart: Dynamic Memory Lifecycle**
- 12. **File Handling**
 - File pointers
 - `fopen()`
 - `fclose()`
 - `fprintf()`
 - `fscanf()`
 - Reading/writing files
 - **Flowchart: File Handling Process**
- 13. **Preprocessor & Advanced C**
 - `#include`

- `#define`
- Macros
- Conditional compilation
- Command-line arguments
- Storage classes

14. Data Structures & C Projects

- Linked Lists
- Stacks
- Queues
- Searching
- Sorting
- Real-world C projects
- **Flowcharts for major algorithms**

15. C Interview Preparation & Career Guidance

- Important interview questions
- Coding problems
- Output-based questions
- Debugging
- Common mistakes
- C programming roadmap
- Placement preparation

Module 1: Introduction to C Programming

1.1 Learning Objectives

After completing this module, you will be able to:

- Understand what C programming is.
 - Learn the history and importance of C.
 - Understand the features of C.
 - Identify applications of C.
 - Understand the basic structure of a C program.
 - Understand how a C program is compiled and executed.
 - Write and execute your first C program.
 - Understand tokens and comments.
 - Use basic C programming terminology.
-

1.2 What is C?

C is a general-purpose, procedural programming language developed for system programming and application development.

C provides low-level memory access while also supporting structured programming concepts.

It is known for being:

- Fast
- Efficient
- Portable
- Structured
- Flexible
- Close to hardware

C is also considered a foundational programming language because many modern languages and systems are influenced by its design.

1.3 History of C

C was developed by **Dennis Ritchie** at **Bell Labs** in the early 1970s.

It evolved from earlier programming languages such as:

BCPL

↓

B

↓

C

C became especially important because it was used extensively in the development of the **UNIX operating system**.

1.4 Why Learn C?

Learning C provides a strong understanding of fundamental programming concepts.

It helps you understand:

- Variables
- Data types
- Memory

- Pointers
- Functions
- Arrays
- Structures
- Algorithms
- Operating-system concepts

C is particularly valuable for understanding **how programs interact with computer memory and hardware**.

1.5 Features of C

1. Procedural

C follows a procedural programming approach where programs are organized into functions and sequences of operations.

2. Fast

C programs can execute efficiently because C provides relatively low-level control over system resources.

3. Portable

C programs can generally be compiled for different platforms with appropriate modifications and compiler support.

4. Structured

Programs can be divided into smaller functions and logical blocks.

5. Low-Level Access

C provides features such as pointers that allow programmers to work directly with memory addresses.

6. Extensible

Program functionality can be extended through functions, libraries, and reusable modules.

1.6 Applications of C

C is used in many areas of computing.

Operating Systems

Parts of operating systems and system software are commonly implemented in C.

Embedded Systems

C is widely used in:

- Microcontrollers
- Automotive systems
- IoT devices
- Industrial controllers

Compilers

C has been used to implement many compilers and language tools.

Networking

C is used in network software and performance-sensitive applications.

Database Systems

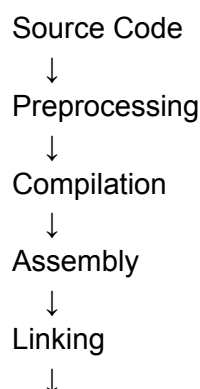
C is used in the implementation of several database systems and database components.

Game Development

C and C-based technologies are used in performance-critical game and graphics systems.

1.7 C Program Development Process

A C program normally goes through several stages:



Executable File



Execution

Let's understand each stage.

Step 1: Source Code

The programmer writes C code in a source file.

Example:

```
#include <stdio.h>
```

```
int main()  
{  
    printf("Hello World");  
    return 0;  
}
```

The file is usually saved with:

program.c

Step 2: Preprocessing

The preprocessor handles directives such as:

```
#include <stdio.h>
```

It processes preprocessor instructions before compilation.

Step 3: Compilation

The compiler translates C source code into lower-level code and checks the program for errors.

Step 4: Assembly

The resulting lower-level representation is converted into machine-oriented instructions.

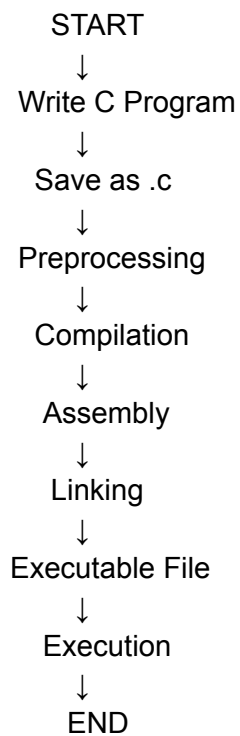
Step 5: Linking

The linker combines the required object code and libraries to create the final executable.

Step 6: Execution

The operating system loads the executable, and the processor executes the program.

1.8 C Program Execution Flowchart



Easy Understanding

Think of it like this:

You write → C processes → compiler translates → linker creates executable → computer runs it.

1.9 Your First C Program

Let's write the traditional first C program.

```
#include <stdio.h>
```

```
int main()
{
    printf("Hello, World!");
    return 0;
}
```

Output:

Hello, World!

1.10 Understanding the Program

Let's break it down.

#include <stdio.h>

```
#include <stdio.h>
```

This includes the Standard Input/Output library header.

It provides declarations for functions such as:

```
printf()
scanf()
```

int main()

```
int main()
```

main() is the function where program execution begins in a hosted C program.

int indicates that the function returns an integer value.

Opening Brace

```
{
```

Marks the beginning of the function body.

printf()

```
printf("Hello, World!");
```

Displays text on the standard output.

return 0;

```
return 0;
```

Returns the value 0 from `main`, conventionally indicating successful termination.

Closing Brace

```
}
```

Marks the end of the function body.

1.11 Basic Structure of a C Program

A typical C program can contain:

Documentation / Comments



Preprocessor Directives



Global Declarations



main() Function



Local Declarations



Statements



User-Defined Functions

Example:

```
#include <stdio.h>
```

```
int add(int a, int b);
```

```
int main()
{
    int result;

    result = add(10, 20);

    printf("%d", result);

    return 0;
}

int add(int a, int b)
{
    return a + b;
}
```

1.12 C Statements

A **statement** is an instruction that performs an operation.

Example:

```
printf("Hello");
```

Another example:

```
x = 10;
```

Most C statements end with a semicolon:

```
;
```

Example:

```
int x = 10;
```

1.13 Semicolon in C

The semicolon marks the end of many C statements.

Example:

```
int age = 20;
```

```
printf("%d", age);
```

Incorrect:

```
int age = 20
```

Correct:

```
int age = 20;
```

1.14 Comments in C

Comments are ignored by the compiler and are used to explain code.

Single-Line Comment

```
// This is a comment
```

Example:

```
// Print the student's age
printf("%d", age);
```

Multi-Line Comment

```
/*
    This is a
    multi-line comment
*/
```

Comments improve code readability and maintainability.

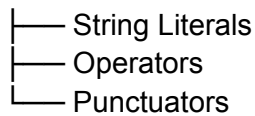
1.15 C Tokens

A **token** is the smallest meaningful element recognized by the C compiler.

Major categories include:

C Tokens

- |
- |— Keywords
- |— Identifiers
- |— Constants



We will study each category in detail in later modules.

1.16 Keywords

Keywords are reserved words that have predefined meanings in C.

Examples:

```
int
char
float
if
else
for
while
return
struct
void
```

You cannot normally use a keyword as an identifier.

Incorrect:

```
int return;
```

1.17 Identifiers

Identifiers are names given to programming elements such as:

- Variables
- Functions
- Arrays
- Structures

Example:

```
int age;
```

Here:

int → keyword
age → identifier

Identifier Rules

An identifier:

- Can contain letters.
- Can contain digits.
- Can contain underscore `_`.
- Cannot start with a digit.
- Cannot be a keyword.
- Is case-sensitive.

Valid:

age
student_name
marks1
_total

Invalid:

1student
student-name
float

1.18 Case Sensitivity

C is **case-sensitive**.

These are different identifiers:

age
Age
AGE

Similarly:

printf()

is different from:

Printf()

1.19 Standard Input and Output

C provides standard functions for input and output.

Output

printf()

Input

scanf()

Example:

```
#include <stdio.h>
```

```
int main()
{
    int age;

    scanf("%d", &age);

    printf("Age = %d", age);

    return 0;
}
```

We will study `scanf()` and format specifiers in detail in a later module.

1.20 Basic C Program Flowchart

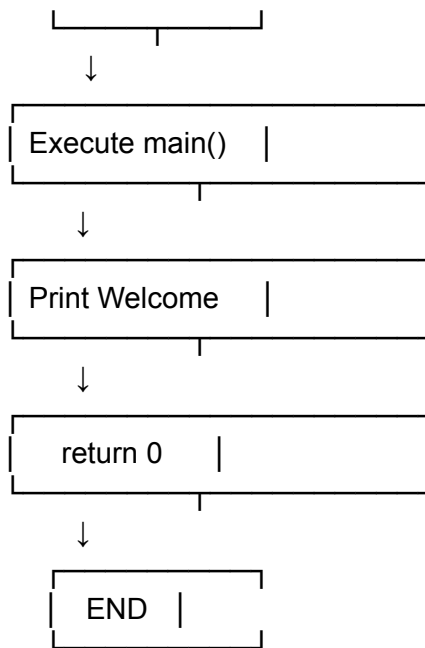
Consider:

```
#include <stdio.h>
```

```
int main()
{
    printf("Welcome to C Programming");
    return 0;
}
```

Flowchart:





Flow Explanation

1. Program starts.
 2. Execution enters `main()`.
 3. `printf()` displays the message.
 4. `return 0` terminates `main`.
 5. Program ends.
-

1.21 C Program Example

Program: Add Two Numbers

```
#include <stdio.h>
```

```
int main()
{
    int a = 10;
    int b = 20;
    int sum;

    sum = a + b;

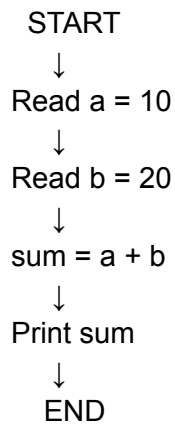
    printf("Sum = %d", sum);

    return 0;
}
```


Output:

Sum = 30

Flowchart

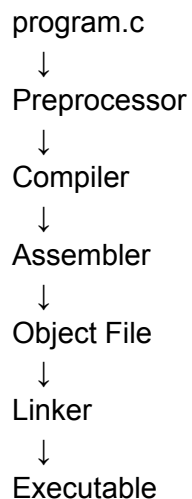


1.22 C Compilation Flow

For a source file named:

program.c

The conceptual process is:



The exact intermediate files and implementation details can vary by compiler and platform.

1.23 Advantages of C

- High performance.
 - Efficient memory usage.
 - Portable.
 - Structured programming.
 - Large ecosystem of libraries and tools.
 - Suitable for system-level programming.
 - Provides direct memory manipulation through pointers.
-

1.24 Limitations of C

- Manual memory management.
 - No built-in object-oriented programming model.
 - No automatic bounds checking for arrays.
 - Pointer-related errors can be difficult to debug.
 - Requires careful memory and resource management.
-

1.25 C vs Other Languages

Feature	C	Python	Java
Type System	Static	Dynamic	Static
Execution	Compiled	Typically interpreted/bytecode-based	Compiled to bytecode
Memory Control	High	Mostly automatic	Mostly automatic
Pointers	Yes	No traditional pointer syntax	No explicit pointer arithmetic
Performance	High	Generally lower	High
OOP	Not built-in	Supported	Core paradigm

1.26 Common Beginner Errors

Error 1: Missing Semicolon

```
printf("Hello")
```

Correct:

```
printf("Hello");
```

Error 2: Incorrect **main**

```
main()
```

For modern standard C, prefer:

```
int main(void)
```

or:

```
int main()
```

Error 3: Incorrect Header

```
#include stdio.h
```

Correct:

```
#include <stdio.h>
```

Error 4: Incorrect Function Name

```
Printf("Hello");
```

Correct:

```
printf("Hello");
```

C is case-sensitive.

1.27 Best Practices

✅ Use meaningful names.

```
int studentAge;
```

instead of:

```
int x;
```

when the meaning is specifically a student's age.

✓ Use indentation.

```
int main()
{
    printf("Hello");
    return 0;
}
```

✓ Add comments where they improve understanding.

✓ Keep functions focused on clear responsibilities.

✓ Compile frequently while developing.

1.28 Interview Questions

1. Who developed C?

Answer:

C was developed by **Dennis Ritchie** at Bell Labs in the early 1970s.

2. Why is C called a middle-level language?

Answer:

C combines high-level programming constructs with low-level capabilities such as direct memory manipulation, so it is often described as a middle-level language.

3. What is the purpose of `main()`?

Answer:

`main()` is the entry point for execution in a hosted C program.

4. What is `stdio.h`?

Answer:

`stdio.h` is a standard C header that declares functions and types used for standard input and output, including `printf()` and `scanf()`.

5. Why is C case-sensitive?

Answer:

C treats uppercase and lowercase letters as different characters, so identifiers such as **age**, **Age**, and **AGE** are distinct.

6. What is a compiler?**Answer:**

A compiler translates source code written in a programming language into a lower-level representation that can ultimately be executed by a computer.

1.29 Practical Lab

Task 1

Write a C program to print:

Hello, C Programming!

Task 2

Write a program to print your:

- Name
- College
- Branch

Task 3

Write a program to add two numbers.

Task 4

Write a program to calculate the area of a rectangle.

Task 5

Draw the flowchart for a program that calculates the sum of two numbers.

Module 2: Variables, Constants & Data Types

2.1 Learning Objectives

After completing this module, you will be able to:

- Understand variables and constants.
 - Learn C's fundamental data types.
 - Understand declaration and initialization.
 - Understand memory representation at a basic level.
 - Learn format specifiers.
 - Understand type conversion and casting.
 - Identify signed and unsigned integer types.
 - Write programs using different data types.
-

2.2 What is a Variable?

A **variable** is a named storage location used to hold a value that can change during program execution.

Example:

```
int age = 20;
```

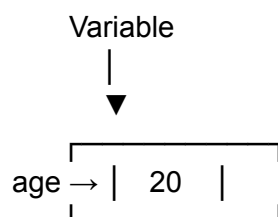
Here:

int → Data type

age → Variable name

20 → Initial value

Conceptually:



If later we write:

```
age = 21;
```

the stored value changes.

2.3 Why Do We Need Variables?

Variables allow programs to store and manipulate information.

For example:

```
int marks = 85;
```

The program can later calculate:

```
marks + 5
```

Variables are commonly used to store:

- Age
 - Marks
 - Salary
 - Temperature
 - Counters
 - Calculated results
 - User input
-

2.4 Variable Declaration

Declaration tells the compiler the variable's **type and name**.

Syntax

```
data_type variable_name;
```

Example:

```
int age;  
float salary;  
char grade;
```

2.5 Variable Initialization

Initialization means assigning an initial value when declaring a variable.

```
int age = 20;  
float percentage = 85.5;  
char grade = 'A';
```

Conceptually:

Declaration



Initialization



Variable Ready

2.6 Declaration vs Initialization

Declaration	Initialization
Specifies type and name	Assigns initial value
<code>int age;</code>	<code>int age = 20;</code>
Does not necessarily assign a useful initial value	Gives an initial value

Example:

```
int age;    // Declaration
age = 20;   // Assignment
```

versus:

```
int age = 20; // Declaration + initialization
```

2.7 Rules for Naming Variables

Variable names must follow C's identifier rules.

Valid

```
int age;
int student_age;
int marks1;
int _count;
```

Invalid

```
int 1age;
int student-age;
int student age;
int float;
```

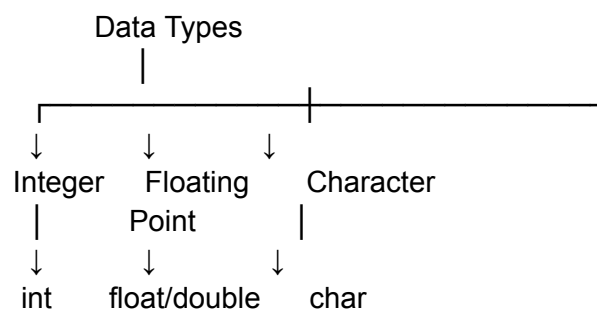

Important Rules

- Cannot start with a digit.
 - Can contain letters, digits, and `_`.
 - Cannot contain spaces.
 - Cannot be a keyword.
 - C is case-sensitive.
-

2.8 Data Types

A **data type** specifies what kind of value a variable can represent and helps determine how the value is stored and interpreted.

C provides several fundamental types.



Common fundamental types include:

- `char`
- `int`
- `float`
- `double`
- `void`

C also provides derived and user-defined types, which we will study later.

2.9 Integer Data Type

`int` is used to represent integer values.

Example:

```
int age = 20;
```

```
int marks = 95;  
int count = 100;
```

Integer values do not contain a fractional part.

```
10  
25  
-50  
0
```

2.10 Character Data Type

char is used to store a character.

Example:

```
char grade = 'A';
```

Character constants use **single quotes**.

Correct:

```
char grade = 'A';
```

Incorrect:

```
char grade = "A";
```

"A" is a string literal, not a character constant.

2.11 Floating-Point Data Types

Floating-point types represent numbers that can contain a fractional part.

float

```
float temperature = 36.5f;
```

double

```
double salary = 45678.75;
```

double generally provides greater precision than **float**.

2.12 void

`void` represents the absence of a value or type.

It is commonly used with functions.

Example:

```
void display()
{
    printf("Hello");
}
```

Here, `void` indicates that the function does not return a value.

2.13 Fundamental Data Types

Data Type	Typical Purpose	Example
<code>char</code>	Character data	<code>'A'</code>
<code>int</code>	Integer values	<code>100</code>
<code>float</code>	Single-precision floating point	<code>10.5f</code>
<code>double</code>	Double-precision floating point	<code>10.5</code>
<code>void</code>	No value/type	<code>void</code> <code>display()</code>

Note: The exact size and range of C types are implementation-dependent. Use `sizeof` and standard limits when you need exact information for a particular environment.

2.14 sizeof Operator

The `sizeof` operator determines the size, in bytes, of a type or object.

Example:

```
#include <stdio.h>
```

```
int main()
{
    printf("%zu\n", sizeof(int));
    printf("%zu\n", sizeof(float));
    printf("%zu\n", sizeof(double));
    printf("%zu\n", sizeof(char));

    return 0;
}
```

The exact values depend on the compiler and platform.

2.15 Signed and Unsigned Types

Integer types can have signed or unsigned forms.

Signed

Can represent negative and non-negative values.

```
signed int temperature = -10;
```

Unsigned

Represents only non-negative values.

```
unsigned int count = 100;
```

Example:

```
signed int
  ↓
negative ← 0 → positive
```

```
unsigned int
  ↓
0 → positive
```

2.16 Short and Long Integer Types

C provides integer type modifiers such as:

short int
long int
long long int

Examples:

short int a;
long int b;
long long int c;

Their exact ranges depend on the implementation.

2.17 Constants

A **constant** is a value that does not change during program execution.

Examples:

10
25.5
'A'
"Hello"

2.18 Integer Constants

Examples:

10
100
-25
0

Example:

int age = 20;

Here 20 is an integer constant.

2.19 Floating-Point Constants

Examples:

```
10.5  
3.14  
-25.75
```

Example:

```
float pi = 3.14f;
```

2.20 Character Constants

Character constants are enclosed in single quotes.

```
'A'  
'B'  
'7'  
'#'
```

Example:

```
char symbol = '#';
```

2.21 String Literals

A string literal is a sequence of characters enclosed in double quotes.

```
"Hello"  
"Welcome to C"  
"Prasanna"
```

Example:

```
printf("Welcome to C");
```

Strings are stored as arrays of characters and are covered in detail in the **Strings module**.

2.22 Using **const**

The **const** qualifier can be used when an object should not be modified through that identifier after initialization.

Example:

```
const int MAX = 100;
```

Attempting to modify it:

```
MAX = 200;
```

is not allowed.

Example

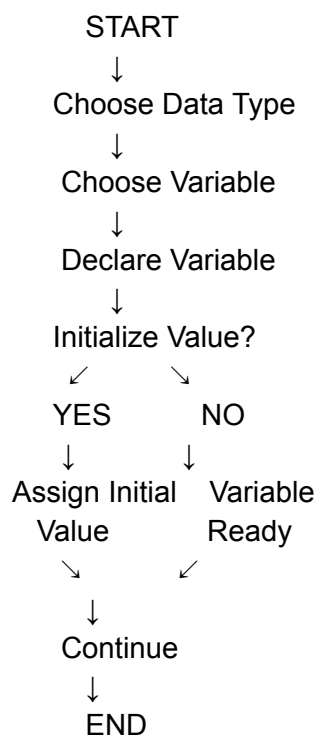
```
#include <stdio.h>
```

```
int main()
{
    const float PI = 3.14159f;

    printf("%f", PI);

    return 0;
}
```

2.23 Variable Declaration Flowchart



2.24 Memory Concept

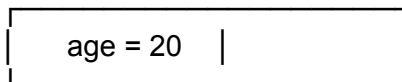
When a variable is created, the implementation allocates storage for its object.

For example:

```
int age = 20;
```

Conceptually:

Memory



The exact memory representation depends on the type and implementation.

Later, pointers will allow us to examine addresses and manipulate objects more directly.

2.25 Format Specifiers

Format specifiers tell formatted I/O functions how to interpret or display values.

Common examples:

Data	Common <code>printf</code> Specifier
<code>int</code>	<code>%d</code>
<code>unsigned int</code>	<code>%u</code>
<code>float</code>	<code>%f</code>
<code>double</code>	<code>%f</code> in <code>printf</code>
<code>char</code>	<code>%c</code>
<code>String</code>	<code>%s</code>

Example:


```
#include <stdio.h>

int main()
{
    int age = 20;
    float percentage = 85.5f;
    char grade = 'A';

    printf("Age = %d\n", age);
    printf("Percentage = %f\n", percentage);
    printf("Grade = %c\n", grade);

    return 0;
}
```

2.26 `scanf()` and Format Specifiers

`scanf()` is commonly used to read formatted input.

Example:

```
int age;
```

```
scanf("%d", &age);
```

For a floating-point variable:

```
float marks;
```

```
scanf("%f", &marks);
```

For a character:

```
char grade;
```

```
scanf(" %c", &grade);
```

For `double`:

```
double salary;
```

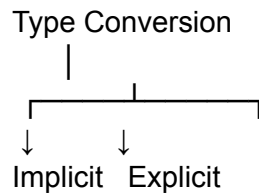
```
scanf("%lf", &salary);
```

The `&` operator supplies the address of the object to `scanf()` for these ordinary scalar inputs.

2.27 Type Conversion

Type conversion occurs when a value is converted from one data type to another.

There are two major forms:



2.28 Implicit Type Conversion

The compiler performs the conversion automatically according to C's conversion rules.

Example:

```
int a = 10;  
double b;
```

```
b = a;
```

The integer value is converted to **double**.

Conceptually:

```
int  
↓  
double
```

2.29 Explicit Type Conversion

The programmer explicitly requests a conversion using a cast.

Syntax

(type) expression

Example:

```
int a = 10;  
int b = 3;
```

```
float result = (float)a / b;
```

Output:

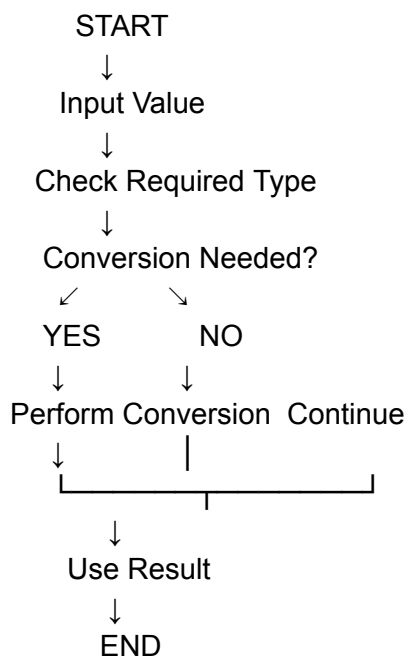
3.333333

Without the cast:

```
float result = a / b;
```

the division is performed as integer division before the result is converted to `float`.

2.30 Type Casting Flowchart



2.31 Example: Student Marks

```
#include <stdio.h>
```

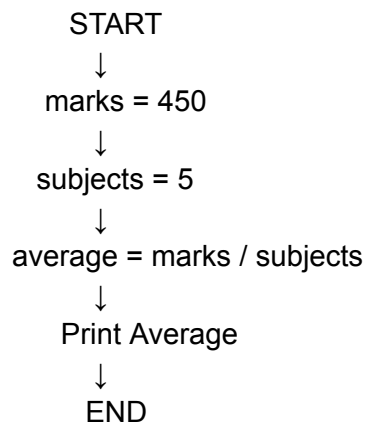
```
int main()  
{  
    int marks = 450;  
    int subjects = 5;
```

```
float average = (float)marks / subjects;  
  
printf("Average = %.2f", average);  
  
return 0;  
}
```

Output:

Average = 90.00

Flowchart



2.32 Multiple Variable Declaration

You can declare multiple variables of the same type in one declaration.

```
int a, b, c;
```

You can also initialize them:

```
int a = 10, b = 20, c = 30;
```

However, separate declarations may sometimes improve readability.

2.33 Variable Scope Preview

The **scope** of a variable determines where it can be accessed.

For example:

```
int main()
{
    int age = 20;

    printf("%d", age);

    return 0;
}
```

Here, **age** is a local variable whose scope is limited to the block in which it is declared.

Scope and storage duration will be studied in greater detail later.

2.34 Common Errors

Error 1: Using an Undeclared Variable

Incorrect:

```
age = 20;
```

Correct:

```
int age;
age = 20;
```

Error 2: Assigning a Character Incorrectly

Incorrect:

```
char grade = "A";
```

Correct:

```
char grade = 'A';
```

Error 3: Confusing **%d** and **%f**

For an **int**:

```
int age = 20;
printf("%d", age);
```

For a `float`:

```
float marks = 85.5f;  
printf("%f", marks);
```

Error 4: Unexpected Integer Division

```
int a = 5;  
int b = 2;
```

```
float result = a / b;
```

The result is `2.0`, not `2.5`, because `a / b` is evaluated as integer division.

Use:

```
float result = (float)a / b;
```

2.35 Best Practices

✓ Choose meaningful variable names.

```
int studentAge;
```

✓ Initialize variables when appropriate.

```
int count = 0;
```

✓ Use `const` for values that should not be modified.

```
const int MAX_STUDENTS = 100;
```

✓ Use the correct format specifier.

✓ Avoid unnecessary type conversions.

✓ Use parentheses when they make conversions or expressions clearer.

2.36 Practical Programs

Program 1: Store and Display Student Information

```
#include <stdio.h>
```

```
int main()
{
    int age = 20;
    float percentage = 85.5f;
    char grade = 'A';

    printf("Age: %d\n", age);
    printf("Percentage: %.2f\n", percentage);
    printf("Grade: %c\n", grade);

    return 0;
}
```

Program 2: Calculate Area

```
#include <stdio.h>
```

```
int main()
{
    float length = 10.5f;
    float width = 5.0f;

    float area = length * width;

    printf("Area = %.2f", area);

    return 0;
}
```

2.37 Interview Questions

1. What is a variable?

A variable is a named object used to store a value that can change during program execution.

2. What is a constant?

A constant is a value that does not change as part of the program's intended computation.

3. What is the difference between **float** and **double**?

Both represent floating-point values, but `double` generally provides greater precision than `float`.

4. What is type casting?

Type casting is an explicit conversion of an expression to a specified type using the cast syntax.

Example:

`(float)a`

5. What is `sizeof`?

`sizeof` is an operator that determines the size in bytes of a type or object.

6. What is the difference between `char` and a string?

`char` represents a single character, while a string is a sequence of characters stored in a character array and terminated by a null character.

2.38 Practical Lab

Task 1

Create variables to store:

- Student name
- Age
- Percentage
- Grade

Display all values.

Task 2

Write a program to calculate the area of a circle.

Task 3

Write a program to convert Celsius to Fahrenheit.

Task 4

Write a program demonstrating integer-to-floating-point conversion.

Task 5

Use `sizeof` to display the sizes of several fundamental types on your compiler.

Task 6

Create a `const` variable for the maximum number of students and attempt to modify it. Observe the compiler diagnostic.

Module 3: Operators & Expressions

3.1 Learning Objectives

After completing this module, you will be able to:

- Understand operators and operands.
 - Use arithmetic operators.
 - Understand relational and equality operators.
 - Use logical operators.
 - Understand assignment operators.
 - Use increment and decrement operators.
 - Understand bitwise operators.
 - Learn conditional and `sizeof` operators.
 - Understand operator precedence and associativity.
 - Evaluate C expressions correctly.
 - Apply operators in practical programs.
-

3.2 What is an Operator?

An **operator** is a symbol that tells the compiler to perform an operation on one or more operands.

Example:

```
int sum = a + b;
```

Here:

+ → Operator

a → Operand

b → Operand

So:

Operand + Operand

↓
Result

3.3 What is an Operand?

An **operand** is a value or expression on which an operator acts.

Example:

a + b

- a → Operand
- + → Operator
- b → Operand

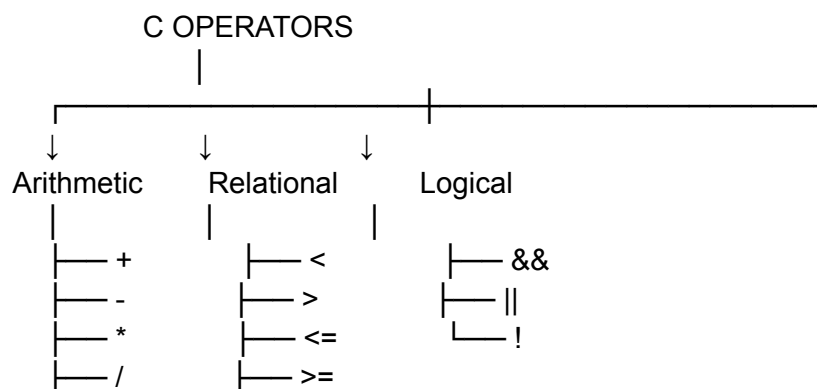
Another example:

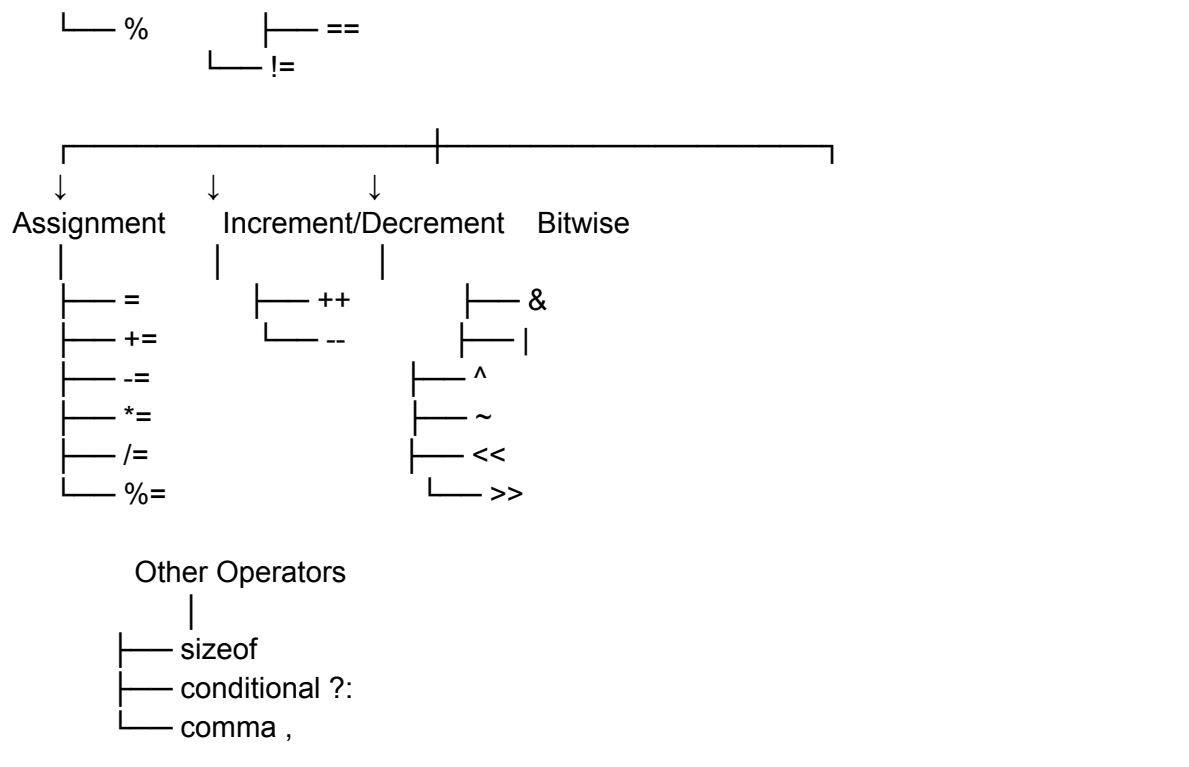
x * 10

- x → Operand
 - * → Operator
 - 10 → Operand
-

3.4 Types of Operators in C

C provides several categories of operators:





3.5 Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder

Addition

```
int result = 10 + 20;
```

Result:

30

Subtraction

```
int result = 20 - 10;
```

Result:

10

Multiplication

```
int result = 10 * 5;
```

Result:

50

Division

```
int result = 20 / 5;
```

Result:

4

Remember: when both operands are integers, `/` performs integer division.

```
int result = 5 / 2;
```

Result:

2

Modulus %

The modulus operator gives the remainder.

```
int result = 10 % 3;
```

Result:

1

Example

$10 \div 3$

Quotient = 3

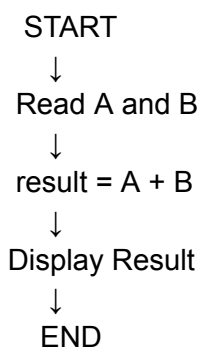
Remainder = 1

3.6 Arithmetic Expression Flowchart

Example:

result = a + b;

Flowchart:



3.7 Relational Operators

Relational operators compare two values.

Operator	Meaning
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal
==	Equal to
!=	Not equal to

The result of a comparison is an `int` value: **0** for false and **1** for true.

Example

```
int a = 10;  
int b = 20;
```

```
printf("%d", a < b);
```

Output:

1

Because `10 < 20` is true.

Example

```
printf("%d", a > b);
```

Output:

0

Because `10 > 20` is false.

3.8 Important: `=` vs `==`

This is one of the most common beginner mistakes.

Assignment

```
a = 10;
```

Means:

Store 10 in `a`.

Equality Comparison

```
a == 10
```

Means:

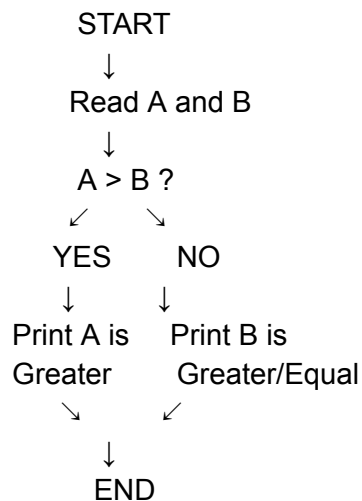
Check whether `a` is equal to 10.

Remember:

`=` → Assignment

== → Equality comparison

3.9 Relational Decision Flowchart



3.10 Logical Operators

Logical operators combine or modify conditions.

Operator	Name
&&	Logical AND
,	
!	Logical NOT

In C, logical operators produce 0 or 1.

3.11 Logical AND &&

The result is true only when **both operands are nonzero**.

A	B	A && B
---	---	--------

Fals e	Fals e	False
Fals e	True	False
True	Fals e	False
True	True	True

Example:

```
int age = 20;
int marks = 80;
```

```
printf("%d", age >= 18 && marks >= 40);
```

Output:

1

Both conditions are true.

3.12 Logical OR ||

The result is true if **at least one operand is nonzero**.

A	B	A B
Fals e	Fals e	False
Fals e	True	True
True	Fals e	True
True	True	True

Example:

```
int age = 16;
```



```
printf("%d", age < 18 || age > 60);
```

Output:

1

3.13 Logical NOT !

! reverses the logical value.

```
int x = 0;
```

```
printf("%d", !x);
```

Output:

1

Because zero is false.

Example:

```
int x = 10;
```

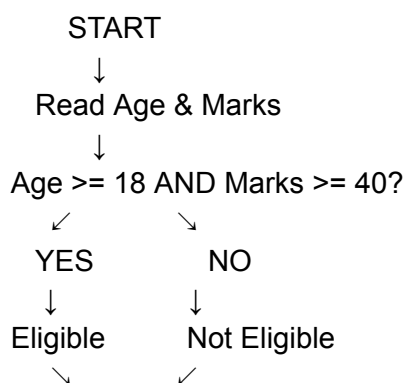
```
printf("%d", !x);
```

Output:

0

Because a nonzero value is true.

3.14 Logical Operator Flowchart



↓
END

3.15 Assignment Operators

Assignment operators assign values to variables.

Basic Assignment

`x = 10;`

Compound Assignment Operators

Operator	Example	Equivalent
<code>=</code>	<code>x = 5</code>	Assign
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>x -= 5</code>	<code>x = x - 5</code>
<code>*=</code>	<code>x *= 5</code>	<code>x = x * 5</code>
<code>/=</code>	<code>x /= 5</code>	<code>x = x / 5</code>
<code>%=</code>	<code>x %= 5</code>	<code>x = x % 5</code>

Example

`int x = 10;`

`x += 5;`

Now:

`x = 15`

3.16 Increment Operator ++

The increment operator increases a value by 1.

```
x++;
```

Equivalent to:

```
x = x + 1;
```

Example:

```
int x = 5;
```

```
x++;
```

```
printf("%d", x);
```

Output:

6

3.17 Decrement Operator --

The decrement operator decreases a value by 1.

```
x--;
```

Equivalent to:

```
x = x - 1;
```

Example:

```
int x = 5;
```

```
x--;
```

```
printf("%d", x);
```

Output:

4

3.18 Prefix vs Postfix

This is an important interview concept.

Prefix Increment

`++x;`

The increment occurs before the value is used in the surrounding expression.

Example:

```
int x = 5;  
int y = ++x;
```

Result:

```
x = 6  
y = 6
```

Postfix Increment

`x++;`

The old value is used for the surrounding expression, then `x` is incremented.

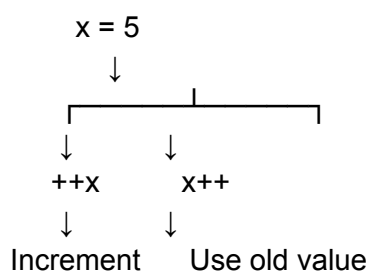
Example:

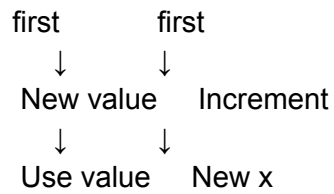
```
int x = 5;  
int y = x++;
```

Result:

```
x = 6  
y = 5
```

3.19 Prefix vs Postfix Flowchart





3.20 Bitwise Operators

Bitwise operators work on the individual bits of integer operands.

Operator	Meaning
&	Bitwise AND
,	,
^	Bitwise XOR
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

3.21 Bitwise AND &

Example:

5 = 0101

3 = 0011

Perform AND:

0101

0011

0001

Result:

1

3.22 Bitwise OR |

5 = 0101

3 = 0011

0101

0011

0111

Result:

7

3.23 Bitwise XOR ^

XOR produces 1 when the corresponding bits are different.

5 = 0101

3 = 0011

0101

0011

0110

Result:

6

3.24 Bitwise NOT ~

The ~ operator flips every bit of its operand.

For signed integers, the resulting value depends on the integer representation used by the implementation; modern systems commonly use two's-complement representation.

Example:

```
int x = 5;
```

```
int result = ~x;
```

Do not memorize a universal decimal result without considering the type and implementation.

3.25 Left Shift <<

Left shift moves bits to the left.

Example:

5 = 0101

5 << 1

Conceptually:

0101

↓

1010

For suitable nonnegative values, shifting left by one position corresponds to multiplying by 2, provided the result remains representable.

3.26 Right Shift >>

Right shift moves bits to the right.

Example:

8 = 1000

8 >> 1

Conceptually:

1000

↓

0100

For nonnegative values, this corresponds to integer division by 2 for each shift, with truncation.

For negative signed values, right-shift behavior is implementation-defined, so avoid oversimplified assumptions.

3.27 Bitwise Example

```
#include <stdio.h>
```

```
int main()
{
    int a = 5;
    int b = 3;

    printf("AND = %d\n", a & b);
    printf("OR = %d\n", a | b);
    printf("XOR = %d\n", a ^ b);

    return 0;
}
```

Output:

```
AND = 1
OR = 7
XOR = 6
```

3.28 Conditional Operator ? :

The conditional operator is the only ternary operator in standard C.

Syntax

```
condition ? expression1 : expression2;
```

Example:

```
int age = 20;

const char *result =
    (age >= 18) ? "Eligible" : "Not Eligible";
```

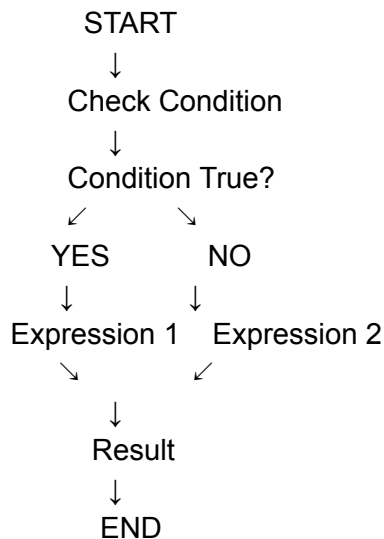
If the condition is true:

Eligible

Otherwise:

Not Eligible

3.29 Conditional Operator Flowchart



3.30 **sizeof** Operator

sizeof determines the size in bytes of a type or object.

Example:

```
#include <stdio.h>
```

```
int main()
{
    int x;

    printf("%zu", sizeof(x));

    return 0;
}
```

The exact size depends on the implementation.

You can also use:

```
sizeof(int)
```

sizeof(float)
sizeof(double)

3.31 Comma Operator

The comma operator evaluates its left operand and then its right operand; the result of the overall expression is the value of the right operand.

Example:

```
int x;
```

```
x = (10, 20);
```

After execution:

```
x = 20
```

Parentheses are important here because commas can also act as separators in declarations and function arguments.

3.32 Operator Precedence

When an expression contains multiple operators, C uses **precedence** to determine which operations bind more strongly.

Example:

```
int result = 10 + 5 * 2;
```

Multiplication has higher precedence than addition.

Therefore:

```
5 * 2 = 10
```

```
10 + 10 = 20
```

Result:

```
20
```

3.33 Using Parentheses

Parentheses can make the intended order explicit.

```
int result = (10 + 5) * 2;
```

Now:

$10 + 5 = 15$

$15 * 2 = 30$

Result:

30

Best Practice

When an expression could be confusing, use parentheses rather than relying on memory alone.

3.34 Operator Precedence — Simplified View

From relatively high to lower precedence:

Postfix

↓

Unary

↓

Multiplicative

↓

Additive

↓

Relational

↓

Equality

↓

Logical AND

↓

Logical OR

↓

Conditional

↓
Assignment

This is a simplified learning view; the full C precedence table contains additional operators and details.

3.35 Associativity

When operators have the same precedence, **associativity** determines the grouping direction.

For example, multiplication is left-associative:

$20 / 5 * 2$

is grouped as:

$(20 / 5) * 2$

Result:

8

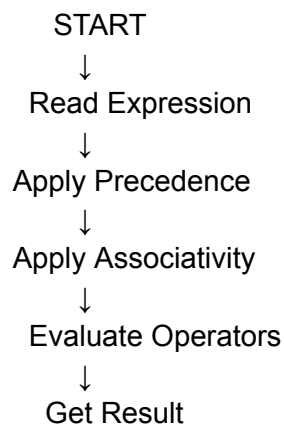
Assignment operators are right-associative:

$a = b = 10;$

is grouped as:

$a = (b = 10);$

3.36 Expression Evaluation Flowchart



↓
END

3.37 Example: Arithmetic Expression

`int result = 10 + 20 * 3;`

Step 1:

$20 * 3 = 60$

Step 2:

$10 + 60 = 70$

Final result:

70

3.38 Example: Parentheses

`int result = (10 + 20) * 3;`

Step 1:

$10 + 20 = 30$

Step 2:

$30 * 3 = 90$

Final result:

90

3.39 Short-Circuit Evaluation

Logical operators `&&` and `||` use short-circuit evaluation.

AND

If the left operand of `&&` evaluates to false, the right operand is not evaluated.

```
if (x != 0 && y / x > 2)
{
    ...
}
```

If $x \neq 0$ is false, the division is not evaluated.

OR

If the left operand of $||$ evaluates to true, the right operand is not evaluated.

```
if (x == 0 || y / x > 2)
{
    ...
}
```

This can be useful for writing safe conditional expressions.

3.40 Practical Program: Even or Odd

```
#include <stdio.h>
```

```
int main()
{
    int number;

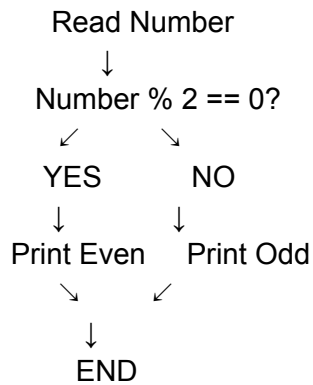
    printf("Enter a number: ");
    scanf("%d", &number);

    if (number % 2 == 0)
    {
        printf("Even");
    }
    else
    {
        printf("Odd");
    }

    return 0;
}
```

Flowchart

START
↓



3.41 Practical Program: Largest of Two Numbers

```
#include <stdio.h>

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (a > b)
    {
        printf("%d is larger", a);
    }
    else if (b > a)
    {
        printf("%d is larger", b);
    }
    else
    {
        printf("Both are equal");
    }

    return 0;
}
```

3.42 Common Mistakes

Mistake 1: Using `=` instead of `==`

Incorrect:

```
if (age = 18)
```

Correct:

```
if (age == 18)
```

Mistake 2: Unexpected Integer Division

```
int x = 5;
```

```
int y = 2;
```

```
float result = x / y;
```

Result:

2.0

Use:

```
float result = (float)x / y;
```

to obtain:

2.5

Mistake 3: Confusing `&` and `&&`

`&` → Bitwise AND

`&&` → Logical AND

Mistake 4: Confusing `|` and `||`

`|` → Bitwise OR

`||` → Logical OR

3.43 Best Practices

✓ Use parentheses when expressions are complex.

✓ Don't rely on confusing operator combinations.

- ✓ Understand the difference between assignment and comparison.
 - ✓ Use logical operators carefully.
 - ✓ Be careful with signed bitwise operations.
 - ✓ Avoid unnecessarily complicated expressions.
-

3.44 Interview Questions

1. What is an operator?

An operator is a symbol that instructs the compiler to perform an operation on one or more operands.

2. What is the difference between `=` and `==`?

`=` performs assignment, while `==` tests equality.

3. What is the modulus operator?

`%` produces the remainder of integer division.

4. What is the difference between `++x` and `x++`?

`++x` increments before the value is used in the surrounding expression; `x++` uses the old value first and then increments.

5. What is operator precedence?

Operator precedence determines which operators bind more strongly when an expression contains multiple operators.

6. What is short-circuit evaluation?

It is the behavior where `&&` or `||` may skip evaluating the right operand when the result is already determined by the left operand.

7. What are bitwise operators?

Bitwise operators manipulate the individual bits of integer operands.

3.45 Practical Lab

Task 1

Write a program to perform addition, subtraction, multiplication, division, and modulus.

Task 2

Write a program to check whether a number is positive, negative, or zero.

Task 3

Write a program to find the largest of three numbers.

Task 4

Write a program to check whether a student is eligible based on age and marks using `&&`.

Task 5

Write a program demonstrating prefix and postfix increment.

Task 6

Write a program demonstrating `&`, `|`, and `^`.

Task 7

Write a program using the conditional operator to find the larger of two numbers.

3.46 Module Summary

In this module, you learned:

- Operators
- Operands
- Arithmetic operators
- Relational operators
- Equality operators
- Logical operators
- Assignment operators
- Increment and decrement
- Prefix and postfix

- Bitwise operators
 - Conditional operator
 - `sizeof`
 - Comma operator
 - Operator precedence
 - Associativity
 - Short-circuit evaluation
 - Expressions
 - Practical programs
 - Common mistakes
 - Best practices
 - Interview questions
-

Professional Diagrams for Your PDF

Diagram 1 — Operator Classification

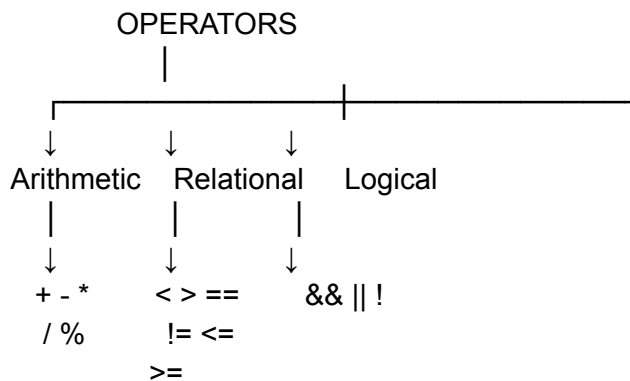


Diagram 2 — Expression Evaluation

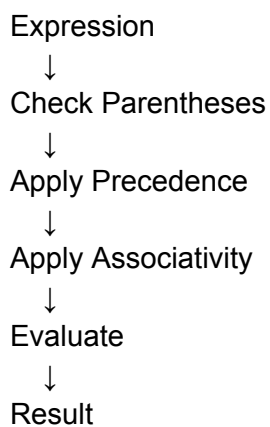


Diagram 3 — Decision Using Operators



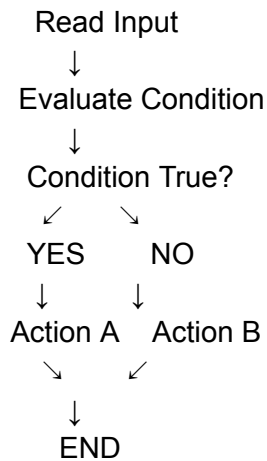


Diagram 4 — Prefix vs Postfix

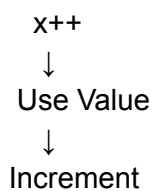
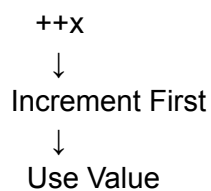
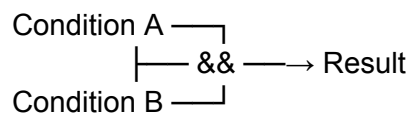


Diagram 5 — Logical AND



Both TRUE → TRUE
 Otherwise → FALSE

Module 4: Input, Output & Decision-Making Statements

4.1 Learning Objectives

After completing this module, you will be able to:

- Understand standard input and output.

- Use `printf()` effectively.
 - Use `scanf()` to accept user input.
 - Understand format specifiers.
 - Understand conditional statements.
 - Use `if`, `if-else`, and `else-if`.
 - Implement nested conditions.
 - Understand `switch`.
 - Use `break` and `default`.
 - Build decision-making flowcharts.
 - Solve practical problems using conditions.
-

4.2 Introduction

A typical program follows this basic pattern:

Input → Processing → Output

But many real-world programs need to **make decisions**.

For example:

- If marks are above 40 → Pass.
- If age is 18 or above → Eligible.
- If password is correct → Login successful.
- If balance is sufficient → Allow withdrawal.

C provides **decision-making statements** to implement such logic.

4.3 Standard Input and Output

C provides standard I/O facilities through:

```
#include <stdio.h>
```

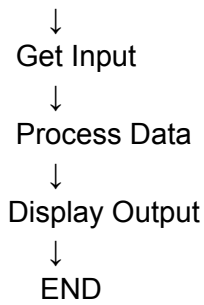
Two commonly used functions are:

`printf()` → Output

`scanf()` → Input

Basic flow:

START



4.4 printf()

`printf()` is used to write formatted output to the standard output stream.

Syntax

```
printf("format string", arguments);
```

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    printf("Hello World");

    return 0;
}
```

Output:

Hello World

4.5 Printing Variables

```
#include <stdio.h>
```

```
int main(void)
{
    int age = 20;

    printf("Age = %d", age);

    return 0;
}
```

Output:

Age = 20

Here:

%d → placeholder for int

age → value supplied to printf()

4.6 Common `printf()` Format Specifiers

Data Type	Common Specifier
<code>int</code>	<code>%d</code>
<code>unsigned int</code>	<code>%u</code>
<code>float</code>	<code>%f</code>
<code>double</code>	<code>%f</code>
<code>char</code>	<code>%c</code>
String	<code>%s</code>
Pointer address	<code>%p</code>

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int age = 21;
```

```
    float percentage = 85.5f;
```

```
    char grade = 'A';
```

```
    printf("Age: %d\n", age);
```

```
    printf("Percentage: %.2f\n", percentage);
```

```
    printf("Grade: %c\n", grade);
```

```
    return 0;  
}
```

Output:

Age: 21
Percentage: 85.50
Grade: A

4.7 Escape Sequences

Escape sequences represent special characters in string literals.

Escape Sequence	Meaning
<code>\n</code>	New line
<code>\t</code>	Horizontal tab
<code>\\</code>	Backslash
<code>\"</code>	Double quote
<code>\'</code>	Single quote
<code>\0</code>	Null character

Example

```
printf("Hello\nWorld");
```

Output:

Hello
World

Tab

```
printf("Name\tAge");
```

Output:

Name Age

4.8 scanf()

`scanf( )` reads formatted input from the standard input stream.

Syntax

```
scanf("format", &variable);
```

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int age;
```

```
    printf("Enter your age: ");
```

```
    scanf("%d", &age);
```

```
    printf("Your age is %d", age);
```

```
    return 0;
```

```
}
```

If the user enters:

20

Output:

Your age is 20

4.9 Why Do We Use & with scanf()?

For ordinary scalar variables, `scanf( )` needs the **address** where it should store the input.

Example:

```
int age;
```

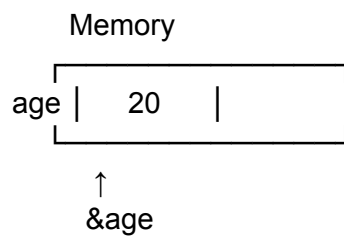
```
scanf("%d", &age);
```

Here:

age → value

&age → address of age

Conceptually:



Pointers and addresses will be studied in detail later.

4.10 Reading Different Data Types

Integer

```
int age;
```

```
scanf("%d", &age);
```

Float

```
float marks;
```

```
scanf("%f", &marks);
```

Double

```
double salary;
```

```
scanf("%lf", &salary);
```

Character

```
char grade;
```

```
scanf(" %c", &grade);
```

The space before `%c` helps skip leading whitespace such as a leftover newline.

4.11 Reading Multiple Values

You can read multiple values in one `scanf()` call.

```
int a, b;
```

```
scanf("%d %d", &a, &b);
```

Input:

10 20

Now:

a = 10

b = 20

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int a, b;
```

```
    printf("Enter two numbers: ");
```

```
    scanf("%d %d", &a, &b);
```

```
    printf("Sum = %d", a + b);
```

```
    return 0;
```

```
}
```

4.12 Input → Processing → Output

Consider:

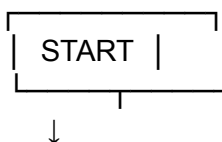
```
int a, b, sum;
```

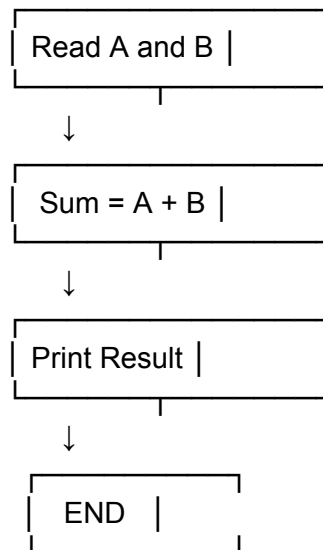
```
scanf("%d %d", &a, &b);
```

```
sum = a + b;
```

```
printf("%d", sum);
```

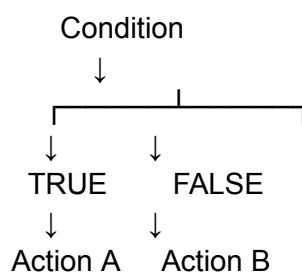
Flowchart:





4.13 Decision-Making Statements

Decision-making allows a program to choose between different paths.



Main decision statements in C include:

- `if`
 - `if-else`
 - `else-if` ladder
 - Nested `if`
 - `switch`
-

4.14 `if` Statement

The `if` statement executes a block when a condition is true.

Syntax

```
if (condition)
{
    statements;
}
```

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    int age = 20;

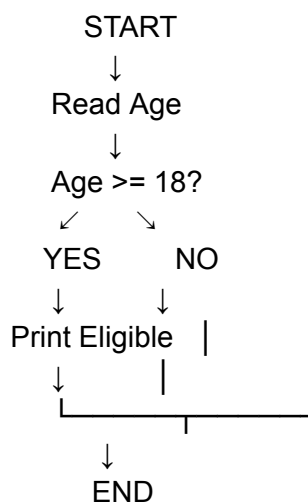
    if (age >= 18)
    {
        printf("Eligible");
    }

    return 0;
}
```

Output:

Eligible

4.15 **if** Flowchart



If the condition is false, the **if** body is skipped.

4.16 if-else

`if-else` provides two alternative paths.

Syntax

```
if (condition)
{
    statements;
}
else
{
    statements;
}
```

Example:

```
#include <stdio.h>

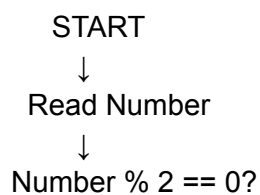
int main(void)
{
    int number;

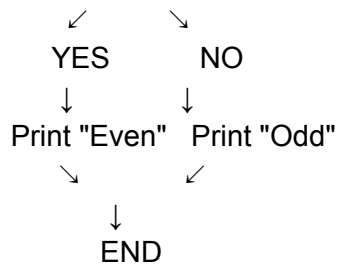
    printf("Enter a number: ");
    scanf("%d", &number);

    if (number % 2 == 0)
    {
        printf("Even");
    }
    else
    {
        printf("Odd");
    }

    return 0;
}
```

4.17 if-else Flowchart





4.18 **else-if** Ladder

An **else-if** ladder is useful when there are multiple mutually exclusive conditions.

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int marks;
```

```
    printf("Enter marks: ");
```

```
    scanf("%d", &marks);
```

```
    if (marks >= 90)
```

```
    {
```

```
        printf("Grade A");
```

```
    }
```

```
    else if (marks >= 75)
```

```
    {
```

```
        printf("Grade B");
```

```
    }
```

```
    else if (marks >= 60)
```

```
    {
```

```
        printf("Grade C");
```

```
    }
```

```
    else if (marks >= 40)
```

```
    {
```

```
        printf("Grade D");
```

```
    }
```

```
    else
```

```
    {
```

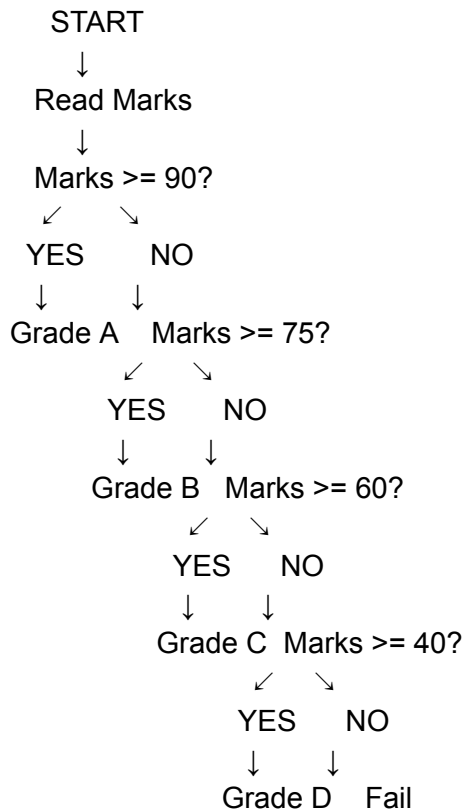
```
        printf("Fail");
```

```
    }
```

```
    return 0;
```

}

4.19 Else-If Flowchart



4.20 Nested **if**

An **if** statement inside another **if** statement is called a **nested if**.

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int age;
```

```
    int has_id;
```

```
    printf("Enter age: ");
```

```
    scanf("%d", &age);
```

```
    printf("Do you have an ID? (1 = Yes, 0 = No): ");
```



```

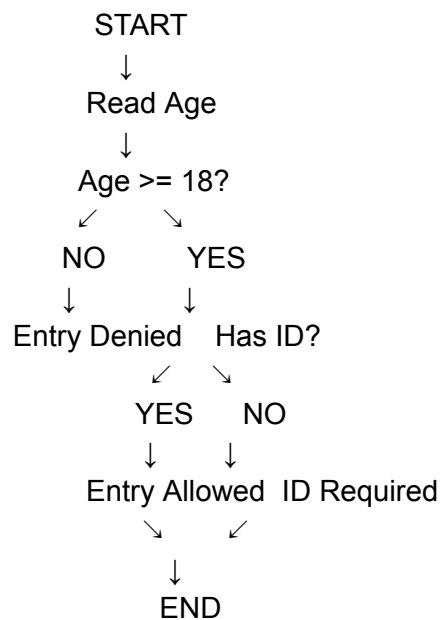
scanf("%d", &has_id);

if (age >= 18)
{
    if (has_id)
    {
        printf("Entry allowed");
    }
    else
    {
        printf("ID required");
    }
}
else
{
    printf("Entry not allowed");
}

return 0;
}

```

4.21 Nested **if** Flowchart



4.22 **switch** Statement

switch is useful when one expression needs to be compared against several constant case values.

Syntax

```
switch (expression)
{
    case constant1:
        statements;
        break;

    case constant2:
        statements;
        break;

    default:
        statements;
}
```

4.23 Example of **switch**

```
#include <stdio.h>

int main(void)
{
    int choice;

    printf("Enter choice (1-3): ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            printf("Add");
            break;

        case 2:
            printf("Update");
            break;

        case 3:
            printf("Delete");
            break;

        default:
            printf("Invalid choice");
    }
}
```

```
}  
  
return 0;  
}
```

4.24 Understanding **break**

break terminates the nearest enclosing **switch** or loop.

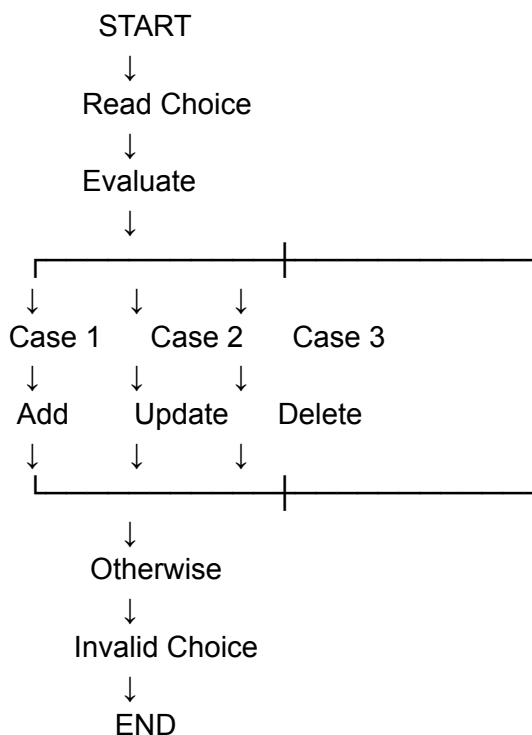
In a **switch**, it prevents execution from continuing into the next case.

Example:

```
case 1:  
    printf("One");  
    break;
```

Without **break**, execution can continue into the following case statements. This behavior is called **fall-through**.

4.25 **switch** Flowchart



4.26 `if-else` vs `switch`

Feature	<code>if-else</code>	<code>switch</code>
Range conditions	Excellent	Not directly
Multiple constant choices	Possible	Excellent
Complex logical conditions	Yes	No
Readability for menus	Moderate	Often better
Relational expressions	Yes	No
<code>break</code> required	No	Usually used to prevent fall-through

Example:

Use `if`

```
if (marks >= 40)
```

Use `switch`

```
switch (choice)
{
    case 1:
        ...
}
```

4.27 Nested `switch`

C also permits a `switch` inside another control statement, including another `switch`.

Example:

```
switch (department)
{
    case 1:
        switch (year)
        {
            case 1:
                printf("First Year");
                break;
        }
        break;
```

```
}
```

Use nested structures carefully because excessive nesting can reduce readability.

4.28 Decision-Making Example: Positive, Negative, or Zero

```
#include <stdio.h>
```

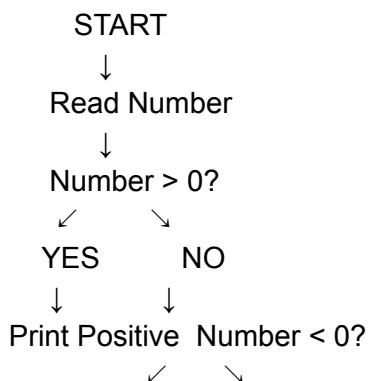
```
int main(void)
{
    int number;

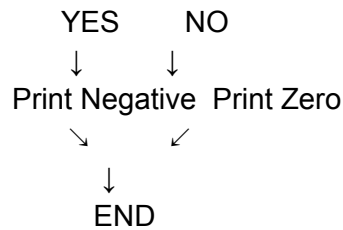
    printf("Enter number: ");
    scanf("%d", &number);

    if (number > 0)
    {
        printf("Positive");
    }
    else if (number < 0)
    {
        printf("Negative");
    }
    else
    {
        printf("Zero");
    }

    return 0;
}
```

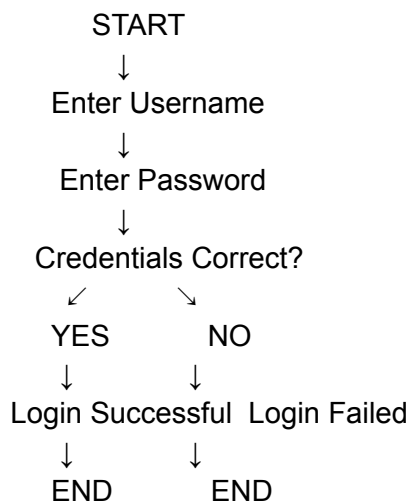
Flowchart





4.29 Real-World Example: Login System

Consider a simple conceptual login system.



C implementation:

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char username[20];
    char password[20];

    printf("Username: ");
    scanf("%19s", username);

    printf("Password: ");
    scanf("%19s", password);

    if (strcmp(username, "admin") == 0 &&
        strcmp(password, "1234") == 0)
    {
        printf("Login successful");
    }
}
```

```
    else
    {
        printf("Invalid credentials");
    }

    return 0;
}
```

Note: This is only a learning example. Real authentication systems should never store passwords in plain text.

4.30 **switch** Example: Simple Calculator

```
#include <stdio.h>

int main(void)
{
    char operator;
    double a, b;

    printf("Enter expression (e.g. 10 + 5): ");
    scanf("%lf %c %lf", &a, &operator, &b);

    switch (operator)
    {
        case '+':
            printf("Result = %.2f", a + b);
            break;

        case '-':
            printf("Result = %.2f", a - b);
            break;

        case '*':
            printf("Result = %.2f", a * b);
            break;

        case '/':
            if (b != 0)
                printf("Result = %.2f", a / b);
            else
                printf("Division by zero is not allowed");
            break;
    }
}
```

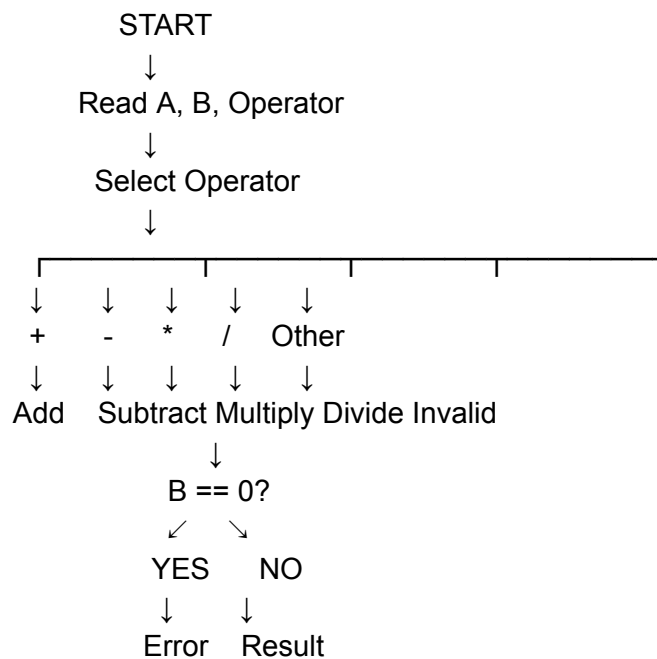
```

        default:
            printf("Invalid operator");
    }

    return 0;
}

```

4.31 Flowchart: Calculator



4.32 Common Mistakes

Mistake 1: Using `=` instead of `==`

Incorrect:

```
if (age = 18)
```

Correct:

```
if (age == 18)
```

Mistake 2: Forgetting `break`


```
case 1:
    printf("One");
```

```
case 2:
    printf("Two");
```

This may cause fall-through.

Better:

```
case 1:
    printf("One");
    break;
```

Mistake 3: Missing Braces

For clarity, use braces:

```
if (condition)
{
    statement;
}
```

Mistake 4: Incorrect `scanf()`

Incorrect:

```
int age;
scanf("%d", age);
```

Correct:

```
scanf("%d", &age);
```

Module 5: Loops & Iteration

5.1 Learning Objectives

After completing this module, you will be able to:

- Understand the concept of iteration.
- Understand `for`, `while`, and `do-while` loops.
- Compare different types of loops.

- Use nested loops.
 - Use `break` and `continue`.
 - Understand infinite loops.
 - Create loop-based flowcharts.
 - Solve counting, summation, factorial, and pattern problems.
 - Apply loops to real-world programming problems.
-

5.2 What is a Loop?

A **loop** repeatedly executes a block of statements while a specified condition remains true or until a termination condition is reached.

Instead of writing:

```
printf("1");  
printf("2");  
printf("3");  
printf("4");  
printf("5");
```

we can use a loop:

```
for (int i = 1; i <= 5; i++)  
{  
    printf("%d\n", i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

5.3 Why Do We Need Loops?

Loops are useful when the same operation must be performed repeatedly.

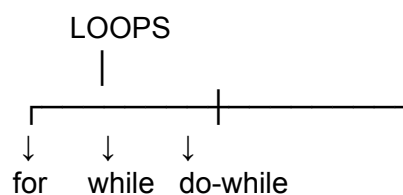
Examples:

- Printing numbers.
- Reading multiple values.

- Calculating totals.
 - Processing arrays.
 - Searching data.
 - Repeating menu operations.
 - Generating patterns.
 - Performing calculations until a condition is satisfied.
-

5.4 Types of Loops in C

C provides three primary loop statements:



for

Best when the number of iterations is known or naturally controlled by initialization, condition, and update.

while

Best when repetition depends primarily on a condition and the number of iterations may not be known beforehand.

do-while

Useful when the body must execute **at least once** before the condition is tested.

5.5 for Loop

Syntax

```
for (initialization; condition; update)
{
    statements;
}
```

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    for (int i = 1; i <= 5; i++)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

Output:

```
1
2
3
4
5
```

5.6 Understanding the **for** Loop

Consider:

```
for (int i = 1; i <= 5; i++)
```

It has three parts:

initialization ; condition ; update

Initialization

```
int i = 1
```

Executed once before the loop begins.

Condition

```
i <= 5
```

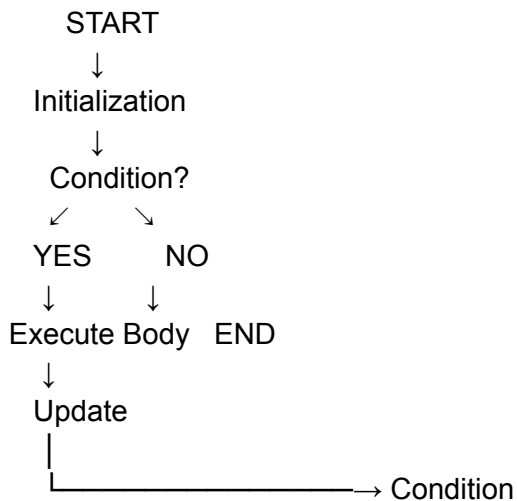
Checked before each iteration.

Update

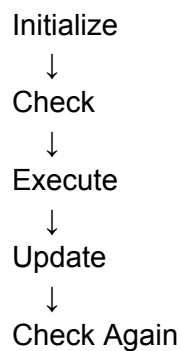
```
i++
```

Executed after each iteration of the loop body.

5.7 **for** Loop Flowchart



Easy Flow



5.8 Program: Print 1 to 10

```
#include <stdio.h>
```

```
int main(void)
{
    for (int i = 1; i <= 10; i++)
    {
        printf("%d ", i);
    }

    return 0;
}
```

Output:

1 2 3 4 5 6 7 8 9 10

5.9 Program: Print Even Numbers

```
#include <stdio.h>
```

```
int main(void)
{
    for (int i = 2; i <= 20; i += 2)
    {
        printf("%d ", i);
    }

    return 0;
}
```

Output:

2 4 6 8 10 12 14 16 18 20

5.10 **while** Loop

The **while** loop checks its condition **before** executing the body.

Syntax

```
while (condition)
{
    statements;
}
```

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    int i = 1;

    while (i <= 5)
    {
```

```

        printf("%d\n", i);
        i++;
    }

    return 0;
}

```

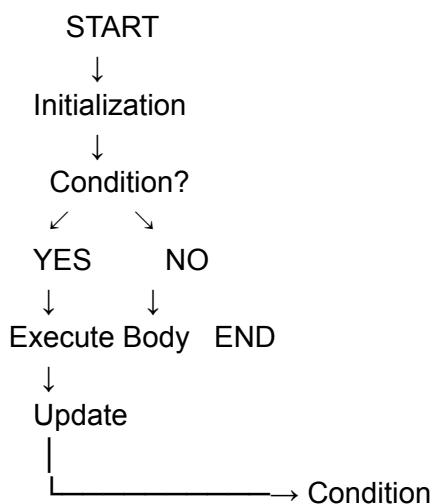
Output:

```

1
2
3
4
5

```

5.11 **while** Loop Flowchart



The important point is that the condition is checked **before** the body executes.

5.12 **while** vs **for**

Both can implement many of the same algorithms.

for

```

for (int i = 1; i <= 5; i++)
{
    printf("%d ", i);
}

```

```
}
```

while

```
int i = 1;
```

```
while (i <= 5)
{
    printf("%d ", i);
    i++;
}
```

The choice is usually based on readability and the nature of the loop.

5.13 do-while Loop

A **do-while** loop executes its body **before** checking the condition.

Syntax

```
do
{
    statements;
}
while (condition);
```

Notice the semicolon after the **while** condition.

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    int i = 1;

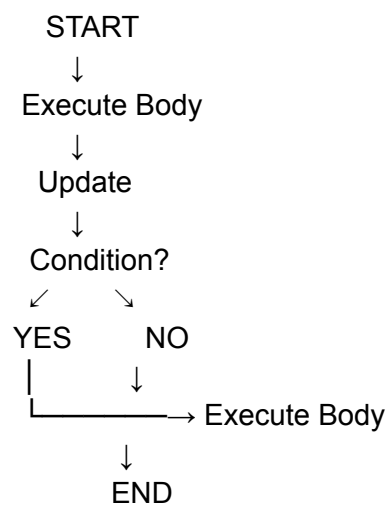
    do
    {
        printf("%d\n", i);
        i++;
    }
    while (i <= 5);

    return 0;
}
```

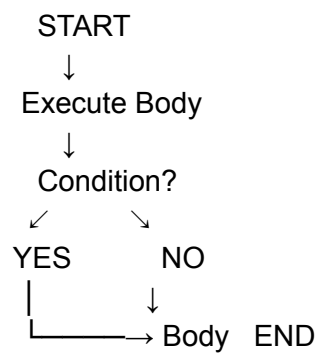

Output:

1
2
3
4
5

5.14 do-while Flowchart



A clearer representation is:



The key concept:

The body executes at least once.

5.15 Difference Between **while** and **do-while**

Consider:

```
int x = 10;
```

while

```
while (x < 5)
{
    printf("Hello");
}
```

Output:

Nothing

Because the condition is false initially.

do-while

```
do
{
    printf("Hello");
}
while (x < 5);
```

Output:

Hello

Because the body executes before the condition is tested.

5.16 Loop Comparison

Feature	for	while	do-while
Condition checked	Before body	Before body	After body
Guaranteed first execution	No	No	Yes

Common use	Count-controlled loops	Condition-controlled loops	At-least-once operations
Initialization syntax	Inside loop header	Usually before loop	Usually before loop

5.17 Nested Loops

A loop inside another loop is called a **nested loop**.

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    for (int i = 1; i <= 3; i++)
    {
        for (int j = 1; j <= 3; j++)
        {
            printf("%d %d\n", i, j);
        }
    }

    return 0;
}
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

5.18 Nested Loop Concept

Suppose:

Outer loop → 3 iterations

Inner loop → 3 iterations

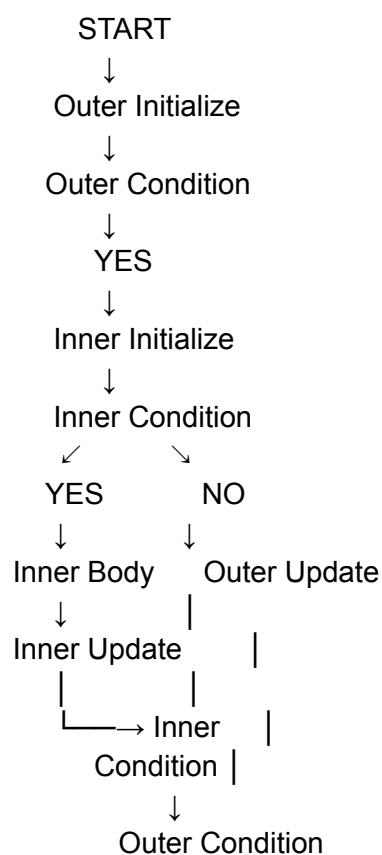
The inner loop executes completely for each outer-loop iteration.

Therefore:

$$3 \times 3 = 9$$

inner-loop executions.

5.19 Nested Loop Flowchart



5.20 Pattern Printing

Nested loops are commonly used for pattern printing.

Example:

*
**

Program:

```
#include <stdio.h>
```

```
int main(void)
{
    for (int i = 1; i <= 5; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            printf("*");
        }

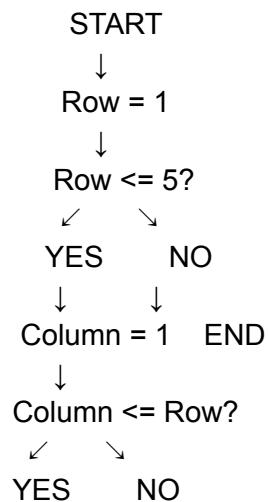
        printf("\n");
    }

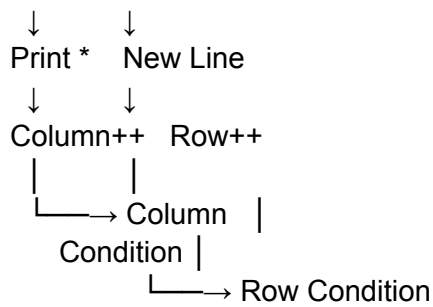
    return 0;
}
```

5.21 Pattern Flow

Row 1 → *
Row 2 → **
Row 3 → ***
Row 4 → ****
Row 5 → *****

Flowchart:





5.22 break Statement

break immediately terminates the **nearest enclosing loop or switch**.

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    for (int i = 1; i <= 10; i++)
    {
        if (i == 5)
        {
            break;
        }

        printf("%d ", i);
    }

    return 0;
}
```

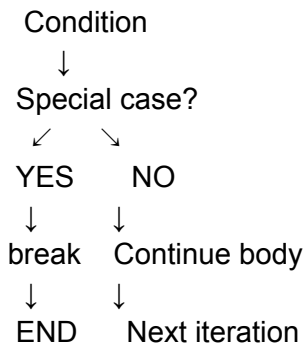
Output:

1 2 3 4

When **i** becomes 5, the loop terminates.

5.23 break Flowchart

Loop
↓



5.24 `continue` Statement

`continue` skips the remaining statements in the current iteration and proceeds to the next iteration of the nearest enclosing loop.

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    for (int i = 1; i <= 5; i++)
    {
        if (i == 3)
        {
            continue;
        }

        printf("%d ", i);
    }

    return 0;
}
```

Output:

1 2 4 5

When `i == 3`, printing is skipped for that iteration.

5.25 `break` vs `continue`

break

Terminates the nearest loop

Control moves outside the loop

Stops further iterations

continue

Skips current iteration

Control moves toward the next iteration

Loop continues

Remember:

`break` → Exit loop

`continue` → Skip iteration

5.26 Infinite Loop

An infinite loop continues indefinitely because its termination condition never becomes false or there is no termination path.

Example:

```
while (1)
{
    printf("Running...\n");
}
```

Another example:

```
for (;;)
{
    printf("Running...\n");
}
```

Infinite loops can be intentional in systems such as servers or embedded programs, but accidental infinite loops are usually bugs.

5.27 How to Avoid Accidental Infinite Loops

Always ensure that the loop has a valid termination path.

Example:


```
int i = 1;

while (i <= 5)
{
    printf("%d ", i);
    i++;
}
```

If you forget:

```
i++;
```

then `i` may remain 1 forever.

5.28 Sum of First N Numbers

Problem:

Calculate the sum of numbers from 1 to `N`.

Example:

`N = 5`

Calculation:

$1 + 2 + 3 + 4 + 5 = 15$

Program:

```
#include <stdio.h>
```

```
int main(void)
{
    int n;
    int sum = 0;

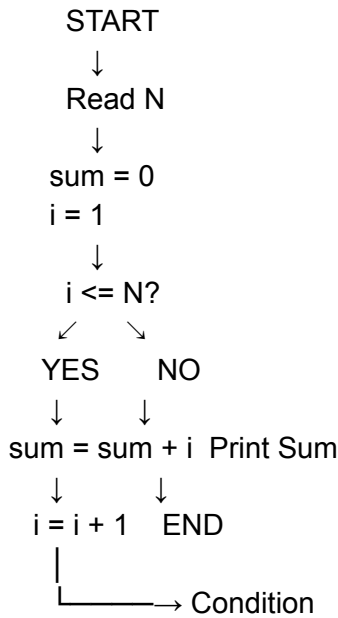
    printf("Enter N: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++)
    {
        sum += i;
    }

    printf("Sum = %d", sum);
}
```

```
    return 0;
}
```

5.29 Sum Flowchart



5.30 Factorial Using a Loop

Factorial of n is:

$$n! = n \times (n-1) \times \dots \times 1$$

Example:

$$\begin{aligned} 5! &= 5 \times 4 \times 3 \times 2 \times 1 \\ &= 120 \end{aligned}$$

Program:

```
#include <stdio.h>

int main(void)
{
    int n;
    unsigned long long factorial = 1;

    printf("Enter a non-negative integer: ");
```

```

scanf("%d", &n);

if (n < 0)
{
    printf("Factorial is not defined for negative integers.");
    return 0;
}

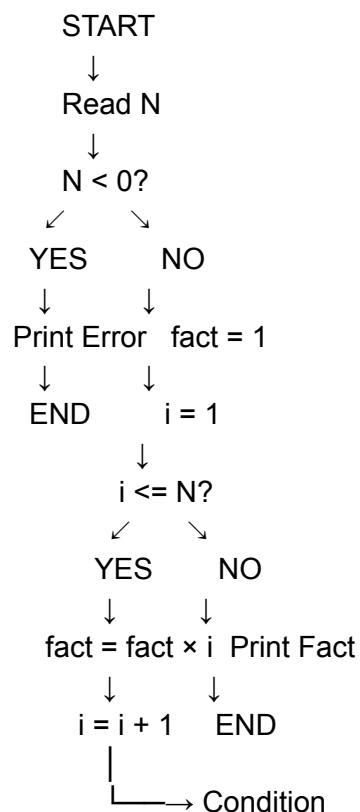
for (int i = 1; i <= n; i++)
{
    factorial *= (unsigned long long)i;
}

printf("Factorial = %llu", factorial);

return 0;
}

```

5.31 Factorial Flowchart



5.32 Reverse a Number

Example:

Input: 1234

Output: 4321

Program:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int number;
```

```
    int reverse = 0;
```

```
    printf("Enter a number: ");
```

```
    scanf("%d", &number);
```

```
    while (number != 0)
```

```
    {
```

```
        int digit = number % 10;
```

```
        reverse = reverse * 10 + digit;
```

```
        number /= 10;
```

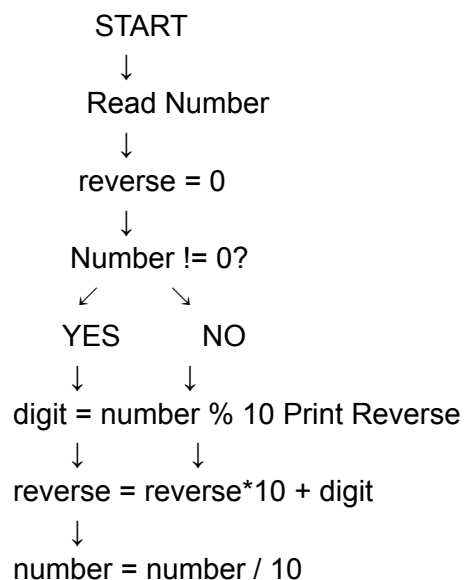
```
    }
```

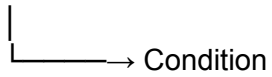
```
    printf("Reverse = %d", reverse);
```

```
    return 0;
```

```
}
```

5.33 Reverse Number Flowchart





5.34 Count Digits

Example:

Input: 12345

Output: 5

Program:

```
#include <stdio.h>

int main(void)
{
    int number;
    int count = 0;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (number == 0)
    {
        count = 1;
    }
    else
    {
        if (number < 0)
        {
            number = -number;
        }

        while (number != 0)
        {
            number /= 10;
            count++;
        }
    }

    printf("Number of digits = %d", count);

    return 0;
}
```

5.35 Multiplication Table

Program:

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    for (int i = 1; i <= 10; i++)
    {
        printf("%d × %d = %d\n", n, i, n * i);
    }

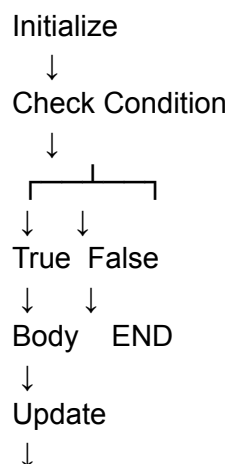
    return 0;
}
```

Example output for 5:

```
5 × 1 = 5
5 × 2 = 10
5 × 3 = 15
...
5 × 10 = 50
```

5.36 Loop Control Flow

The general loop process is:



Condition

This basic concept is essential for understanding almost every loop-based program.

5.37 Common Mistakes

Mistake 1: Forgetting the Update

Incorrect:

```
int i = 1;

while (i <= 5)
{
    printf("%d", i);
}
```

This creates an infinite loop because `i` never changes.

Correct:

```
int i = 1;

while (i <= 5)
{
    printf("%d", i);
    i++;
}
```

Mistake 2: Wrong Condition

```
for (int i = 1; i >= 5; i++)
```

The condition is false immediately.

Correct for 1 through 5:

```
for (int i = 1; i <= 5; i++)
```

Mistake 3: Extra Semicolon

Avoid accidentally writing:

```
for (int i = 0; i < 5; i++);  
{  
    printf("%d", i);  
}
```

That semicolon terminates the loop body.

Mistake 4: Confusing **break** and **continue**

break → Exit loop

continue → Skip current iteration

5.38 Best Practices

- ✓ Keep loop conditions simple.
 - ✓ Make sure loop variables progress toward termination.
 - ✓ Use **for** when initialization, condition, and update naturally belong together.
 - ✓ Use **while** when repetition is primarily condition-driven.
 - ✓ Use **do-while** when at least one execution is required.
 - ✓ Avoid unnecessarily deep nested loops.
 - ✓ Use meaningful variable names.
-

5.39 Interview Questions

1. What is a loop?

A loop repeatedly executes a block of statements according to a controlling condition.

2. What are the three loops in C?

- **for**
- **while**
- **do-while**

3. Which loop executes at least once?

`do-while`.

4. What is the difference between `while` and `do-while`?

`while` checks the condition before executing the body, while `do-while` executes the body first and checks the condition afterward.

5. What is a nested loop?

A loop inside another loop is called a nested loop.

6. What does `break` do?

It terminates the nearest enclosing loop or switch.

7. What does `continue` do?

It skips the remaining statements in the current iteration and proceeds toward the next iteration of the nearest enclosing loop.

8. What is an infinite loop?

An infinite loop is a loop that does not reach its termination condition or has no termination path.

5.40 Practical Lab

Task 1

Print numbers from 1 to 100.

Task 2

Print all even numbers from 1 to 100.

Task 3

Print all odd numbers from 1 to 100.

Task 4

Calculate the sum of numbers from 1 to N.

Task 5

Calculate the factorial of a number.

Task 6

Print the multiplication table of a number.

Task 7

Reverse a number.

Task 8

Count the digits of a number.

Task 9

Check whether a number is a palindrome.

Task 10

Print this pattern:

```
*  
**  
***  
****  
*****
```

Task 11

Print:

```
1  
12  
123  
1234  
12345
```

Task 12

Create a menu-driven program using a `do-while` loop and `switch`.

Diagram 1 — `for` Loop

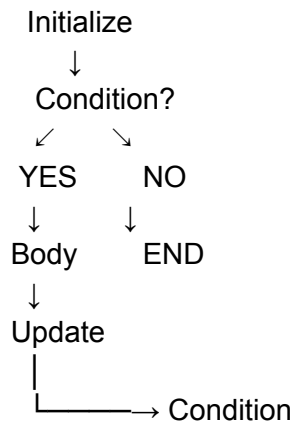


Diagram 2 — while Loop

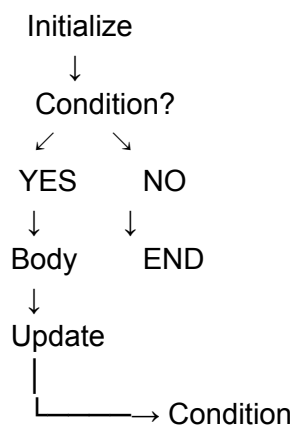


Diagram 3 — do-while Loop

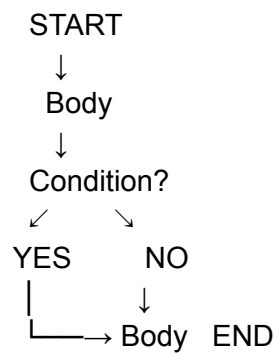


Diagram 4 — break vs continue

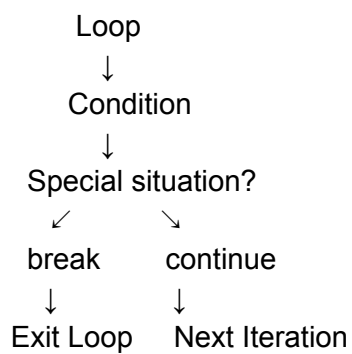
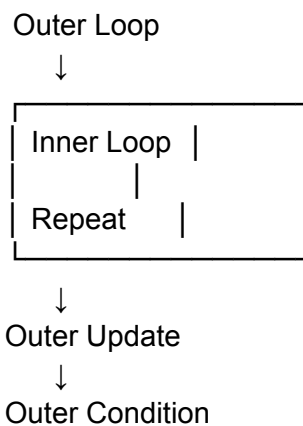


Diagram 5 — Nested Loop



Module 6: Functions

Functions are one of the most important concepts in C because they help us divide a large program into **small, reusable, and manageable blocks**.

6.1 Learning Objectives

After completing this module, you will be able to:

- Understand what a function is.
 - Understand why functions are required.
 - Learn function declaration, definition, and calling.
 - Understand parameters and arguments.
 - Understand return values.
 - Differentiate library and user-defined functions.
 - Understand functions with and without parameters.
 - Understand functions with and without return values.
 - Learn call by value.
 - Understand recursion.
 - Understand local and global scope basics.
 - Design programs using modular functions.
-

6.2 What is a Function?

A **function** is a named block of C code designed to perform a specific task.

Instead of writing an entire program inside `main()`, we can divide it into smaller functions.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Here, `add()` performs one specific task:

Adding two numbers.

6.3 Why Do We Need Functions?

Imagine a program containing 1,000 lines of code.

Without functions:

```
main()
|
|— 100 lines
|— 200 lines
|— 300 lines
|— 400 lines
|— ...
```

This becomes difficult to understand and maintain.

With functions:

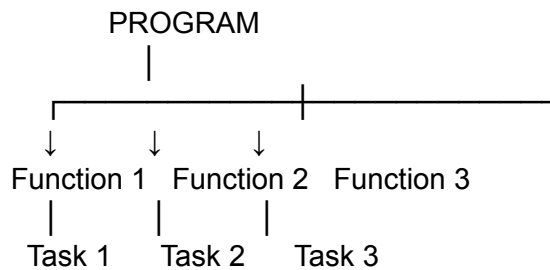
```
main()
|
|— login()
|— calculate()
|— display()
|— logout()
```

This makes the program:

- Easier to understand.
- Easier to test.
- Easier to debug.
- Easier to maintain.
- More reusable.

6.4 Function Architecture

A typical function-based program looks like:



6.5 Function Components

A function generally involves:

1. Declaration / Prototype
2. Definition
3. Function Call

Example:

```
#include <stdio.h>
```

```
int add(int, int);    // Declaration
```

```
int main(void)
```

```
{
```

```
    int result;
```

```
    result = add(10, 20); // Function Call
```

```
    printf("%d", result);
```

```
    return 0;
```

```
}
```

```
int add(int a, int b)    // Definition
```

```
{
```

```
    return a + b;
```

```
}
```

6.6 Function Declaration

A function declaration, commonly called a **prototype**, tells the compiler about a function before it is used.

Syntax:

```
return_type function_name(parameter_types);
```

Example:

```
int add(int, int);
```

This tells the compiler:

- Function name → `add`
- Return type → `int`
- Two parameters → both `int`

You can also name the parameters:

```
int add(int a, int b);
```

6.7 Function Definition

The function definition contains the actual implementation.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Structure:

```
Return Type
  ↓
Function Name
  ↓
Parameters
  ↓
Function Body
```

6.8 Function Call

A function is executed when it is called.

Example:

```
int result = add(10, 20);
```

Here:

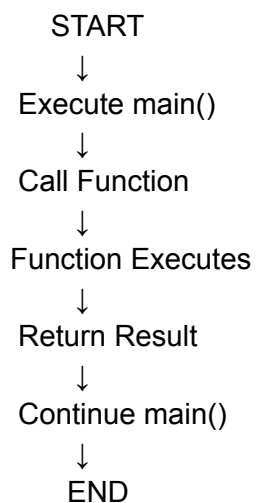
```
add(10, 20)
```

is the function call.

Execution flow:

```
main()
↓
add()
↓
return result
↓
main()
```

6.9 Function Execution Flowchart



6.10 Example: Simple Function

```
#include <stdio.h>
```



```
void greet(void)
{
    printf("Welcome to C Programming!");
}
```

```
int main(void)
{
    greet();

    return 0;
}
```

Output:

Welcome to C Programming!

6.11 Function With No Parameters and No Return Value

Example:

```
#include <stdio.h>
```

```
void display(void)
{
    printf("Hello");
}
```

```
int main(void)
{
    display();

    return 0;
}
```

Here:

Parameter → None

Return Value → None

6.12 Function With Parameters

A function can receive values from the caller.

Example:

```
#include <stdio.h>
```

```
void displayNumber(int n)
{
    printf("Number = %d", n);
}
```

```
int main(void)
{
    displayNumber(25);

    return 0;
}
```

Here:

25 → Argument

n → Parameter

6.13 Parameter vs Argument

This is an important interview concept.

Parameter

A variable specified in the function definition.

```
void display(int n)
```

n is a parameter.

Argument

The actual value passed during the function call.

```
display(25);
```

25 is an argument.

Remember:

Parameter → Function receives

Argument → Function call provides

6.14 Function With Return Value

A function can calculate something and return the result.

Example:

```
#include <stdio.h>
```

```
int square(int n)
{
    return n * n;
}
```

```
int main(void)
{
    int result;

    result = square(5);

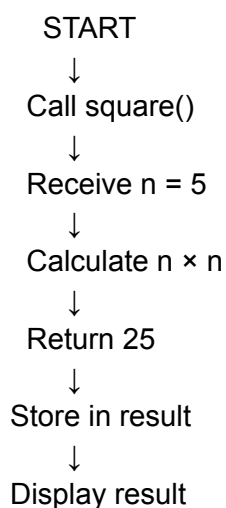
    printf("Square = %d", result);

    return 0;
}
```

Output:

Square = 25

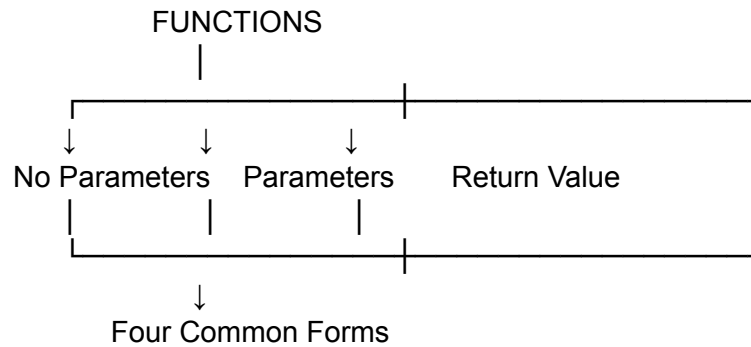
6.15 Function Return Flowchart



↓
END

6.16 Four Basic Function Categories

Functions can be classified based on parameters and return values.



Type 1

No parameters + No return value

```
void display(void);
```

Type 2

Parameters + No return value

```
void display(int n);
```

Type 3

No parameters + Return value

```
int getNumber(void);
```

Type 4

Parameters + Return value

```
int add(int a, int b);
```

6.17 Function Category Table

Type	Parameters	Return Value	Example
1	No	No	<code>void display(void)</code>
2	Yes	No	<code>void display(int n)</code>
3	No	Yes	<code>int getNumber(void)</code>
4	Yes	Yes	<code>int add(int a, int b)</code>

The fourth form is especially common because it allows a function to receive input and produce a result.

6.18 Library Functions

C provides many functions through its standard library.

Examples:

```
printf()
scanf()
strlen()
strcmp()
malloc()
free()
```

They are provided through appropriate header files.

Example:

```
#include <stdio.h>
```

for standard I/O functions.

And:

```
#include <string.h>
```

for string functions such as `strlen()` and `strcmp()`.

6.19 User-Defined Functions

Functions created by the programmer are called **user-defined functions**.

Example:

```
int multiply(int a, int b)
{
    return a * b;
}
```

This function is created by the programmer to perform multiplication.

6.20 Library vs User-Defined Functions

Library Function	User-Defined Function
Provided by libraries	Created by programmer
Examples: <code>printf()</code> , <code>strlen()</code>	Examples: <code>add()</code> , <code>calculate()</code>
Declared through headers	Programmer provides declaration/definition
Reusable standard functionality	Application-specific functionality

6.21 Function Prototype

A prototype is useful when the function definition appears after `main()`.

Example:

```
#include <stdio.h>

int multiply(int, int);

int main(void)
{
    printf("%d", multiply(5, 4));

    return 0;
}
```

```
}

int multiply(int a, int b)
{
    return a * b;
}
```

Without a suitable declaration visible before the call, modern C compilation should not rely on an implicit function declaration.

6.22 Function Scope and Local Variables

A variable declared inside a function or block normally has **block scope**.

Example:

```
void test(void)
{
    int x = 10;

    printf("%d", x);
}
```

`x` is accessible within its scope.

It cannot be directly accessed from another function.

6.23 Global Variables

A variable declared outside all functions has **file scope**.

Example:

```
#include <stdio.h>

int count = 10;

void display(void)
{
    printf("%d", count);
}
```

```
int main(void)
{
    display();

    return 0;
}
```

Here `count` has file scope and can be accessed by functions in the same source file according to its linkage and declarations.

Best Practice

Use global variables carefully. Excessive global state can make programs harder to understand, test, and maintain.

6.24 Local vs Global Variables

Local	Global
Declared inside a block/function	Declared outside functions
Limited scope	File scope
Usually easier to reason about	Can create shared state
Preferred for most temporary data	Use when genuinely appropriate

6.25 Call by Value

C uses **pass-by-value** semantics.

When an argument is passed to a function, the function receives its own parameter initialized from that value.

Example:

```
#include <stdio.h>
```

```
void change(int x)
{
    x = 100;
}
```



```
int main(void)
{
    int a = 10;

    change(a);

    printf("%d", a);

    return 0;
}
```

Output:

10

Why?

```
main()
a = 10
↓
change(a)
↓
x gets a copy of 10
↓
x = 100
↓
a remains 10
```

6.26 Call by Value Flowchart

```
main()
↓
a = 10
↓
call change(a)
↓
x receives copy
↓
x = 100
↓
return
↓
a is still 10
```

6.27 Modifying Caller Data Using Pointers

Although C is pass-by-value, you can pass a pointer value to a function so the function can modify the object pointed to.

Example:

```
#include <stdio.h>
```

```
void change(int *x)
{
    *x = 100;
}
```

```
int main(void)
{
    int a = 10;

    change(&a);

    printf("%d", a);

    return 0;
}
```

Output:

100

Here:

&a → address of a

x → receives that address

*x → accesses a

Pointers will be studied in depth in the **Pointers module**.

6.28 Recursion

Recursion occurs when a function calls itself.

Example:

```
void display(int n)
{
    if (n > 0)
    {
        printf("%d ", n);
        display(n - 1);
    }
}
```

Call:

```
display(5);
```

Output:

```
5 4 3 2 1
```

6.29 Components of Recursion

Every recursive solution should have:

1. Base Case

Stops recursion.

```
if (n == 0)
```

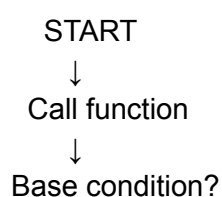
2. Recursive Case

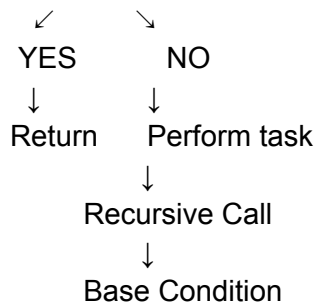
Calls the function again with a smaller or otherwise progressing problem.

```
display(n - 1);
```

Without an appropriate base case or progress toward it, recursion can continue indefinitely and eventually exhaust the call stack.

6.30 Recursion Flowchart





6.31 Recursive Factorial

Mathematically:

$$n! = n \times (n-1)!$$

with:

$$0! = 1$$

C program:

```
#include <stdio.h>
```

```
unsigned long long factorial(unsigned int n)
{
    if (n == 0)
    {
        return 1;
    }

    return n * factorial(n - 1);
}
```

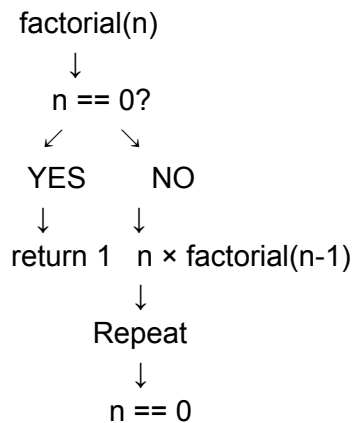
```
int main(void)
{
    printf("%llu", factorial(5));

    return 0;
}
```

Output:

120

6.32 Recursive Factorial Flowchart



6.33 Recursion vs Iteration

Recursion	Iteration
Function calls itself	Loop repeats statements
Uses call stack	Uses loop control
Can be elegant for recursive structures	Often more memory-efficient
Must have a valid base case	Must have a termination condition
Can cause stack overflow if too deep	Usually avoids recursive call-stack growth

Neither is universally better; choose based on the problem and clarity.

6.34 Function Return Statement

The `return` statement exits the current function.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Once `return` executes:

- The function terminates.
- A value is returned if the function's return type requires one.

For a `void` function:

```
void display(void)
{
    printf("Hello");
    return;
}
```

The `return;` statement can be used to exit without a value.

6.35 Multiple Return Paths

A function can have multiple `return` statements.

Example:

```
int absolute(int n)
{
    if (n < 0)
    {
        return -n;
    }

    return n;
}
```

The function returns through whichever path is reached.

6.36 Practical Program: Maximum of Two Numbers

```
#include <stdio.h>
```

```
int maximum(int a, int b)
{
    if (a > b)
```

```

    {
        return a;
    }

    return b;
}

int main(void)
{
    int result = maximum(25, 40);

    printf("Maximum = %d", result);

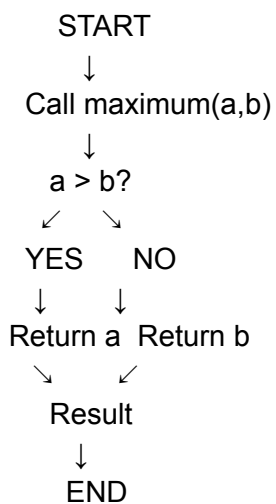
    return 0;
}

```

Output:

Maximum = 40

Flowchart



6.37 Practical Program: Check Even Number

```
#include <stdio.h>
```

```

int isEven(int n)
{
    return n % 2 == 0;
}

```

```

int main(void)
{
    int number;

    printf("Enter number: ");
    scanf("%d", &number);

    if (isEven(number))
    {
        printf("Even");
    }
    else
    {
        printf("Odd");
    }

    return 0;
}

```

This demonstrates how functions can encapsulate reusable logic.

6.38 Function-Based Program Architecture

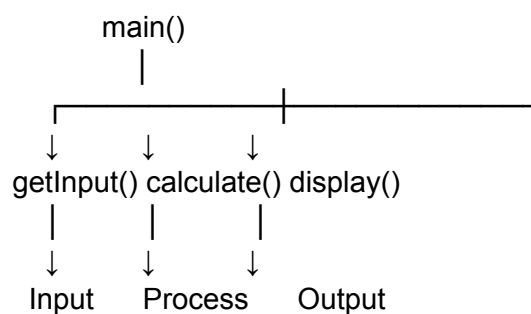
Instead of putting everything in `main()`:

```

int main(void)
{
    // 200 lines of code
}

```

we can design:



This is called **modular programming**.

6.39 Benefits of Modular Programming

Reusability

A function can be called multiple times.

Maintainability

Changes can be made in one place.

Testing

Individual functions can be tested independently.

Readability

The overall program becomes easier to understand.

Debugging

Errors can be isolated more easily.

6.40 Common Mistakes

Mistake 1: Calling a function before providing a declaration

Make sure an appropriate prototype is visible before the call.

```
int add(int, int);
```

Mistake 2: Wrong Return Type

Incorrect:

```
int getName(void)
{
    printf("Hello");
}
```

A non-`void` function should return an appropriate value along every reachable path where required.

Mistake 3: Forgetting Arguments

If the function requires parameters:

```
int add(int a, int b)
```

call it with suitable arguments:

```
add(10, 20);
```

Mistake 4: Infinite Recursion

Incorrect:

```
void test(void)
{
    test();
}
```

There is no base case or terminating condition.

6.41 Best Practices

✓ Give functions meaningful names.

```
calculateAverage()
```

is clearer than:

```
fun1()
```

✓ Keep each function focused on one responsibility.

✓ Use parameters instead of unnecessary global variables.

✓ Keep functions reasonably small and readable.

✓ Use prototypes where appropriate.

✓ Prefer clear return values.

✓ Document non-obvious behavior.

6.42 Interview Questions

1. What is a function?

A function is a named block of code designed to perform a specific task.

2. What are the three important parts of a user-defined function?

Commonly:

- Function declaration/prototype
- Function definition
- Function call

3. What is a function prototype?

A function prototype declares a function's name, return type, and parameter types before the function is used.

4. What is the difference between a parameter and an argument?

A parameter appears in the function definition; an argument is the actual value supplied during the function call.

5. Does C support call by reference?

C itself uses pass-by-value. Reference-like modification of caller objects can be achieved by passing pointers.

6. What is recursion?

Recursion is a technique where a function calls itself to solve a problem through smaller subproblems.

7. What is the difference between local and global variables?

Local variables have limited block scope, while variables declared outside functions can have file scope and potentially be accessible across multiple functions in that source file.

8. Why are functions important?

They improve modularity, readability, reuse, testing, debugging, and maintainability.

6.43 Practical Lab

Task 1

Create a function to add two numbers.

Task 2

Create a function to calculate the square of a number.

Task 3

Create a function to find the maximum of three numbers.

Task 4

Create a function to check whether a number is even or odd.

Task 5

Create a function to calculate factorial using iteration.

Task 6

Create a recursive function to calculate factorial.

Task 7

Create a function to check whether a number is prime.

Task 8

Create a function to reverse a number.

Task 9

Create separate functions for:

Input

Processing

Output

and build a small calculator.

Task 10

Create a menu-driven program using functions and `switch`.

Diagram 1 — Function Execution

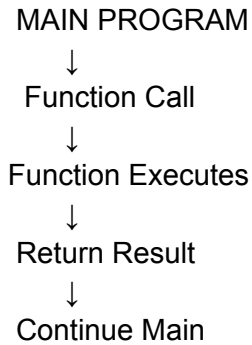


Diagram 2 — Function Components

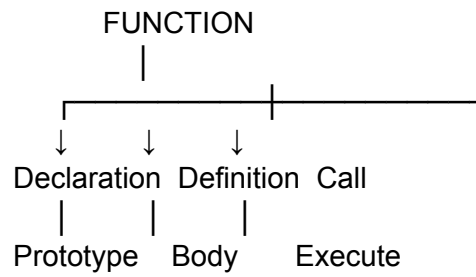
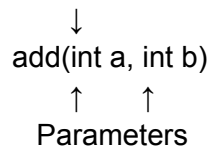


Diagram 3 — Parameter & Argument

Function Definition



Function Call

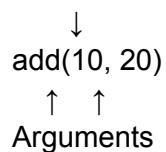
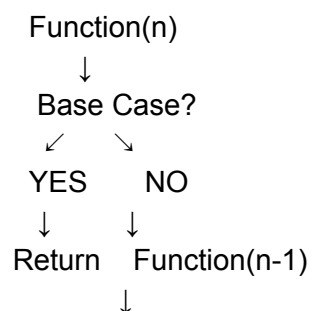
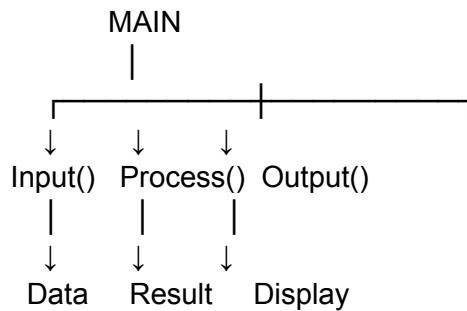


Diagram 4 — Recursion



Repeat

Diagram 5 — Modular Programming



Module 6 Key Takeaway

Remember the complete function flow:

Declaration



Definition



Function Call



Parameters Receive Values



Function Executes



Return Value



Caller Continues

Module 7: Arrays

Arrays are one of the most important topics in C. They introduce you to **contiguous memory, indexing, traversal, searching, sorting, and multidimensional data**.

7.1 Learning Objectives

After completing this module, you will be able to:

- Understand arrays and their characteristics.
- Declare and initialize arrays.
- Access elements using indexes.

- Traverse arrays using loops.
 - Understand array memory layout.
 - Work with one-dimensional arrays.
 - Work with two-dimensional arrays.
 - Pass arrays to functions.
 - Search elements.
 - Find minimum and maximum values.
 - Calculate sum and average.
 - Sort arrays.
 - Understand multidimensional arrays.
 - Solve array-based programming problems.
-

7.2 What is an Array?

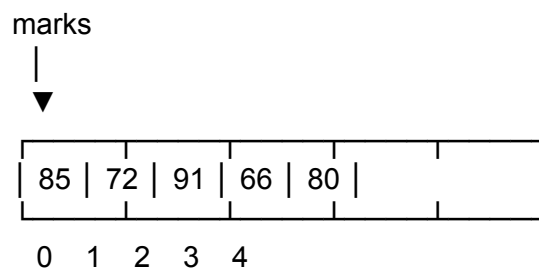
An **array** is a collection of elements of the **same type** stored in contiguous memory locations.

Example:

```
int marks[5];
```

This creates an array capable of storing five `int` elements.

Conceptually:



The numbers below the elements are **indexes**.

7.3 Why Do We Need Arrays?

Suppose you want to store marks of 100 students.

Without an array:

```
int mark1;
```

```
int mark2;  
int mark3;  
...  
int mark100;
```

This is difficult to manage.

With an array:

```
int marks[100];
```

Now all 100 values can be accessed using one variable name and an index.

```
marks[0]  
marks[1]  
marks[2]  
...  
marks[99]
```

7.4 Important Characteristics of Arrays

An array generally has these characteristics:

Same Data Type

```
int numbers[5];
```

All elements are `int`.

Fixed Size

For an ordinary array declaration, its number of elements is determined when the array is created.

Contiguous Storage

Array elements occupy consecutive memory locations.

Zero-Based Indexing

The first element is at index `0`.

7.5 Array Declaration

Syntax

```
data_type array_name[size];
```

Example:

```
int marks[5];
```

Other examples:

```
float prices[10];  
char letters[26];  
double values[20];
```

7.6 Array Indexing

Consider:

```
int numbers[5];
```

The valid indexes are:

0
1
2
3
4

Not:

1
2
3
4
5

Conceptually:

Index:	0	1	2	3	4
	↓	↓	↓	↓	↓
Array:	[10]	[20]	[30]	[40]	[50]

Access:

```
numbers[0]  
numbers[1]  
numbers[4]
```

7.7 Accessing Array Elements

Example:

```
#include <stdio.h>

int main(void)
{
    int numbers[5] = {10, 20, 30, 40, 50};

    printf("%d\n", numbers[0]);
    printf("%d\n", numbers[2]);
    printf("%d\n", numbers[4]);

    return 0;
}
```

Output:

```
10
30
50
```

7.8 Array Initialization

You can initialize an array during declaration.

```
int numbers[5] = {10, 20, 30, 40, 50};
```

The values are assigned according to index:

```
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50
```

7.9 Partial Initialization

You can provide fewer initializers than the array size.

```
int numbers[5] = {10, 20};
```

The remaining elements are initialized to zero.

Conceptually:

Index:	0	1	2	3	4
Value:	10	20	0	0	0

7.10 Size Inference

When an initializer is present, the array size can sometimes be omitted.

```
int numbers[] = {10, 20, 30, 40};
```

The compiler determines the number of elements from the initializer.

Conceptually:

Size = 4

7.11 Modifying Array Elements

Array elements can be changed individually.

```
int numbers[3] = {10, 20, 30};
```

```
numbers[1] = 100;
```

Now:

10 100 30

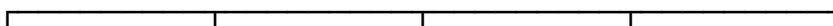
7.12 Array Memory Representation

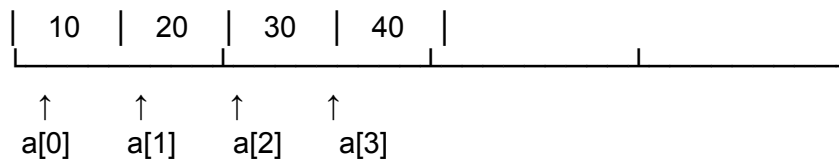
Suppose:

```
int a[4] = {10, 20, 30, 40};
```

Conceptually:

Contiguous Memory





The actual byte addresses depend on the implementation and runtime environment.

7.13 Why Array Index Starts at 0

For an array:

```
int a[5];
```

the first element is:

```
a[0]
```

and the last is:

```
a[4]
```

The number of elements is 5, but the highest valid index is 4.

Formula:

Last valid index = size - 1

7.14 Array Traversal

Traversal means visiting each array element.

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int numbers[5] = {10, 20, 30, 40, 50};
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        printf("%d ", numbers[i]);
```

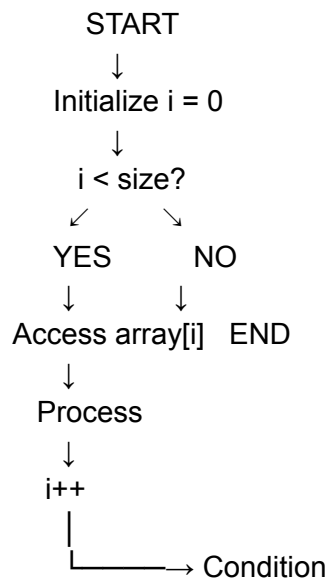
```
    }
```

```
    return 0;
}
```

Output:

10 20 30 40 50

7.15 Array Traversal Flowchart



7.16 Reading Array Elements from User

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int numbers[5];
```

```
    printf("Enter 5 numbers:\n");
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        scanf("%d", &numbers[i]);
```

```
    }
```

```
    printf("Elements:\n");
```

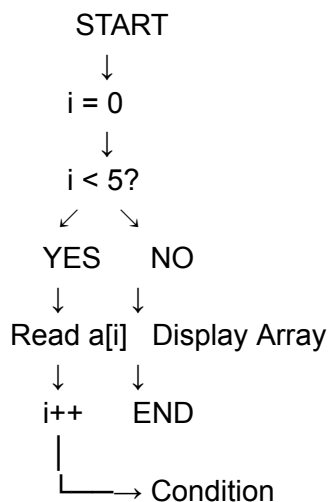
```

for (int i = 0; i < 5; i++)
{
    printf("%d ", numbers[i]);
}

return 0;
}

```

7.17 Array Input Flowchart



7.18 Finding Sum of Array Elements

Example:

$$10 + 20 + 30 + 40 + 50 = 150$$

Program:

```
#include <stdio.h>
```

```

int main(void)
{
    int numbers[5] = {10, 20, 30, 40, 50};
    int sum = 0;

    for (int i = 0; i < 5; i++)
    {
        sum += numbers[i];
    }
}

```

```

    }

    printf("Sum = %d", sum);

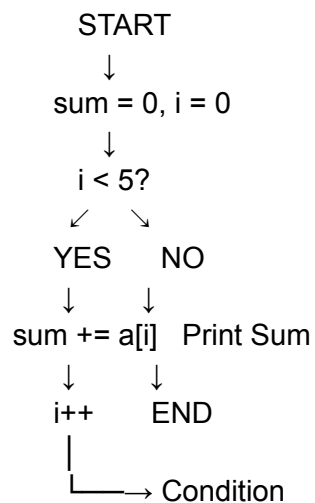
    return 0;
}

```

Output:

Sum = 150

7.19 Sum Flowchart



7.20 Average of Array Elements

Formula:

Average = Sum / Number of Elements

Program:

```
#include <stdio.h>
```

```

int main(void)
{
    int marks[5] = {80, 90, 75, 85, 70};
    int sum = 0;

    for (int i = 0; i < 5; i++)
    {

```

```
        sum += marks[i];
    }

    double average = (double)sum / 5;

    printf("Average = %.2f", average);

    return 0;
}
```

Output:

Average = 80.00

7.21 Finding Maximum Element

Program:

```
#include <stdio.h>

int main(void)
{
    int numbers[5] = {25, 10, 75, 40, 60};
    int max = numbers[0];

    for (int i = 1; i < 5; i++)
    {
        if (numbers[i] > max)
        {
            max = numbers[i];
        }
    }

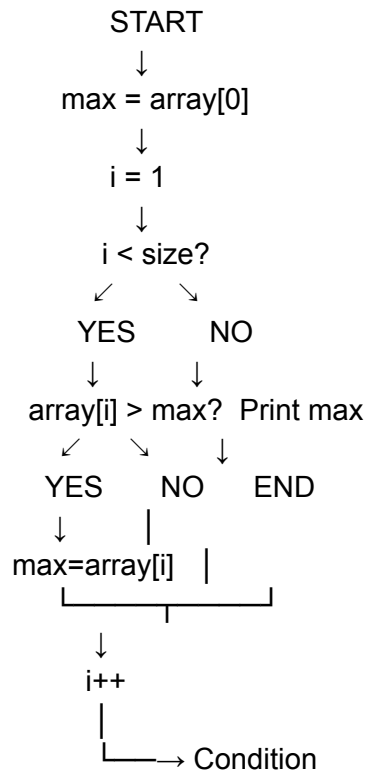
    printf("Maximum = %d", max);

    return 0;
}
```

Output:

Maximum = 75

7.22 Maximum Element Flowchart



7.23 Finding Minimum Element

Program:

```
#include <stdio.h>
```

```
int main(void)
{
    int numbers[5] = {25, 10, 75, 40, 60};
    int min = numbers[0];

    for (int i = 1; i < 5; i++)
    {
        if (numbers[i] < min)
        {
            min = numbers[i];
        }
    }

    printf("Minimum = %d", min);

    return 0;
}
```

Output:

Minimum = 10

7.24 Linear Search

Linear search checks elements one by one until the target is found or the array ends.

Example:

Array: 10 20 30 40 50

Target: 30

Search:

10 → No

20 → No

30 → Found

7.25 Linear Search Program

```
#include <stdio.h>
```

```
int main(void)
{
    int numbers[5] = {10, 20, 30, 40, 50};
    int target;
    int found = 0;

    printf("Enter target: ");
    scanf("%d", &target);

    for (int i = 0; i < 5; i++)
    {
        if (numbers[i] == target)
        {
            printf("Found at index %d", i);
            found = 1;
            break;
        }
    }

    if (!found)
    {
```

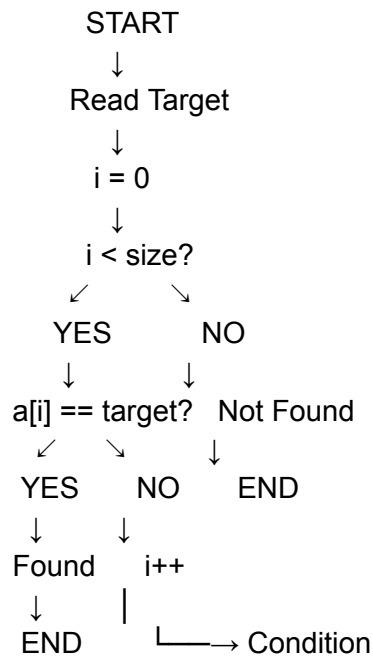
```

        printf("Not found");
    }

    return 0;
}

```

7.26 Linear Search Flowchart



7.27 Two-Dimensional Arrays

A two-dimensional array is commonly used to represent a table or matrix.

Example:

```
int matrix[2][3];
```

This represents:

2 rows × 3 columns

Conceptually:

	Column		
	0	1	2
Row 0	10	20	30

Row 1	40	50	60

7.28 2D Array Declaration

Syntax

`data_type array_name[rows][columns];`

Example:

```
int matrix[3][3];
```

This creates:

3 rows × 3 columns = 9 elements

7.29 2D Array Initialization

```
int matrix[2][3] =
{
    {10, 20, 30},
    {40, 50, 60}
};
```

Conceptually:

```
10 20 30
40 50 60
```

7.30 Accessing 2D Array Elements

Syntax:

`matrix[row][column]`

Example:

```
matrix[0][0]
```

returns:

10

And:

matrix[1][2]

returns:

60

7.31 Traversing a 2D Array

Two loops are normally required.

```
#include <stdio.h>
```

```
int main(void)
{
    int matrix[2][3] =
    {
        {10, 20, 30},
        {40, 50, 60}
    };

    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            printf("%d ", matrix[i][j]);

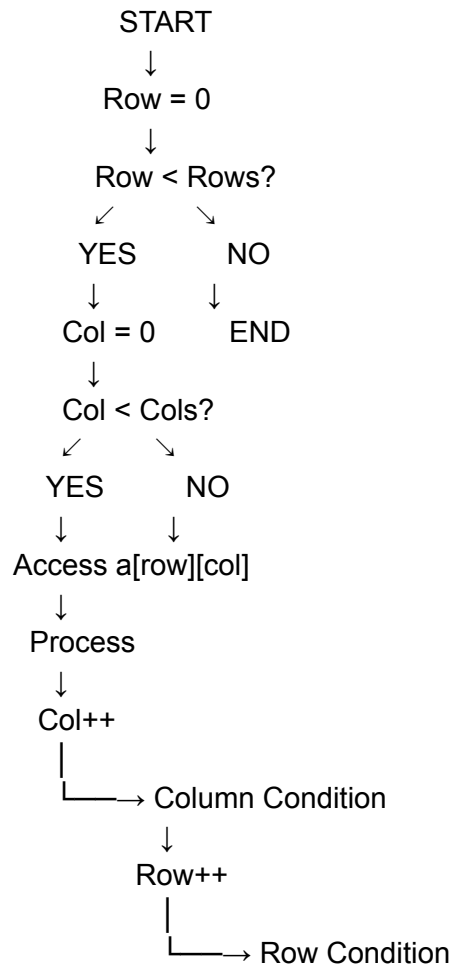
            printf("\n");
        }

        return 0;
    }
}
```

Output:

```
10 20 30
40 50 60
```

7.32 2D Array Traversal Flowchart



7.33 Matrix Addition

Two matrices of the same dimensions can be added element by element.

For example:

A =	B =
1 2	5 6
3 4	7 8

Then:

A + B =

6 8
10 12

7.34 Matrix Addition Program

```
#include <stdio.h>

int main(void)
{
    int a[2][2] =
    {
        {1, 2},
        {3, 4}
    };

    int b[2][2] =
    {
        {5, 6},
        {7, 8}
    };

    int sum[2][2];

    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            sum[i][j] = a[i][j] + b[i][j];
        }
    }

    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            printf("%d ", sum[i][j]);
        }

        printf("\n");
    }

    return 0;
}
```

7.35 Passing an Array to a Function

Arrays can be passed to functions.

Example:

```
#include <stdio.h>
```

```
void display(int a[], int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("%d ", a[i]);
    }
}
```

```
int main(void)
{
    int numbers[] = {10, 20, 30, 40};

    display(numbers, 4);

    return 0;
}
```

Important:

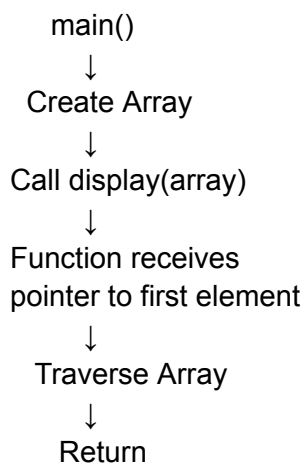
Array $\rightarrow$ Function

↓

Pointer to its first element is passed

The function also needs the number of elements separately in the usual 1D-array interface.

7.36 Array and Function Flow



7.37 Important: Array Size and Function Parameters

Consider:

```
void display(int a[])
```

Inside the function, `a` is adjusted to a pointer parameter.

Therefore:

```
sizeof(a)
```

inside that function does **not** give the size of the original array.

Pass the size explicitly:

```
void display(int a[], int size)
```

Then:

```
for (int i = 0; i < size; i++)
```

7.38 Sorting an Array

Sorting means arranging elements in a particular order.

Ascending

10 20 30 40 50

Descending

50 40 30 20 10

7.39 Bubble Sort

Bubble sort repeatedly compares adjacent elements and swaps them when they are in the wrong order.

Example:

5 3 4 1

First pass:

5 3 → swap
3 5 4 1

5 4 → swap
3 4 5 1

5 1 → swap
3 4 1 5

After more passes:

1 3 4 5

7.40 Bubble Sort Program

```
#include <stdio.h>
```

```
int main(void)
{
    int a[] = {5, 3, 4, 1};
    int n = sizeof(a) / sizeof(a[0]);

    for (int i = 0; i < n - 1; i++)
    {
        int swapped = 0;

        for (int j = 0; j < n - 1 - i; j++)
        {
            if (a[j] > a[j + 1])
            {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;

                swapped = 1;
            }
        }

        if (!swapped)
        {
            break;
        }
    }
}
```

```

for (int i = 0; i < n; i++)
{
    printf("%d ", a[i]);
}

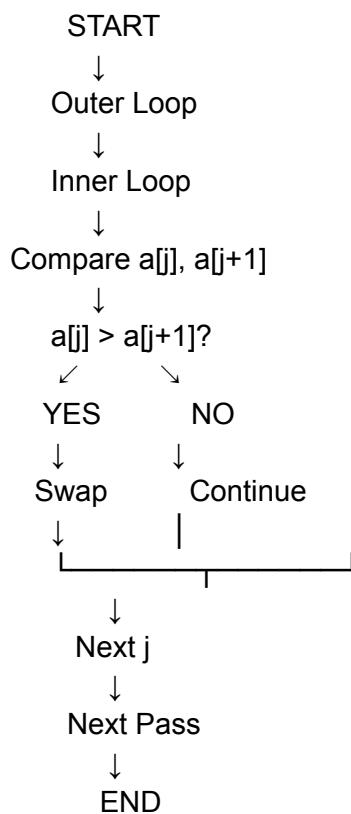
return 0;
}

```

Output:

1 3 4 5

7.41 Bubble Sort Flowchart



7.42 Array of Characters

An array of `char` can store a sequence of characters.

Example:

```
char name[] = "Prasanna";
```

Conceptually:

P r a s a n n a \0

The '`\0`' character marks the end of a C string.

Strings will be studied in much greater detail in the **Strings module**.

7.43 Common Array Errors

Error 1: Out-of-Bounds Access

If:

```
int a[5];
```

valid indexes are:

0 to 4

This is invalid:

```
a[5]
```

Accessing outside the array's bounds results in **undefined behavior**.

Error 2: Incorrect Loop Condition

Incorrect:

```
for (int i = 0; i <= 5; i++)
```

for an array of 5 elements.

Correct:

```
for (int i = 0; i < 5; i++)
```

Error 3: Forgetting **&** During Input

Correct:

```
scanf("%d", &a[i]);
```

Error 4: Incorrect Size Calculation

For an array in the same scope where it is actually an array object:

```
int a[] = {10, 20, 30};
```

```
int n = sizeof(a) / sizeof(a[0]);
```

This gives the number of elements.

But this technique does not work after the array has decayed to a pointer parameter inside a function.

7.44 Array Complexity Preview

Traversal

For n elements:

Time Complexity: $O(n)$

Linear Search

Worst case:

$O(n)$

Bubble Sort

Basic bubble sort:

$O(n^2)$

The optimized version above can terminate early when the array is already sorted, although its worst-case complexity remains $O(n^2)$.

7.45 Best Practices

✓ Always stay within valid array bounds.

✓ Use meaningful names.

```
int studentMarks[50];
```

is better than:

```
int x[50];
```

when the purpose is student marks.

✓ Pass the array size explicitly to functions.

✓ Prefer `sizeof(array) / sizeof(array[0])` when determining the element count of an actual array object in the same scope.

✓ Use nested loops carefully with 2D arrays.

7.46 Interview Questions

1. What is an array?

An array is a collection of elements of the same type stored in contiguous memory.

2. What is the first index of an array?

The first index is `0`.

3. What is the last valid index of an array of size `n`?

$n - 1$

4. What happens when you access an array outside its bounds?

The behavior is undefined.

5. What is array traversal?

Visiting or processing array elements, usually one by one.

6. What is a two-dimensional array?

It is an array whose elements are themselves arranged in multiple dimensions, commonly represented as rows and columns.

7. How is an array passed to a function?

For a normal 1D array parameter, the array argument is converted to a pointer to its first element.

8. Why is the array size usually passed separately to a function?

Because the function parameter does not retain the original array object's element count.

9. What is linear search?

A search technique that examines elements sequentially until the target is found or the array is exhausted.

10. What is bubble sort?

A comparison-based sorting algorithm that repeatedly compares adjacent elements and swaps them when necessary.

7.47 Practical Lab

Task 1

Read **N** numbers into an array and display them.

Task 2

Find the sum and average of array elements.

Task 3

Find the maximum and minimum elements.

Task 4

Count even and odd elements.

Task 5

Search for an element using linear search.

Task 6

Reverse an array.

Example:

Input:

1 2 3 4 5

Output:

5 4 3 2 1

Task 7

Sort an array in ascending order.

Task 8

Sort an array in descending order.

Task 9

Remove duplicate values from an array.

Task 10

Perform matrix addition.

Task 11

Find the transpose of a matrix.

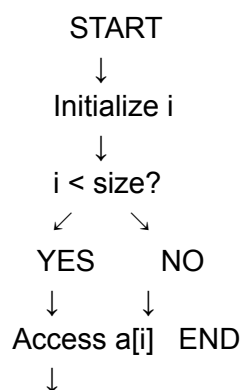
Task 12

Perform matrix multiplication.

Task 13

Find the sum of each row and each column.

Diagram 1 — Array Traversal



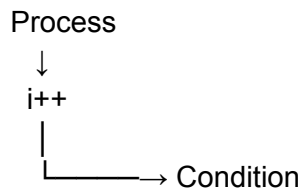


Diagram 2 — Linear Search

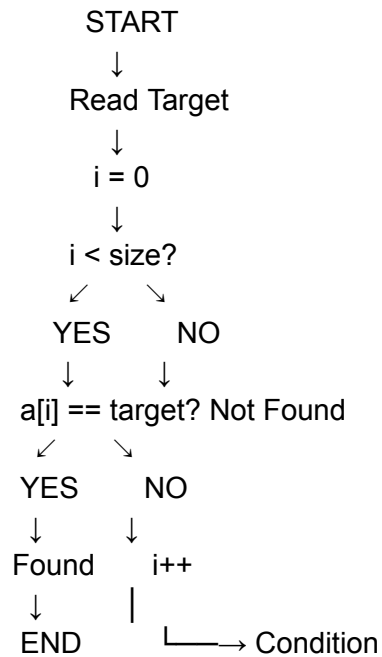
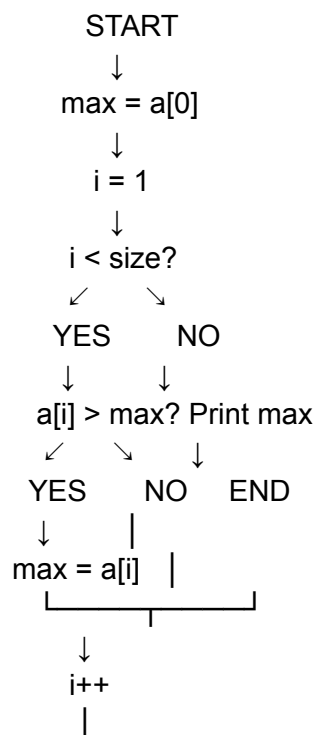


Diagram 3 — Find Maximum



└──→ Condition

Diagram 4 — 2D Array Traversal

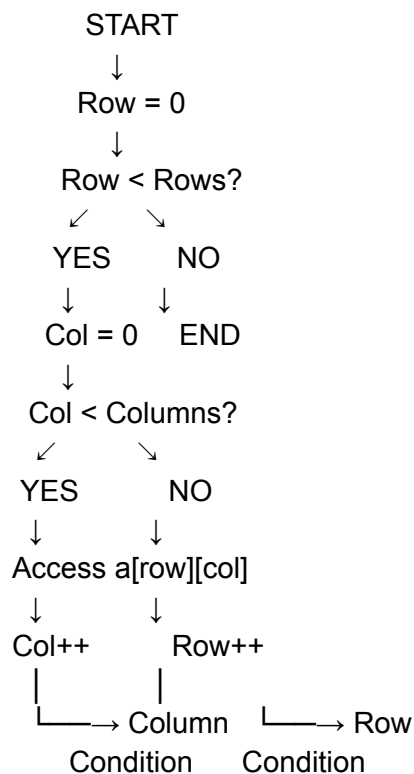
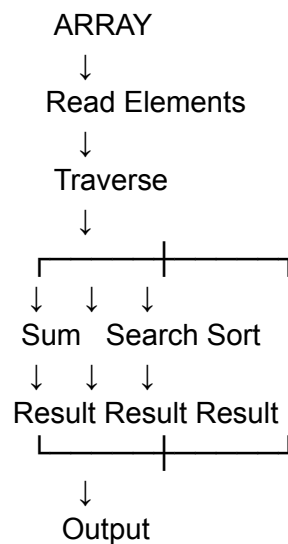


Diagram 5 — Array Processing



Module 7 Key Takeaway

Remember these fundamentals:

Array Declaration



```
int a[5];
```

Indexing



a[0] → First element

a[4] → Last element

Traversal



```
for (i = 0; i < size; i++)
```

Searching



Compare each element

Sorting



Compare + Rearrange

Module 8: Strings in C

8.1 What is a String?

A **string** in C is a sequence of characters stored in a character array and terminated by the null character '`\0`'.

Example:

```
char name[] = "Prasanna";
```

Memory concept:

```
P r a s a n n a \0
```

8.2 Character vs String

```
char ch = 'A';    // Character
```

```
char name[] = "A"; // String
```

Remember:

'A' → Character

"A" → String

- Character → single quotes ' '
 - String → double quotes " "
-

8.3 Declaring a String

```
char name[20];
```

This creates a character array capable of storing a string including its terminating '`\0`', provided sufficient space is available.

Another way:

```
char name[] = "Prasanna";
```

The compiler determines the required array size from the initializer.

8.4 Null Character '`\0`'

The null character marks the **end of a C string**.

```
H e l l o \0
```

Its value is zero.

It is different from:

```
'0'
```

because:

```
'\0' → null character
```

```
'0' → character zero
```

8.5 Reading a String

For a single word:

```
char name[20];
```

```
scanf("%19s", name);
```

Notice that we normally **do not use &** here because the array expression supplies a pointer to its first element.

For strings containing spaces, use `fgets()`:

```
char name[50];
```

```
fgets(name, sizeof name, stdin);
```

`fgets()` is generally safer than the obsolete `gets()` function.

8.6 Displaying a String

Use `%s`:

```
printf("%s", name);
```

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    char name[] = "Prasanna";

    printf("Name: %s", name);

    return 0;
}
```

Output:

```
Name: Prasanna
```

8.7 String Length — `strlen()`

`strlen()` returns the number of characters in a string **excluding** `'\0'`.

Header:

```
#include <string.h>
```

Example:

```
char name[] = "Hello";  
  
printf("%zu", strlen(name));
```

Output:

5

8.8 String Copy — `strcpy()`

`strcpy()` copies a string into another character array.

```
char source[] = "Hello";  
char destination[20];  
  
strcpy(destination, source);
```

Now:

```
destination = "Hello"
```

Include:

```
#include <string.h>
```

⚠ The destination must have enough space for the copied string and its terminating `'\0'`.

8.9 String Concatenation — `strcat()`

`strcat()` appends one string to another.

```
char first[30] = "Hello ";  
char second[] = "World";
```

```
strcat(first, second);
```

Result:

Hello World

The destination array must have enough capacity for the combined result.

8.10 String Comparison — `strcmp()`

`strcmp()` compares two strings lexicographically.

`strcmp(str1, str2)`

Possible results:

0 → Strings are equal
< 0 → str1 comes before str2
> 0 → str1 comes after str2

Example:

```
if (strcmp("apple", "apple") == 0)
{
    printf("Equal");
}
```

Output:

Equal

⚠ Do not compare C strings using:

`str1 == str2`

That compares pointer values, not string contents.

8.11 Important String Functions

Function	Purpose
<code>strlen()</code>	Find string length
<code>strcpy()</code>	Copy string
<code>strcat()</code>	Concatenate strings
<code>strcmp()</code>	Compare strings

Header:

```
#include <string.h>
```

8.12 String Traversal

You can process a string character by character.

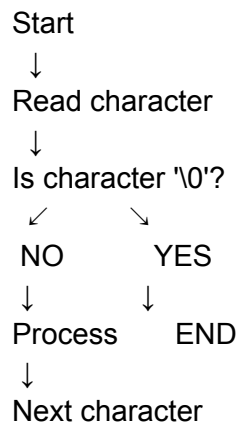
```
#include <stdio.h>
```

```
int main(void)
{
    char name[] = "Hello";

    for (int i = 0; name[i] != '\0'; i++)
    {
        printf("%c\n", name[i]);
    }

    return 0;
}
```

Flow:



8.13 Reverse a String

A string can be reversed by processing characters from the end toward the beginning.

Example:

Input → HELLO
Output → OLLEH

One approach:

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char str[] = "HELLO";

    for (int i = (int)strlen(str) - 1; i >= 0; i--)
    {
        putchar(str[i]);
    }

    return 0;
}
```

8.14 Count Vowels

Example:

```
#include <stdio.h>

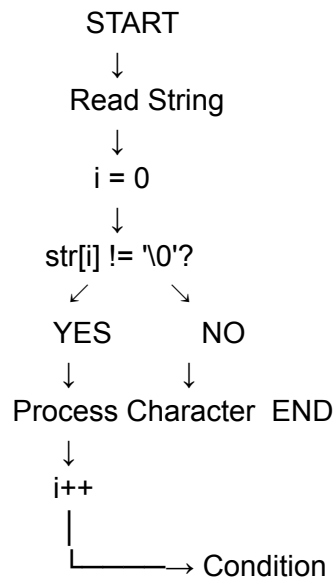
int main(void)
{
    char str[] = "education";
    int count = 0;

    for (int i = 0; str[i] != '\0'; i++)
    {
        if (str[i] == 'a' || str[i] == 'e' ||
            str[i] == 'i' || str[i] == 'o' ||
            str[i] == 'u')
        {
            count++;
        }
    }

    printf("Vowels = %d", count);

    return 0;
}
```

8.15 String Flowchart



8.16 Common Mistakes

Mistake 1: Forgetting Space for '\\0'

```
char name[5] = "Hello";
```

This is invalid because "Hello" requires **6 characters of storage** including '\\0'.

Correct:

```
char name[6] = "Hello";
```

or simply:

```
char name[] = "Hello";
```

Mistake 2: Using == for String Contents

Incorrect:

```
if (name1 == name2)
```

Correct:

```
if (strcmp(name1, name2) == 0)
```

Mistake 3: Using `gets()`

Do not use:

```
gets(name);
```

`gets()` was removed from the C standard because it cannot safely limit input length.

Use:

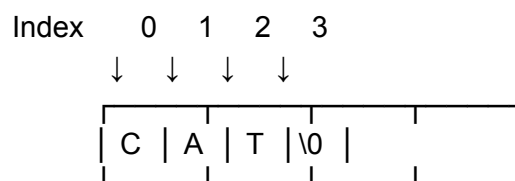
```
fgets(name, sizeof name, stdin);
```

8.17 String Memory Concept

For:

```
char word[] = "CAT";
```

conceptually:



Therefore:

```
strlen(word) = 3
```

but the array requires **4 characters of storage**.

8.18 Interview Questions

1. What is a string in C?

A character sequence terminated by `'\0'`.

2. Which header contains common string functions?

```
#include <string.h>
```

3. What does `strlen()` return?

The number of characters before the terminating null character.

4. Why is '`\0`' important?

It marks the end of a C string.

5. Can we compare strings using `==`?

No. Use `strcmp()` to compare their contents.

6. Which function is preferred for reading a line safely?

`fgets()`.

8.19 Practical Lab

1. Find the length of a string.
 2. Copy one string into another.
 3. Concatenate two strings.
 4. Compare two strings.
 5. Reverse a string.
 6. Count vowels and consonants.
 7. Count spaces and words.
 8. Check whether a string is a palindrome.
 9. Convert lowercase characters to uppercase.
 10. Find the frequency of a particular character.
-



Module 8 Quick Revision

String



Character Array



Ends with '`\0`'



Common Functions

- `strlen()` → Length
- `strcpy()` → Copy
- `strcat()` → Join
- `strcmp()` → Compare

★ Remember

'A' → Character

"A" → String

'\0' → String terminator

strlen() → Length

strcpy() → Copy

strcat() → Concatenate

strcmp() → Compare

fgets() → Read a line safely

This is the **brief version of Module 8**, suitable for quick revision and keeping the notes compact.

Module 9: Pointers in C

Pointers are one of the **most important and powerful concepts in C**. They connect directly with memory, arrays, strings, functions, dynamic memory allocation, and data structures.

9.1 Learning Objectives

After this module, you will understand:

- What a pointer is.
 - Memory addresses.
 - Address-of operator **&**.
 - Dereference operator *****.
 - Pointer declaration and initialization.
 - Pointer arithmetic.
 - Relationship between arrays and pointers.
 - Pointers and functions.
 - Pointers to pointers.
 - **NULL** pointers.
 - Common pointer errors.
-

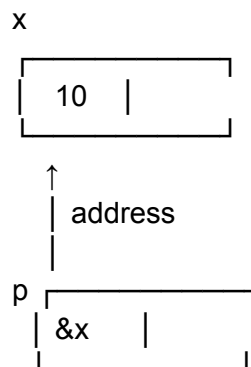
9.2 What is a Pointer?

A **pointer** is a variable that stores the address of another object.

Example:

```
int x = 10;  
int *p = &x;
```

Conceptually:



Here:

```
x → stores 10  
&x → address of x  
p → stores address of x  
*p → value stored at that address
```

9.3 Address-of Operator &

The **&** operator obtains the address of an object.

```
int x = 25;  
  
printf("%p", (void *)&x);
```

The exact address is implementation-dependent.

9.4 Dereference Operator *

The ***** operator can be used to access the object pointed to by a pointer.

```
int x = 25;  
int *p = &x;
```

```
printf("%d", *p);
```

Output:

25

So:

$\&p \rightarrow$ address of p's target

$*p \rightarrow$ value of p's target

More precisely:

$p \rightarrow$ address stored in pointer

$*p \rightarrow$ object reached through that address

9.5 Pointer Declaration

Syntax

```
data_type *pointer_name;
```

Examples:

```
int *p;
```

```
float *f;
```

```
char *c;
```

```
double *d;
```

The pointed-to type matters because it determines how dereferencing and pointer arithmetic are interpreted.

9.6 Pointer Initialization

Always initialize a pointer before dereferencing it.

```
int x = 100;
```

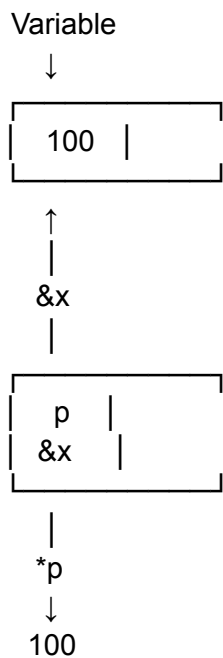
```
int *p = &x;
```

Now:

```
printf("%d", *p);
```

prints:

9.7 Pointer Flow



9.8 Changing a Value Through a Pointer

```
#include <stdio.h>
```

```
int main(void)
{
    int x = 10;
    int *p = &x;

    *p = 50;

    printf("%d", x);

    return 0;
}
```

Output:

50

Why?

$p \rightarrow$ address of x

$*p \rightarrow$ x itself

Therefore:

$*p = 50;$

changes x .

9.9 Pointer and `scanf()`

Pointers explain why we normally use `&` with `scanf()` for ordinary scalar variables.

```
int age;
```

```
scanf("%d", &age);
```

Conceptually:

User Input

↓

`scanf()`

↓

`&age`

↓

Address of `age`

↓

Store input in `age`

9.10 Pointer Data Types

A pointer has a type.

```
int *p;
```

```
char *c;
```

```
float *f;
```

```
double *d;
```

The pointer's type tells the compiler what kind of object it points to.

For example:

```
int x = 10;
```

```
int *p = &x;
```

`p` points to an `int`.

9.11 Pointer Size

The size of a pointer is determined by the implementation and address model.

For example, on many modern 64-bit systems, object pointers are commonly 8 bytes, but **you should not assume this universally**.

Check with:

```
printf("%zu", sizeof(p));
```

The size of the pointer is not necessarily the size of the object it points to.

9.12 NULL Pointer

A null pointer does not point to a valid object.

Example:

```
int *p = NULL;
```

Use:

```
#include <stddef.h>
```

or another standard header that defines `NULL`.

Before dereferencing:

```
if (p != NULL)
{
    printf("%d", *p);
}
```

Important

Never do:

```
*p
```

when `p` is `NULL`.

That would result in undefined behavior.

9.13 Pointer vs Normal Variable

Normal Variable	Pointer
Stores a value	Stores an address
<code>x</code>	<code>p</code>
<code>x = 10</code>	<code>p = &x</code>
<code>x</code> gives value	<code>*p</code> accesses pointed-to value
<code>&x</code> gives address	<code>p</code> contains an address

9.14 Pointer Arithmetic

Pointers can participate in arithmetic operations under specific rules.

Suppose:

```
int a[] = {10, 20, 30, 40};  
int *p = a;
```

Then:

`p`

points to `a[0]`.

If:

```
p++;
```

then it points to `a[1]`.

Conceptually:

`p`
↓

10 20 30 40
↑
p++

Pointer arithmetic is scaled according to the size of the pointed-to type.

9.15 Pointer Increment

```
int a[] = {10, 20, 30};
```

```
int *p = a;
```

```
printf("%d\n", *p);
```

```
p++;
```

```
printf("%d\n", *p);
```

Output:

10
20

9.16 Pointer Decrement

```
p--;
```

moves the pointer to the previous element when that operation is valid within the same array object.

Example:

10 20 30

↑
p

```
p--
```

↑
p

9.17 Pointer Addition

$p + 2$

means the pointer is advanced by two elements of its pointed-to type.

For:

```
int a[5];  
int *p = a;
```

then:

```
p      → a[0]  
p + 1  → a[1]  
p + 2  → a[2]
```

9.18 Pointer and Arrays

One of the most important relationships in C:

For most expressions, an array name is converted to a pointer to its first element.

Example:

```
int a[5] = {10, 20, 30, 40, 50};
```

Then:

a

in an expression behaves like:

$\&a[0]$

So:

$a[0]$

is equivalent to:

$*(a + 0)$

and:

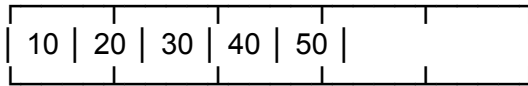
$a[2]$

is equivalent to:

*(a + 2)

9.19 Array and Pointer Relationship

Array:



↑
|
a

Conceptually:

a → first element
a + 1 → second element
a + 2 → third element

9.20 Traversing an Array Using a Pointer

```
#include <stdio.h>
```

```
int main(void)
{
    int a[] = {10, 20, 30, 40, 50};
    int n = sizeof(a) / sizeof(a[0]);

    for (int *p = a; p < a + n; p++)
    {
        printf("%d ", *p);
    }

    return 0;
}
```

Output:

10 20 30 40 50

9.21 Pointer and Function

Pointers allow a function to modify an object belonging to its caller.

Example:

```
#include <stdio.h>
```

```
void update(int *p)
{
    *p = 100;
}
```

```
int main(void)
{
    int x = 10;

    update(&x);

    printf("%d", x);

    return 0;
}
```

Output:

100

Flow:

```
main()
↓
x = 10
↓
update(&x)
↓
p receives address of x
↓
*p = 100
↓
x becomes 100
```

9.22 Swapping Two Numbers Using Pointers

This is a very important interview program.

```
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void)
{
    int x = 10;
    int y = 20;

    swap(&x, &y);

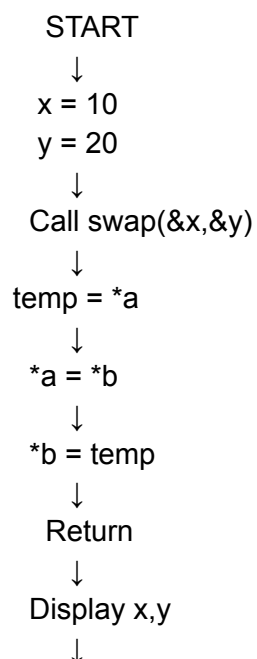
    printf("x = %d\n", x);
    printf("y = %d\n", y);

    return 0;
}
```

Output:

```
x = 20
y = 10
```

9.23 Swap Flowchart



END

9.24 Pointer to Pointer

A pointer can itself have an address, so we can have a **pointer to a pointer**.

Example:

```
int x = 10;
int *p = &x;
int **q = &p;
```

Conceptually:

```
x = 10
↑
p = &x
↑
q = &p
```

Therefore:

```
x → 10
*p → 10
**q → 10
```

9.25 Example of Pointer to Pointer

```
#include <stdio.h>
```

```
int main(void)
{
    int x = 10;
    int *p = &x;
    int **q = &p;

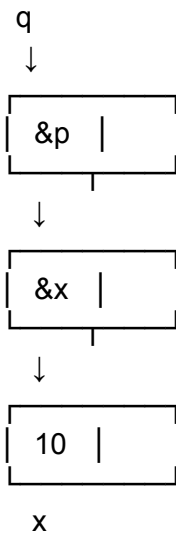
    printf("%d\n", x);
    printf("%d\n", *p);
    printf("%d\n", **q);

    return 0;
}
```

Output:

10
10
10

9.26 Pointer to Pointer Diagram



9.27 **const** and Pointers

Pointers can be combined with **const** in different ways.

Pointer to constant data

```
const int *p;
```

You cannot modify the pointed-to integer through **p**.

Constant pointer

```
int *const p = &x;
```

The pointer itself cannot be changed to point somewhere else after initialization.

Constant pointer to constant data

```
const int *const p = &x;
```

Neither the pointer nor the pointed-to value can be modified through **p**.

This distinction is important in professional C programming.

9.28 Dangling Pointer

A **dangling pointer** is a pointer that refers to an object whose lifetime has ended.

Example:

```
int *p;
```

```
{  
    int x = 10;  
    p = &x;  
}
```

```
/* x's lifetime has ended here */
```

Now **p** is dangling.

Dereferencing it is undefined behavior.

9.29 Wild Pointer

A pointer that has not been initialized can contain an indeterminate value.

Example:

```
int *p;
```

Do not do:

```
printf("%d", *p);
```

Initialize it:

```
int *p = NULL;
```

until you have a valid target.

9.30 Common Pointer Errors

Error 1: Dereferencing **NULL**

```
int *p = NULL;
```

```
printf("%d", *p);
```

✗ Undefined behavior.

Error 2: Uninitialized Pointer

```
int *p;
```

```
*p = 10;
```

✗ Invalid because **p** does not point to a valid object.

Error 3: Wrong Pointer Type

Avoid incompatible pointer conversions unless you understand the required representation and alignment rules.

Error 4: Returning Address of a Local Variable

Incorrect:

```
int *getValue(void)
{
    int x = 10;
    return &x;
}
```

x ceases to exist when the function returns, so the returned pointer becomes invalid.

9.31 Pointer and **sizeof**

Consider:

```
int x;
```

```
int *p = &x;
```

Then:

sizeof(x)

gives the size of the `int`.

Whereas:

sizeof(p)

gives the size of the pointer.

They are different concepts.

9.32 Why Pointers Are Important

Pointers are fundamental to:

- Arrays
- Strings
- Functions
- Dynamic memory
- Structures
- Linked lists
- Trees
- Graphs
- Operating systems
- Embedded systems
- Low-level programming

Understanding pointers is therefore essential for becoming strong in C.

9.33 Interview Questions

1. What is a pointer?

A pointer is an object that stores the address of another object.

2. What does `&` do?

It obtains the address of an object.

3. What does `*` do with a pointer?

It dereferences the pointer to access the pointed-to object.

4. What is a **NULL** pointer?

A null pointer is a pointer value that is guaranteed not to point to any object or function.

5. What is pointer arithmetic?

Operations such as incrementing or decrementing a pointer to move between elements of an array object.

6. What is a dangling pointer?

A pointer that refers to an object whose lifetime has ended.

7. What is a pointer to pointer?

A pointer that stores the address of another pointer.

```
int **p;
```

8. Why are pointers used in functions?

They allow functions to access or modify caller-owned objects and support many data structures and dynamic-memory techniques.

9.34 Practical Lab

1. Print the address of a variable.
 2. Access a variable through a pointer.
 3. Modify a variable using a pointer.
 4. Swap two numbers using pointers.
 5. Find the largest of two numbers using pointers.
 6. Traverse an array using a pointer.
 7. Calculate the sum of an array using pointers.
 8. Demonstrate pointer-to-pointer.
 9. Create a function that modifies multiple variables using pointers.
 10. Experiment with **const** pointers.
-

9.35 Quick Revision

Variable



Stores Value

Pointer



Stores Address

&x



Address of x

*p



Value of object pointed to by p

Array Connection

a[i]

=

*(a + i)

Function Connection

main()



function(&x)



pointer receives address



*p modifies x

Pointer Safety

Initialize



Validate



Dereference

Module 10: Structures, Unions & Enumerations

This module introduces **user-defined data types** in C. These are essential for representing real-world entities such as **students, employees, products, bank accounts, and records**.

10.1 Learning Objectives

After this module, you will understand:

- Structures
 - Structure declaration and initialization
 - Accessing structure members
 - Array of structures
 - Nested structures
 - Structures with functions
 - Pointers to structures
 - `->` operator
 - Unions
 - Difference between structures and unions
 - Enumerations (`enum`)
 - `typedef`
 - Practical applications
-

10.2 Why Do We Need Structures?

Suppose you want to store information about a student:

Name

Roll Number

Marks

Grade

These values have different data types:

```
char name[30];
```

```
int roll;
```

```
float marks;
```

```
char grade;
```

Managing them as separate variables becomes inconvenient.

A structure allows us to group related data:

Student

```
|— name
```

```
|— roll
```

```
|— marks
```

```
|— grade
```

10.3 What is a Structure?

A **structure** is a user-defined data type that groups related variables, potentially of different data types, under one name.

Example:

```
struct Student
{
    char name[30];
    int roll;
    float marks;
};
```

Here:

Student

- └─ name → char array
- └─ roll → int
- └─ marks → float

10.4 Structure Syntax

General syntax:

```
struct StructureName
{
    data_type member1;
    data_type member2;
    data_type member3;
};
```

Example:

```
struct Employee
{
    int id;
    char name[50];
    double salary;
};
```

10.5 Creating Structure Variables

After defining a structure:

```
struct Student
{
    char name[30];
    int roll;
    float marks;
};
```

Create a variable:

```
struct Student s1;
```

Now **s1** contains:

```
s1
├── name
├── roll
└── marks
```

10.6 Accessing Structure Members

The **dot operator** `.` is used to access members of a structure variable.

Example:

```
s1.roll = 101;
s1.marks = 85.5f;
```

For character arrays, you generally initialize/copy the string rather than assign a string literal directly after declaration.

Example:

```
strcpy(s1.name, "Prasanna");
```

10.7 Complete Structure Example

```
#include <stdio.h>
#include <string.h>
```

```
struct Student
{
    char name[30];
```

```
    int roll;
    float marks;
};

int main(void)
{
    struct Student s1;

    strcpy(s1.name, "Prasanna");
    s1.roll = 101;
    s1.marks = 88.5f;

    printf("Name: %s\n", s1.name);
    printf("Roll: %d\n", s1.roll);
    printf("Marks: %.2f\n", s1.marks);

    return 0;
}
```

Output:

```
Name: Prasanna
Roll: 101
Marks: 88.50
```

10.8 Structure Initialization

A structure can be initialized when it is declared.

```
struct Student s1 =
{
    "Prasanna",
    101,
    88.5f
};
```

The values correspond to the members in declaration order.

10.9 Designated Initializers

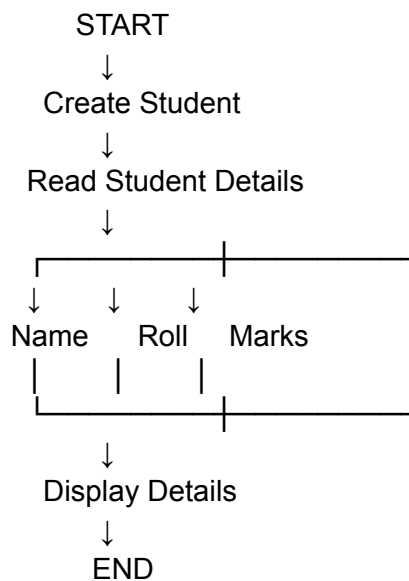
A more explicit method is:

```
struct Student s1 =
```

```
{  
    .name = "Prasanna",  
    .roll = 101,  
    .marks = 88.5f  
};
```

This improves readability and reduces dependence on member order.

10.10 Structure Flowchart



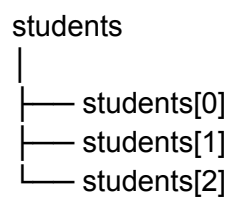
10.11 Array of Structures

You can create an array of structure variables.

Example:

```
struct Student students[3];
```

Conceptually:



Each element contains all structure members.

10.12 Example: Array of Students

```
#include <stdio.h>

struct Student
{
    int roll;
    float marks;
};

int main(void)
{
    struct Student students[3] =
    {
        {101, 85.5f},
        {102, 91.0f},
        {103, 78.5f}
    };

    for (int i = 0; i < 3; i++)
    {
        printf("Roll: %d, Marks: %.2f\n",
            students[i].roll,
            students[i].marks);
    }

    return 0;
}
```

10.13 Nested Structures

A structure can contain another structure as a member.

Example:

```
struct Date
{
    int day;
    int month;
    int year;
};
```

```
struct Student
{
    char name[30];
    int roll;
    struct Date dob;
};
```

Access:

```
student.dob.day
student.dob.month
student.dob.year
```

Conceptually:

```
Student
├── name
├── roll
└── dob
    ├── day
    ├── month
    └── year
```

10.14 Structure and Functions

Structures can be passed to functions.

Example:

```
#include <stdio.h>
```

```
struct Student
{
    int roll;
    float marks;
};
```

```
void display(struct Student s)
{
    printf("Roll = %d\n", s.roll);
    printf("Marks = %.2f\n", s.marks);
}
```

```
int main(void)
{
    struct Student s = {101, 90.5f};
```

```
    display(s);  
  
    return 0;  
}
```

Here, the structure is passed **by value**, so the function receives a copy.

10.15 Pointer to Structure

You can create a pointer to a structure.

```
struct Student s = {101, 90.5f};
```

```
struct Student *p = &s;
```

Now:

p → address of s

10.16 Accessing Structure Through Pointer

You can use:

```
(*p).roll
```

But C provides a more convenient operator:

```
p->roll
```

These are equivalent:

```
(*p).roll
```

```
p->roll
```

10.17 → Operator Example

```
#include <stdio.h>
```

```

struct Student
{
    int roll;
    float marks;
};

int main(void)
{
    struct Student s = {101, 92.5f};
    struct Student *p = &s;

    printf("Roll = %d\n", p->roll);
    printf("Marks = %.2f\n", p->marks);

    return 0;
}

```

10.18 Structure vs Pointer to Structure

Structure Variable	Structure Pointer
<code>s.roll</code>	<code>p->roll</code>
Stores structure object	Stores address of structure
Uses <code>.</code>	Uses <code>-></code>

Remember:

Structure object → `.`
 Structure pointer → `->`

10.19 Structure Memory and Padding

Structure members are stored in memory in declaration order, but the compiler may insert **padding bytes** between or after members to satisfy alignment requirements.

Example:

```

struct Example
{
    char c;

```



```
int x;  
};
```

The structure's size may be larger than:

`sizeof(char) + sizeof(int)`

because of padding.

Check the actual size with:

`sizeof(struct Example)`

This is important when working with:

- Memory optimization
 - Binary files
 - Network protocols
 - Embedded systems
-

10.20 What is a Union?

A **union** is a user-defined type whose members share the same memory location.

Example:

```
union Data  
{  
    int i;  
    float f;  
    char c;  
};
```

Unlike a structure, all members occupy overlapping storage.

10.21 Union Example

```
#include <stdio.h>
```

```
union Data  
{  
    int i;  
    float f;
```

```

    char c;
};

int main(void)
{
    union Data data;

    data.i = 10;

    printf("%d\n", data.i);

    data.f = 20.5f;

    printf("%.2f\n", data.f);

    return 0;
}

```

When `data.f` is assigned, the previously stored `data.i` value should not be treated as still representing the active value.

10.22 Structure vs Union

This is a very important interview question.

Structure	Union
Members have separate storage	Members share storage
Multiple members can hold values simultaneously	Typically one member's stored representation is used at a time
Size is influenced by all members plus padding	Size is at least enough for its largest member, subject to alignment
Uses more memory	Can save memory
<code>.</code> accesses members	<code>.</code> accesses members

Example:

```

struct A
{
    int x;
    float y;
}

```

```
};
```

Both `x` and `y` have separate storage.

But:

```
union A
{
    int x;
    float y;
};
```

`x` and `y` overlap.

10.23 When to Use a Union

Unions are useful when an object needs to represent **one of several possible representations** and sharing storage is desirable.

Examples include:

- Memory-constrained systems
- Embedded programming
- Tagged/variant-style data representations
- Low-level data interpretation

Often, a separate field called a **tag/discriminator** is used to record which union member is currently meaningful.

10.24 What is an Enumeration?

An `enum` defines a type consisting of named integer constants.

Example:

```
enum Day
{
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY
```

```
};
```

By default:

```
MONDAY   → 0
TUESDAY  → 1
WEDNESDAY → 2
THURSDAY → 3
FRIDAY   → 4
```

10.25 Using **enum**

```
#include <stdio.h>
```

```
enum Status
{
    OFF,
    ON
};
```

```
int main(void)
{
    enum Status device = ON;

    printf("%d", device);

    return 0;
}
```

Output:

```
1
```

10.26 Custom Enum Values

You can assign specific integer values.

```
enum Status
{
    ERROR = -1,
    SUCCESS = 0,
    PENDING = 1
};
```

Now:

```
ERROR → -1  
SUCCESS → 0  
PENDING → 1
```

If later enumerators have no explicit value, numbering continues from the previous value.

10.27 Why Use **enum**?

Instead of:

```
int status = 1;
```

you can write:

```
enum Status status = SUCCESS;
```

This improves readability and communicates intent.

10.28 **typedef**

typedef creates an alias for an existing type.

Example:

```
typedef unsigned int uint;
```

Now:

```
uint age;
```

is equivalent in type to:

```
unsigned int age;
```

10.29 **typedef** with Structures

Without **typedef**:

```
struct Student s1;
```

With `typedef`:

```
typedef struct
{
    int roll;
    float marks;
} Student;
```

Now:

```
Student s1;
```

This is very common in C codebases.

10.30 `typedef` with Named Structures

You can also write:

```
typedef struct Student
{
    int roll;
    float marks;
} Student;
```

Then:

```
Student s1;
```

The structure still has the tag `struct Student`, while `Student` is an alias created by `typedef`.

10.31 Practical Example: Employee Record

```
#include <stdio.h>
```

```
typedef struct
{
    int id;
```

```

    char name[30];
    float salary;
} Employee;

int main(void)
{
    Employee e = {101, "Rahul", 45000.0f};

    printf("ID: %d\n", e.id);
    printf("Name: %s\n", e.name);
    printf("Salary: %.2f\n", e.salary);

    return 0;
}

```

10.32 Real-World Data Model

Consider an employee:

```

Employee
|
├── ID
├── Name
├── Salary
└── Department

```

In C:

```

struct Employee
{
    int id;
    char name[50];
    float salary;
    char department[30];
};

```

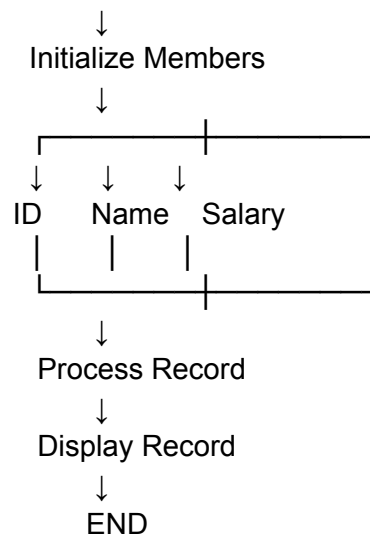
This is much more organized than maintaining unrelated variables.

10.33 Structure Flowchart

```

    START
    ↓
    Create Structure

```



10.34 Structure vs Array

Array	Structure
Same data type	Different data types allowed
Indexed using <code>[]</code>	Members accessed using <code>.</code> or <code>-></code>
Good for collections of similar values	Good for records/entities
Example: marks	Example: student record

Example:

```
int marks[5];
```

vs.

```
struct Student
{
    int roll;
    char name[30];
    float marks;
};
```

10.35 Common Mistakes

Mistake 1: Forgetting **struct**

If no typedef alias exists:

```
struct Student s;
```

is correct.

Just:

```
Student s;
```

is only valid if **Student** has been defined as an alias, for example using **typedef**.

Mistake 2: Using **.** with a Structure Pointer

Incorrect:

```
p.roll
```

If **p** is a pointer:

```
p->roll
```

is correct.

Mistake 3: Confusing Structure and Union

Remember:

Structure → Separate member storage

Union → Shared member storage

Mistake 4: Assuming Union Members Hold Independent Values

A union's members overlap in storage, so assigning one member can change the stored representation seen through another member.

10.36 Interview Questions

1. What is a structure?

A structure is a user-defined type that groups related members, potentially of different data types.

2. What is a union?

A union is a user-defined type whose members share the same storage.

3. What is the difference between structure and union?

Structures allocate separate storage for members, while union members overlap in the same storage.

4. What is `->`?

It accesses a structure or union member through a pointer.

`p->member`

5. What is `typedef`?

It creates an alias for an existing type.

6. What is `enum`?

It defines a type containing named integer constants.

7. Can structures contain other structures?

Yes. This is called a nested structure.

8. Can structures be passed to functions?

Yes.

9. Can a structure contain an array?

Yes.

Example:

```
struct Student
{
    char name[50];
};
```

10. Why are structures important?

They allow programs to model complex real-world entities as organized records.

10.37 Practical Lab

Task 1

Create a **Student** structure with:

Name
Roll Number
Marks

Task 2

Create an **Employee** structure with:

ID
Name
Salary

Task 3

Create an array of 5 students and display their details.

Task 4

Find the student with the highest marks.

Task 5

Create a nested structure containing student and date-of-birth information.

Task 6

Create a structure pointer and access members using **->**.

Task 7

Create a union containing:

int
float
char

and observe shared storage.

Task 8

Create an `enum` for:

MONDAY
TUESDAY
...
SUNDAY

Task 9

Use `typedef` to create a convenient name for a structure.

10.38 Quick Revision

STRUCTURE



Groups different data types



Separate storage



Access → .

UNION



Groups different possible representations



Shared storage



Access → .

STRUCTURE POINTER



Stores address of structure



Access → ->

ENUM



Named integer constants

TYPEDEF



Creates a type alias

Module 10 Key Takeaway

Remember these five concepts:

struct → Group related data

union → Share storage among members

enum → Named integer constants

typedef → Type alias

-> → Access member through structure pointer

Module 11: Dynamic Memory Allocation

Dynamic memory allocation is an advanced and very important C concept. It allows programs to request memory **at runtime** instead of deciding the required memory size completely at compile time.

11.1 Learning Objectives

After this module, you will understand:

- Static vs dynamic memory.
 - Stack and heap basics.
 - `malloc()`
 - `calloc()`
 - `realloc()`
 - `free()`
 - Dynamic arrays.
 - Dynamic memory with pointers.
 - Memory leaks.
 - Dangling pointers.
 - `NULL` checks.
 - Safe memory-management practices.
-

11.2 Why Dynamic Memory?

Consider:

```
int marks[100];
```

This reserves space for 100 integers.

But what if the user wants to enter only 10?

Or 1,000?

A dynamically allocated array can request memory based on the runtime requirement.

User enters N



Allocate memory for N elements



Use memory



Release memory

11.3 Static vs Dynamic Memory

Fixed-size array

```
int numbers[100];
```

Size is determined as part of the array object's creation.

Dynamic memory

```
int *numbers;
```

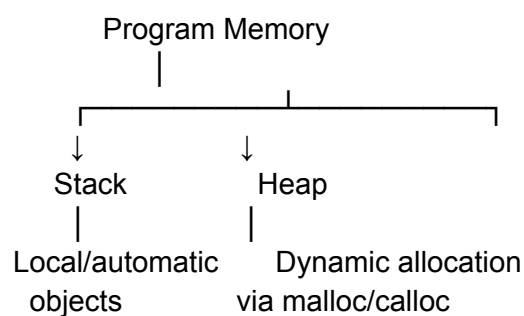
```
numbers = malloc(n * sizeof *numbers);
```

Memory is requested at runtime.

11.4 Stack and Heap

At a high level, program memory can involve different storage regions.

For this module, focus on:



Stack

Typically used for automatic storage such as local variables.

Heap

Used for dynamically allocated storage.

Exact memory organization depends on the implementation, operating system, compiler, and runtime environment.

11.5 Dynamic Memory Functions

The main functions are provided by:

```
#include <stdlib.h>
```

Function	Purpose
<code>malloc(</code> <code>)</code>	Allocate uninitialized storage
<code>calloc(</code> <code>)</code>	Allocate zero-initialized storage
<code>realloc</code> <code>()</code>	Resize an existing allocation
<code>free()</code>	Release allocated storage

11.6 `malloc()`

`malloc()` allocates a specified number of bytes.

Syntax

```
ptr = malloc(number_of_bytes);
```

Example:

```
int *p = malloc(5 * sizeof *p);
```

This requests enough storage for 5 `int` objects.

11.7 Important Property of `malloc()`

Memory returned by `malloc()` is **uninitialized**.

Example:

```
int *p = malloc(5 * sizeof *p);
```

Do not assume:

```
p[0] = 0
```

```
p[1] = 0
```

```
...
```

The contents are indeterminate until you initialize them.

11.8 `malloc()` Example

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(void)
```

```
{
```

```
    int *p = malloc(5 * sizeof *p);
```

```
    if (p == NULL)
```

```
    {
```

```
        printf("Memory allocation failed");
```

```
        return 1;
```

```
    }
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        p[i] = (i + 1) * 10;
```

```
    }
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        printf("%d ", p[i]);
```

```
    }
```

```
    free(p);
```

```
    return 0;
```

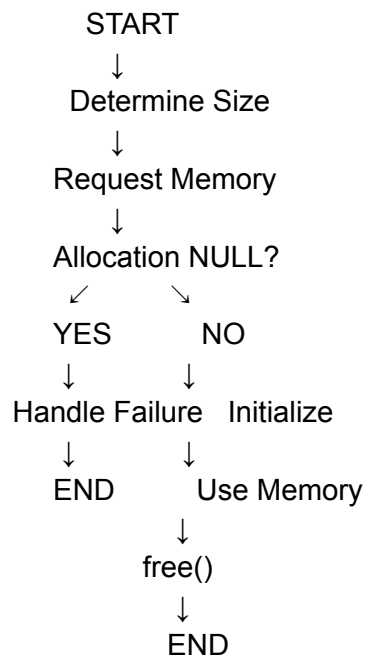


```
}
```

Output:

10 20 30 40 50

11.9 malloc() Flowchart



11.10 calloc()

`calloc()` allocates memory for multiple elements and initializes all allocated bytes to zero.

Syntax

```
ptr = calloc(number_of_elements, size_of_each_element);
```

Example:

```
int *p = calloc(5, sizeof *p);
```

This allocates space for five integers.

For ordinary integer/character objects, this results in their bytes being zeroed; do not generalize this into a guarantee that every possible C type has a semantic "zero value" represented by all-bits-zero.

11.11 malloc() vs calloc()

malloc()

Takes one size argument

Memory is uninitialized

`malloc(n * size)`

calloc()

Takes number of elements + element size

Allocated bytes are initialized to zero

`calloc(n, size)`

Example:

```
malloc(5 * sizeof *p);
```

vs.

```
calloc(5, sizeof *p);
```

11.12 calloc() Example

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(void)
```

```
{
```

```
    int *p = calloc(5, sizeof *p);
```

```
    if (p == NULL)
```

```
    {
```

```
        printf("Allocation failed");
```

```
        return 1;
```

```
    }
```

```
    for (int i = 0; i < 5; i++)
```

```
    {
```

```
        printf("%d ", p[i]);
```

```
    }
```

```
    free(p);
```

```
    return 0;
```

```
}
```

Typical output:

```
0 0 0 0 0
```

11.13 `realloc()`

`realloc()` changes the size of a previously allocated memory block.

Syntax

```
ptr = realloc(ptr, new_size);
```

However, directly assigning the result can lose the original pointer if allocation fails.

Safer pattern:

```
int *temp = realloc(p, new_size);
```

```
if (temp != NULL)
{
    p = temp;
}
```

11.14 Why `realloc()` Is Useful

Suppose:

Initial requirement = 5 integers

Later:

New requirement = 10 integers

Instead of allocating another block and manually copying everything, `realloc()` can resize the existing allocation when possible.

Conceptually:

Initial:

10	20	30	40	50		
----	----	----	----	----	--	--

↓ realloc()

Expanded:



The block may move to a different address.

11.15 **realloc()** Example

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void)
{
    int *p = malloc(3 * sizeof *p);

    if (p == NULL)
    {
        return 1;
    }

    p[0] = 10;
    p[1] = 20;
    p[2] = 30;

    int *temp = realloc(p, 5 * sizeof *p);

    if (temp == NULL)
    {
        free(p);
        return 1;
    }

    p = temp;

    p[3] = 40;
    p[4] = 50;

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", p[i]);
    }
}
```

```
}  
  
free(p);  
  
return 0;  
}
```

Output:

10 20 30 40 50

11.16 Important `realloc()` Behavior

When `realloc()` succeeds:

- Existing contents are preserved up to the smaller of old and new sizes.
- The block may move.
- The old pointer value should no longer be used to access the allocation after a successful move.

When it fails:

```
realloc(p, new_size)
```

returns `NULL`, and the original allocation pointed to by `p` remains valid.

That's why this pattern is important:

```
int *temp = realloc(p, new_size);
```

```
if (temp != NULL)  
{  
    p = temp;  
}
```

11.17 `free()`

`free()` releases memory obtained from dynamic allocation functions.

```
free(p);
```

After freeing:

p

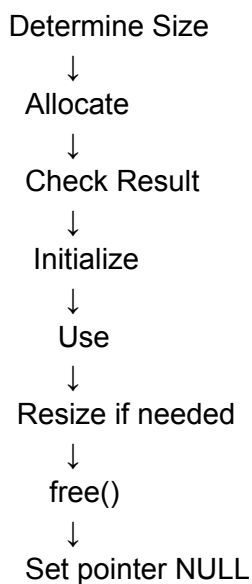
still contains an address value, but it must **not** be dereferenced as if it still pointed to a valid allocation.

A common defensive practice is:

```
free(p);  
p = NULL;
```

11.18 Complete Memory Lifecycle

Professional memory-management flow:



11.19 Dynamic Array

A dynamic array can be implemented using a pointer to allocated storage.

Example:

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main(void)  
{  
    int n;  
  
    printf("Enter number of elements: ");
```

```

scanf("%d", &n);

int *a = malloc((size_t)n * sizeof *a);

if (a == NULL && n > 0)
{
    printf("Allocation failed");
    return 1;
}

for (int i = 0; i < n; i++)
{
    scanf("%d", &a[i]);
}

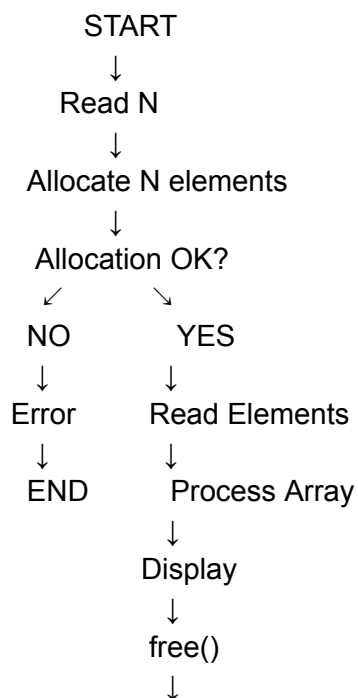
for (int i = 0; i < n; i++)
{
    printf("%d ", a[i]);
}

free(a);

return 0;
}

```

11.20 Dynamic Array Flowchart



END

11.21 Memory Leak

A **memory leak** occurs when dynamically allocated memory is no longer reachable by the program and has not been released.

Example:

```
int *p = malloc(100 * sizeof *p);
```

```
p = NULL;
```

The allocated block has become unreachable without `free()`.

Correct:

```
int *p = malloc(100 * sizeof *p);
```

```
free(p);
```

```
p = NULL;
```

11.22 Dangling Pointer

A dangling pointer refers to memory whose lifetime has ended.

Example:

```
int *p = malloc(sizeof *p);
```

```
free(p);
```

```
/* p is now dangling */
```

Avoid:

```
printf("%d", *p);
```

after `free(p)`.

Better:

```
free(p);
```

```
p = NULL;
```

11.23 Double Free

A **double free** happens when the same allocated block is released more than once.

Dangerous:

```
free(p);  
free(p);
```

After:

```
free(p);
```

use:

```
p = NULL;
```

Then:

```
free(p);
```

is safe because `free(NULL)` has no effect.

11.24 Use-After-Free

This occurs when a program accesses dynamically allocated memory after it has been released.

Example:

```
int *p = malloc(sizeof *p);
```

```
*p = 100;
```

```
free(p);
```

```
printf("%d", *p); // Wrong
```

This produces **undefined behavior**.

11.25 Allocation Failure

Memory allocation can fail.

Always check:

```
int *p = malloc(n * sizeof *p);
```

```
if (p == NULL)
{
    printf("Allocation failed");
    return 1;
}
```

Never assume that allocation always succeeds.

11.26 `malloc()` vs `calloc()` vs `realloc()` vs `free()`

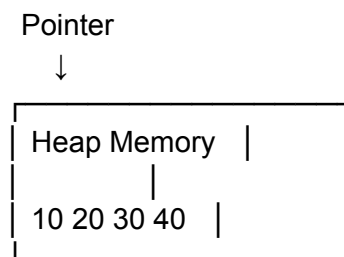
Function	Purpose
<code>malloc()</code>	Allocate memory
<code>calloc()</code>	Allocate and zero-initialize bytes
<code>realloc()</code>	Resize an existing allocation
<code>free()</code>	Release allocation

Easy memory trick:

```
malloc → Allocate
calloc → Allocate + Clear
realloc → Resize
free → Release
```

11.27 Dynamic Memory and Pointers

Dynamic memory is accessed through pointers.



Example:

```
int *p = malloc(4 * sizeof *p);
```

Here:

p → points to dynamically allocated storage

11.28 Dynamic Memory for Structures

You can dynamically allocate a structure.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Student
{
    int roll;
    float marks;
};
```

```
int main(void)
{
    struct Student *s = malloc(sizeof *s);

    if (s == NULL)
    {
        return 1;
    }

    s->roll = 101;
    s->marks = 88.5f;

    printf("%d %.2f", s->roll, s->marks);

    free(s);
}
```

```
    return 0;  
}
```

Notice:

`s->roll`

because `s` is a pointer to a structure.

11.29 Dynamic Array of Structures

Example:

```
struct Student *students =  
    malloc(n * sizeof *students);
```

Then:

```
students[0].roll  
students[1].roll  
students[2].roll
```

can be used to access individual elements.

11.30 `sizeof` with Dynamic Memory

Suppose:

```
int *p = malloc(10 * sizeof *p);
```

Do not expect:

```
sizeof(p)
```

to tell you that 10 integers were allocated.

`sizeof(p)` gives the size of the **pointer itself**.

The allocation size must be tracked separately.

Example:

```
size_t n = 10;
```

```
int *p = malloc(n * sizeof *p);
```

Here `n` records the number of elements.

11.31 Best Practice: `sizeof *pointer`

Instead of:

```
int *p = malloc(n * sizeof(int));
```

you can write:

```
int *p = malloc(n * sizeof *p);
```

This is often preferred because it automatically follows the pointer's declared type.

For example:

```
struct Student *s =  
    malloc(n * sizeof *s);
```

11.32 Common Mistakes

Mistake 1: Forgetting `free()`

```
int *p = malloc(100 * sizeof *p);
```

```
/* use p */
```

```
return 0;
```

Better:

```
free(p);
```

Mistake 2: Dereferencing `NULL`

```
int *p = malloc(100 * sizeof *p);
```

```
if (p == NULL)
```

```
{  
    /* handle failure */  
}
```

Always check before use.

Mistake 3: Using Memory After `free()`

```
free(p);
```

```
*p = 10; // Wrong
```

Mistake 4: Losing the Original Pointer with `realloc()`

Avoid:

```
p = realloc(p, new_size);
```

when you need to preserve `p` if resizing fails.

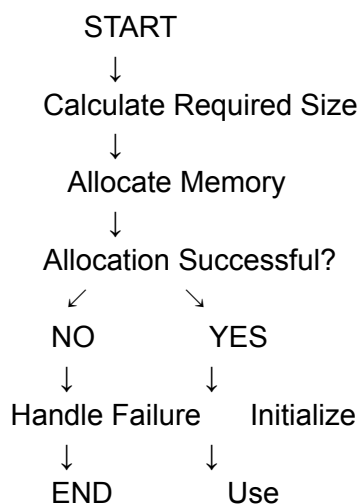
Prefer:

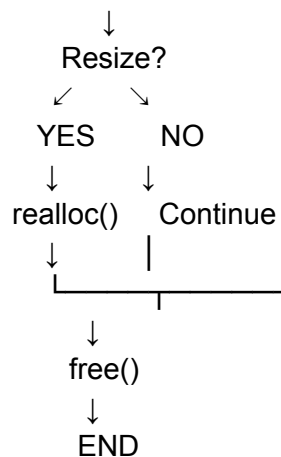
```
int *temp = realloc(p, new_size);
```

```
if (temp != NULL)
```

```
{  
    p = temp;  
}
```

11.33 Memory Management Flowchart





11.34 Interview Questions

1. What is dynamic memory allocation?

It is the process of obtaining and releasing memory at runtime.

2. Which header provides dynamic memory functions?

```
#include <stdlib.h>
```

3. What is **malloc()**?

It allocates a specified number of bytes without initializing their contents.

4. What is **calloc()**?

It allocates storage for multiple elements and initializes all allocated bytes to zero.

5. What is **realloc()**?

It changes the size of an existing dynamically allocated block.

6. What is **free()**?

It releases dynamically allocated storage.

7. What is a memory leak?

Allocated memory that becomes unreachable without being released.

8. What is a dangling pointer?

A pointer that refers to an object whose lifetime has ended or to storage that has been released.

9. Why should we check `malloc()` for `NULL`?

Because allocation can fail.

10. Why is `sizeof *p` useful?

It derives the allocation element size from the pointer's pointed-to type and reduces type duplication.

11.35 Practical Lab

Task 1

Dynamically allocate memory for `N` integers.

Task 2

Read and display dynamically allocated elements.

Task 3

Find the sum and average of a dynamic array.

Task 4

Find the maximum and minimum.

Task 5

Resize an array using `realloc()`.

Task 6

Create a dynamically allocated array of structures.

Task 7

Allocate memory using `calloc()` and observe the initial values.

Task 8

Write a program that properly allocates, uses, and releases memory.

11.36 Quick Revision

malloc()
↓
Allocate memory
↓
Check NULL
↓
Use memory
↓
free()
calloc()
↓
Allocate + zero-initialize bytes
realloc()
↓
Resize allocation
↓
Check returned pointer
free()
↓
Release allocation

Memory Safety

Allocate
↓
Initialize
↓
Use
↓
Release

Avoid:

Memory Leak
Dangling Pointer
Use-After-Free
Double Free

Module 11 Key Takeaway

The four functions you **must remember** are:

malloc() → Allocate

calloc() → Allocate + zero-initialize bytes

realloc() → Resize

free() → Release

Module 12: File Handling in C

File handling allows a C program to **store data permanently** in files instead of losing it when the program terminates.

12.1 Learning Objectives

You will learn:

- What file handling is.
 - **FILE** and file pointers.
 - Opening and closing files.
 - File modes.
 - Reading from files.
 - Writing to files.
 - Appending data.
 - Character-based file operations.
 - String-based file operations.
 - Formatted file operations.
 - Binary files.
 - Error handling.
 - File positioning functions.
-

12.2 What is File Handling?

Normally, data stored in variables exists only while the program is running.

Program starts



Data stored in memory



Program terminates



Data is lost

A file allows persistent storage:

Program



File



Data remains after program terminates

Examples:

students.txt

employees.txt

marks.dat

database.dat

12.3 Why File Handling?

File handling is useful for:

- Storing student records.
 - Saving application data.
 - Reading configuration files.
 - Generating reports.
 - Maintaining logs.
 - Storing large amounts of data.
 - Sharing data between program executions.
-

12.4 FILE Pointer

C provides the **FILE** type for working with files.

Example:

```
FILE *fp;
```

Here:

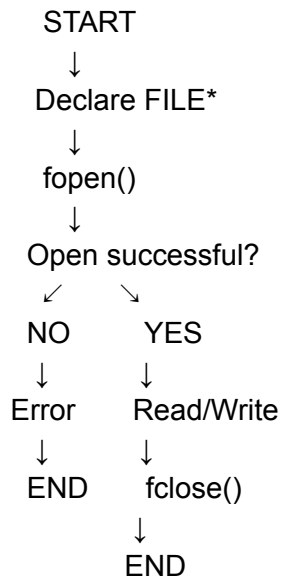
fp → file stream

The actual file is managed through the C standard I/O library.

Include:

```
#include <stdio.h>
```

12.5 Basic File Handling Flow



12.6 Opening a File — **fopen()**

Syntax:

```
FILE *fp = fopen("filename", "mode");
```

Example:

```
FILE *fp = fopen("data.txt", "r");
```

Here:

data.txt → File name

r → Opening mode

fp → File pointer

12.7 Always Check **fopen()**

A file may fail to open.

Professional pattern:

```
FILE *fp = fopen("data.txt", "r");

if (fp == NULL)
{
    printf("Unable to open file\n");
    return 1;
}
```

Possible reasons include:

- File does not exist.
- Permission problems.
- Invalid path.
- Resource limitations.

12.8 Closing a File

Use:

```
fclose(fp);
```

Example:

```
FILE *fp = fopen("data.txt", "r");

if (fp != NULL)
{
    fclose(fp);
}
```

Closing a file releases associated resources and completes pending output operations.

12.9 File Opening Modes

Important modes:

Mode	Purpose
"r"	Read existing file
"w"	Write; creates/truncates file

"a"	Append; creates if needed
"r+"	Read and write existing file
"w+"	Read and write; creates/truncates
"a+"	Read and append; creates if needed

12.10 Read Mode — "r"

```
FILE *fp = fopen("data.txt", "r");
```

The file must already exist.

If it doesn't exist:

`fopen()` → NULL

12.11 Write Mode — "w"

```
FILE *fp = fopen("data.txt", "w");
```

If the file doesn't exist:

Create file

If it already exists:

Existing contents are discarded

⚠ Therefore, don't use "w" when you want to preserve existing contents.

12.12 Append Mode — "a"

```
FILE *fp = fopen("data.txt", "a");
```

If the file doesn't exist, it is created.

If it exists:

New data → End of file

Existing content is preserved.

12.13 Writing to a File — `fprintf()`

`fprintf()` works like `printf()`, but writes formatted output to a specified stream.

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *fp = fopen("student.txt", "w");

    if (fp == NULL)
    {
        return 1;
    }

    fprintf(fp, "Name: Prasanna\n");
    fprintf(fp, "Marks: %d\n", 90);

    fclose(fp);

    return 0;
}
```

File content:

Name: Prasanna
Marks: 90

12.14 `fputc()`

`fputc()` writes one character to a file stream.

Example:

```
FILE *fp = fopen("data.txt", "w");
```

```
if (fp == NULL)
{
    return 1;
}
```

```
fputc('A', fp);
```

```
fclose(fp);
```

File contains:

A

12.15 fputs()

fputs() writes a string to a file stream.

Example:

```
FILE *fp = fopen("data.txt", "w");
```

```
if (fp == NULL)
{
    return 1;
}
```

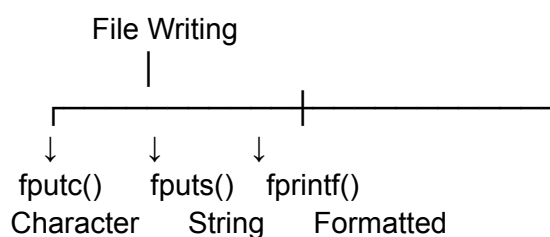
```
fputs("Hello World\n", fp);
```

```
fclose(fp);
```

File:

Hello World

12.16 Writing Methods



12.17 Reading from a File — `fgetc()`

`fgetc()` reads one character from a file stream.

Example:

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *fp = fopen("data.txt", "r");

    if (fp == NULL)
    {
        return 1;
    }

    int ch;

    while ((ch = fgetc(fp)) != EOF)
    {
        putchar(ch);
    }

    fclose(fp);

    return 0;
}
```

Important

Use an `int` for the result of `fgetc()` so it can represent every possible unsigned character value as well as `EOF`.

12.18 `EOF`

`EOF` means **End Of File**.

It is a special negative integer constant returned by input functions such as `fgetc()` when no more characters are available.

Typical pattern:

```
int ch;

while ((ch = fgetc(fp)) != EOF)
{
    ...
}
```

12.19 Reading a String — `fgets()`

`fgets()` can read a line from a file.

```
char line[100];

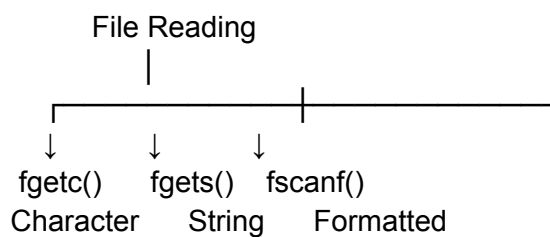
if (fgets(line, sizeof line, fp) != NULL)
{
    printf("%s", line);
}
```

It reads at most:

`sizeof(line) - 1`

characters and adds `'\0'` if successful.

12.20 Reading Methods



12.21 `fscanf()`

`fscanf()` reads formatted input from a file stream.

Example file:

101 95

Program:

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *fp = fopen("marks.txt", "r");

    if (fp == NULL)
    {
        return 1;
    }

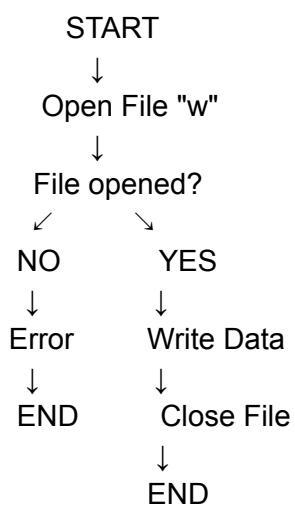
    int roll;
    int marks;

    if (fscanf(fp, "%d %d", &roll, &marks) == 2)
    {
        printf("Roll = %d\n", roll);
        printf("Marks = %d\n", marks);
    }

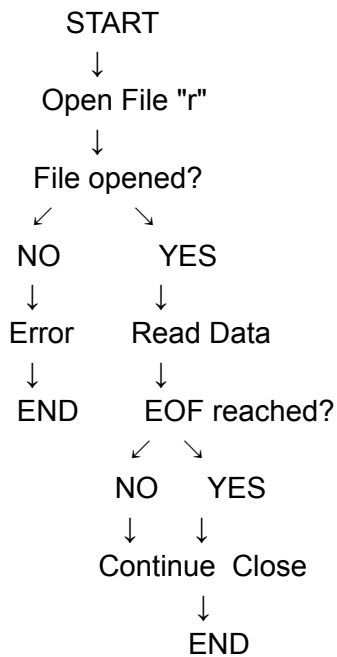
    fclose(fp);

    return 0;
}
```

12.22 File Writing Flowchart



12.23 File Reading Flowchart



12.24 Append Data to a File

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *fp = fopen("data.txt", "a");

    if (fp == NULL)
    {
        return 1;
    }

    fprintf(fp, "New record\n");

    fclose(fp);

    return 0;
}
```

Existing content remains, and the new output is appended.

12.25 Text Files vs Binary Files

Text File

Stores data in a human-readable representation.

Example:

101 Prasanna 90

Common extension:

.txt

Binary File

Stores data as binary bytes according to the representation written by the program.

Common extension:

.dat

Binary files are not necessarily human-readable.

12.26 Binary File Functions

Two important functions:

fread()

fwrite()

Example:

```
fwrite(&value, sizeof value, 1, fp);
```

Reading:

```
fread(&value, sizeof value, 1, fp);
```

12.27 **fwrite()** Example

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *fp = fopen("data.dat", "wb");

    if (fp == NULL)
    {
        return 1;
    }

    int value = 100;

    fwrite(&value, sizeof value, 1, fp);

    fclose(fp);

    return 0;
}
```

12.28 fread() Example

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("data.dat", "rb");

    if (fp == NULL)
    {
        return 1;
    }

    int value;

    if (fread(&value, sizeof value, 1, fp) == 1)
    {
        printf("Value = %d\n", value);
    }

    fclose(fp);

    return 0;
}
```

12.29 File Position

C maintains a current position in a file stream.

Important functions:

fseek()
ftell()
rewind()

12.30 `ftell()`

`ftell()` reports the current file position for a stream.

Example:

```
long position = ftell(fp);
```

Always check for errors where appropriate.

12.31 `fseek()`

`fseek()` changes the file position.

Syntax:

```
fseek(fp, offset, origin);
```

Common origins:

SEEK_SET → Beginning
SEEK_CUR → Current position
SEEK_END → End

Example:

```
fseek(fp, 0, SEEK_END);
```

This moves to the end of the stream for streams where that operation is meaningful.

12.32 `rewind()`

`rewind()` moves the file position back to the beginning.

```
rewind(fp);
```

It also clears the stream's error and EOF indicators.

12.33 File Position Flow

Beginning



Read



Position moves forward



`fseek()`



New Position



Read/Write

12.34 Error Handling

Always verify file operations.

Example:

```
FILE *fp = fopen("data.txt", "r");
```

```
if (fp == NULL)
{
    perror("data.txt");
    return 1;
}
```

`perror()` prints a message describing the last library/system error associated with `errno`.

12.35 feof()

`feof()` checks whether the EOF indicator is set for a stream.

However, don't use:

```
while (!feof(fp))
```

as the primary pattern for reading characters.

Instead, use the return value of the actual input operation:

```
int ch;
```

```
while ((ch = fgetc(fp)) != EOF)
{
    ...
}
```

This is an important interview point.

12.36 Practical Program: Copy One File to Another

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *source = fopen("source.txt", "r");
    FILE *destination = fopen("destination.txt", "w");

    if (source == NULL || destination == NULL)
    {
        if (source != NULL)
            fclose(source);

        if (destination != NULL)
            fclose(destination);

        return 1;
    }
}
```

```
int ch;
```

```

while ((ch = fgetc(source)) != EOF)
{
    fputc(ch, destination);
}

fclose(source);
fclose(destination);

return 0;
}

```

Flow:

```

Source File
↓
Read Character
↓
Write Character
↓
Destination File

```

12.37 Practical Program: Count Characters

```

#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("data.txt", "r");

    if (fp == NULL)
    {
        return 1;
    }

    long count = 0;
    int ch;

    while ((ch = fgetc(fp)) != EOF)
    {
        count++;
    }

    printf("Characters = %ld\n", count);
}

```

```
fclose(fp);  
  
return 0;  
}
```

12.38 File Handling Best Practices

- ✓ Always check whether `fopen()` succeeded.
 - ✓ Close files after use.
 - ✓ Use the correct mode.
 - ✓ Don't use `"w"` if you need to preserve existing contents.
 - ✓ Use `fgets()` for bounded line input.
 - ✓ Check return values from file operations.
 - ✓ Use `int` for `fgetc()` results.
 - ✓ Handle errors properly.
-

12.39 Common Mistakes

Mistake 1

```
FILE *fp = fopen("data.txt", "r");
```

```
fprintf(fp, "Hello");
```

✗ Wrong because `"r"` is a read mode.

Use:

```
FILE *fp = fopen("data.txt", "w");
```

or another appropriate writing mode.

Mistake 2

Forgetting:

```
fclose(fp);
```

This can cause resource leaks and may prevent buffered output from being properly finalized.

Mistake 3

Using "w" accidentally:

```
fopen("data.txt", "w");
```

This truncates an existing file.

Mistake 4

Using:

```
while (!feof(fp))
```

as the main reading condition.

Prefer checking the actual input operation.

12.40 Important Functions

Function	Purpose
<code>fopen()</code>	Open file
<code>fclose()</code>	Close file
<code>fgetc()</code>	Read character
<code>fputc()</code>	Write character
<code>fgets()</code>	Read string/line
<code>fputs()</code>	Write string

`fscanf()` Read formatted data
)

`fprintf` Write formatted data
()

`fread()` Read binary data

`fwrite()` Write binary data
)

`fseek()` Change file position

`ftell()` Get file position

`rewind()` Move to beginning
)

`perror()` Display error description
)

12.41 Interview Questions

1. What is file handling?

It is the process of reading, writing, and managing persistent data stored in files.

2. What is `FILE *`?

It is a pointer to a `FILE` object representing a C stream.

3. Difference between `"r"` and `"w"`?

`"r"` opens an existing file for reading; `"w"` opens for writing and truncates an existing file.

4. Difference between `"w"` and `"a"`?

`"w"` overwrites/truncates existing content; `"a"` preserves existing content and writes at the end.

5. What does `fclose()` do?

It closes the stream and releases associated resources.

6. What is EOF?

It is a special value used by input functions to indicate that no more input is available.

7. What is the difference between `fgetc()` and `fgets()`?

`fgetc()` reads one character; `fgets()` reads a line/string into a character array with a specified limit.

8. What are `fread()` and `fwrite()`?

They perform binary-oriented block input and output.

9. What is `fseek()`?

It changes the current position in a file stream.

10. Why shouldn't we use `while (!feof(fp))` for reading?

Because EOF is only set after an input operation attempts to read past the available data. The input function's return value should control the loop.

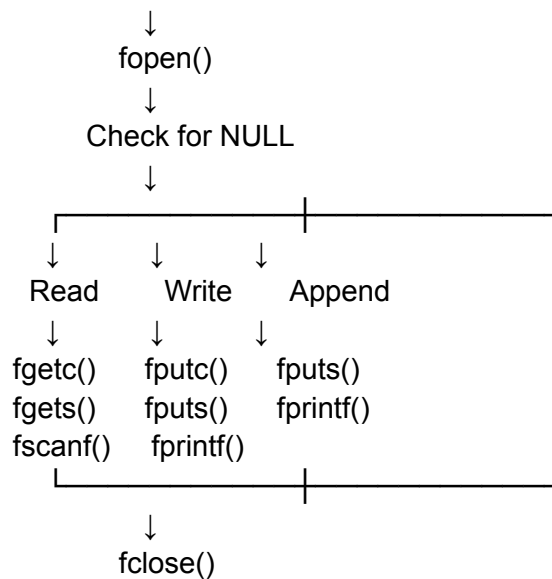
12.42 Practical Lab

1. Create and write a text file.
 2. Read and display a file.
 3. Append data to a file.
 4. Count characters in a file.
 5. Count lines in a file.
 6. Count words in a file.
 7. Copy one file into another.
 8. Search for a word in a file.
 9. Store student records in a file.
 10. Read student records from a file.
 11. Write and read binary data using `fwrite()` and `fread()`.
 12. Experiment with `fseek()` and `ftell()`.
-



Module 12 Quick Revision

FILE HANDLING



★ Must Remember

`fopen()` → Open

`fclose()` → Close

`fgetc()` → Read character

`fputc()` → Write character

`fgets()` → Read line

`fputs()` → Write string

`fscanf()` → Read formatted data

`fprintf()` → Write formatted data

`fread()` → Read binary block

`fwrite()` → Write binary block

`fseek()` → Change position

`ftell()` → Get position

`rewind()` → Go to beginning

Module 13: Preprocessor, Header Files & Macros

The **C preprocessor** works before the actual compilation of C source code. It handles directives such as `#include`, `#define`, and conditional-compilation directives.

13.1 Learning Objectives

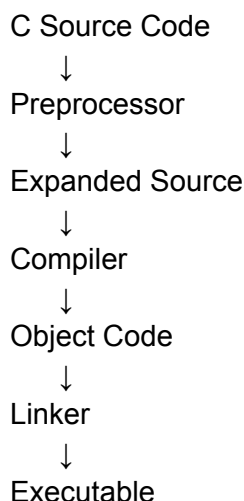
You will learn:

- What the C preprocessor is.
 - Preprocessor directives.
 - `#include`
 - `#define`
 - Object-like and function-like macros.
 - Macro pitfalls.
 - Header files.
 - Conditional compilation.
 - `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif`, `#endif`.
 - Include guards.
 - `#undef`.
 - Compilation stages.
 - Difference between macros and functions.
-

13.2 What is the C Preprocessor?

The **preprocessor** processes source code before the compiler translates it into machine-oriented code.

Basic conceptual flow:



13.3 What is a Preprocessor Directive?

Preprocessor directives begin with `#`.

Examples:

```
#include <stdio.h>
#define PI 3.14159
#ifdef DEBUG
#endif
```

They are instructions to the preprocessor rather than ordinary C statements.

13.4 `#include`

`#include` tells the preprocessor to include the contents of another file.

Example:

```
#include <stdio.h>
```

This makes declarations from the standard I/O header available to your source file.

13.5 Two Common Forms of `#include`

1. System-style header

```
#include <stdio.h>
```

Typically searches implementation-defined system include locations.

2. User/project header

```
#include "student.h"
```

Typically searches the source/project include locations first, followed by implementation-defined include paths.

Exact search behavior is compiler-dependent.

13.6 Why Header Files?

Header files commonly contain:

- Function declarations.
- Type definitions.
- Structure declarations.
- Macro definitions.
- Constants.
- Declarations required by multiple source files.

Example:

```
project/
|
├── main.c
├── student.c
└── student.h
```

Conceptually:

```
student.h
↓
Declarations
```

```
student.c
↓
Definitions
```

```
main.c
↓
Uses them
```

13.7 Creating Your Own Header

math_utils.h

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
```

```
int add(int a, int b);
int square(int n);
```

```
#endif
```

math_utils.c

```
#include "math_utils.h"
```

```
int add(int a, int b)
{
    return a + b;
}
```

```
int square(int n)
{
    return n * n;
}
```

main.c

```
#include <stdio.h>
#include "math_utils.h"

int main(void)
{
    printf("%d\n", add(10, 20));
    printf("%d\n", square(5));

    return 0;
}
```

This demonstrates basic modular program organization.

13.8 #define

`#define` creates a macro.

Example:

```
#define PI 3.14159
```

Then:

```
double area = PI * r * r;
```

The preprocessor substitutes the macro according to its rules before compilation.

13.9 Object-Like Macro

A macro without parameters:

```
#define MAX_SIZE 100
```

Example:

```
int values[MAX_SIZE];
```

Other examples:

```
#define BUFFER_SIZE 1024
```

```
#define COMPANY_NAME "ABC"
```

```
#define DEBUG 1
```

13.10 Function-Like Macro

A macro can accept arguments.

```
#define SQUARE(x) ((x) * (x))
```

Usage:

```
int result = SQUARE(5);
```

Conceptually expands to:

```
int result = ((5) * (5));
```

13.11 Why Parentheses Matter in Macros

Bad:

```
#define SQUARE(x) x * x
```

Then:

```
SQUARE(2 + 3)
```

can expand to:

```
2 + 3 * 2 + 3
```

which does not mean:

```
(2 + 3) * (2 + 3)
```

Better:

```
#define SQUARE(x) ((x) * (x))
```

13.12 Macro Side Effects

Even a carefully parenthesized macro can evaluate its argument more than once.

For example:

```
#define SQUARE(x) ((x) * (x))
```

Then:

```
SQUARE(i++)
```

may evaluate `i++` twice.

This is dangerous and can produce unintended behavior.

For general computation, a function is often safer:

```
int square(int x)
{
    return x * x;
}
```

13.13 Macro vs Function

Macro	Function
Preprocessor substitution	Compiler-level function
No function-call semantics	Has normal function-call semantics
Type checking is different	Parameters have declared types
Can have multiple evaluation issues	Arguments are evaluated once per parameter occurrence in the call
Useful for conditional compilation/constants	Preferred for ordinary reusable computation

Modern C programming generally favors functions when a macro is not specifically needed.

13.14 #undef

`#undef` removes a macro definition.

Example:

```
#define SIZE 100
```

```
#undef SIZE
```

After `#undef`, the macro `SIZE` is no longer defined.

13.15 Conditional Compilation

Conditional compilation allows selected parts of source code to be included or excluded before compilation.

Important directives:

```
#if  
#ifdef  
#ifndef  
#elif  
#else  
#endif
```

13.16 #ifdef

`#ifdef` checks whether a macro is defined.

Example:

```
#define DEBUG
```

```
#ifdef DEBUG  
printf("Debug mode\n");  
#endif
```

Because `DEBUG` is defined, the code inside is included.

13.17 #ifndef

`#ifndef` means:

If this macro is **not defined**.

Example:

```
#ifndef VERSION
#define VERSION 1
#endif
```

This is also commonly used in header guards.

13.18 #if

`#if` evaluates a preprocessor constant expression.

Example:

```
#define VERSION 2

#if VERSION == 2
printf("Version 2\n");
#endif
```

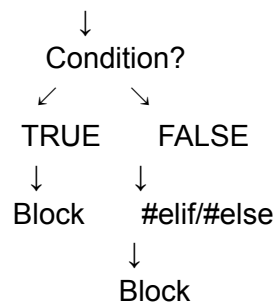
13.19 #else and #elif

Example:

```
#if VERSION == 1
printf("Version 1");
#elif VERSION == 2
printf("Version 2");
#else
printf("Unknown Version");
#endif
```

Flow:

`#if`



13.20 Include Guards

When a header is included multiple times, duplicate declarations can cause problems.

An **include guard** prevents the contents from being processed more than once per translation unit.

Example:

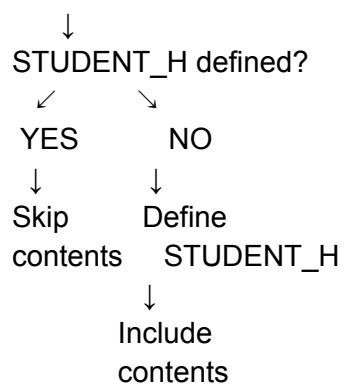
```
#ifndef STUDENT_H
#define STUDENT_H
```

```
struct Student
{
    int roll;
    float marks;
};
```

```
#endif
```

Flow:

Header included



13.21 #pragma once

Many compilers support:

`#pragma once`

as an alternative way to prevent repeated inclusion of a header.

However, `#pragma once` is not part of the ISO C standard itself, while traditional include guards are standard preprocessor techniques.

For portable C, include guards are a dependable choice.

13.22 Predefined Macros

C implementations provide predefined macros.

Common examples include:

```
__FILE__  
__LINE__  
__DATE__  
__TIME__
```

Example:

```
printf("File: %s\n", __FILE__);  
printf("Line: %d\n", __LINE__);
```

These can be useful for diagnostics.

13.23 __FILE__ and __LINE__

Example:

```
#include <stdio.h>
```

```
int main(void)  
{  
    printf("File: %s\n", __FILE__);  
    printf("Line: %d\n", __LINE__);
```

```
    return 0;
}
```

Conceptually:

```
__FILE__ → source-file name
__LINE__ → current source line number
```

13.24 Debugging with Conditional Compilation

You can create a debug macro:

```
#define DEBUG
```

Then:

```
#ifdef DEBUG
printf("Debug: value = %d\n", value);
#endif
```

When `DEBUG` is not defined, the debugging code is excluded during preprocessing.

13.25 Header File Organization

Professional projects commonly separate interfaces from implementations.

Project

```
|
|— main.c
|
|— calculator.c
|
|— calculator.h
```

calculator.h

```
#ifndef CALCULATOR_H
#define CALCULATOR_H
```

```
int add(int a, int b);
```

```
int subtract(int a, int b);
```

```
#endif
```

calculator.c

```
#include "calculator.h"
```

```
int add(int a, int b)
```

```
{  
    return a + b;  
}
```

```
int subtract(int a, int b)
```

```
{  
    return a - b;  
}
```

main.c

```
#include <stdio.h>
```

```
#include "calculator.h"
```

```
int main(void)
```

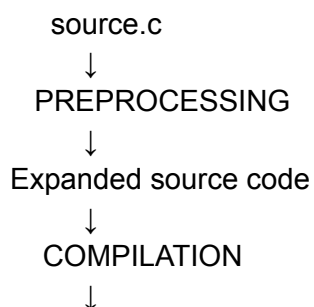
```
{  
    printf("%d\n", add(20, 10));  
    printf("%d\n", subtract(20, 10));
```

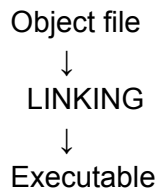
```
  
    return 0;  
}
```

This is the foundation of **multi-file C programming**.

13.26 Compilation Stages

A simplified C build process:





Preprocessing

Handles:

#include
#define
#ifdef
#ifndef
#if

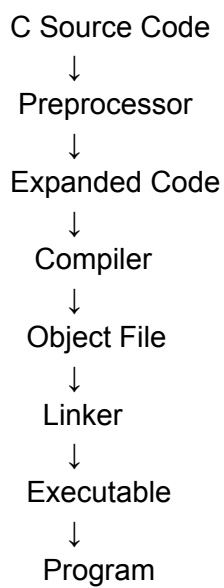
Compilation

Translates C source into an object representation.

Linking

Combines object files and required libraries into an executable.

13.27 Compilation Flowchart



13.28 Common Mistakes

Mistake 1: Missing Header

Using declarations without including the appropriate header can lead to compilation problems or incorrect assumptions.

Mistake 2: Bad Macro

Avoid:

```
#define SQUARE(x) x*x
```

Prefer:

```
#define SQUARE(x) ((x) * (x))
```

But remember that even the improved macro can evaluate `x` more than once.

Mistake 3: Forgetting Include Guards

A reusable project header should generally protect itself against repeated inclusion.

Mistake 4: Using Macros for Everything

Not every constant or computation needs a macro.

For typed constants and calculations, other C mechanisms may be clearer.

13.29 Important Preprocessor Directives

Directive	Purpose
<code>#include</code>	Include header/source content
<code>e</code>	
<code>#define</code>	Define macro

<code>#undef</code>	Remove macro definition
<code>#ifdef</code>	Test whether macro is defined
<code>#ifndef</code>	Test whether macro is not defined
<code>#if</code>	Conditional compilation
<code>#elif</code>	Additional condition
<code>#else</code>	Alternative branch
<code>#endif</code>	End conditional section

13.30 Interview Questions

1. What is a preprocessor?

A tool that processes C source code before compilation.

2. What is `#include`?

A preprocessing directive used to include another file's contents.

3. What is `#define`?

It defines a macro.

4. What is an include guard?

A preprocessor technique that prevents a header's contents from being processed multiple times in one translation unit.

5. What is the difference between `#ifdef` and `#ifndef`?

`#ifdef` → macro IS defined

`#ifndef` → macro IS NOT defined

6. What is conditional compilation?

Selecting source sections for compilation based on preprocessor conditions.

7. Why are parentheses important in macros?

They help prevent operator-precedence problems during textual substitution.

8. What is a major danger of function-like macros?

Arguments can be evaluated multiple times.

9. What is the difference between a macro and a function?

A macro is processed by the preprocessor, while a function is handled by the compiler as a normal callable function.

10. What happens after preprocessing?

Conceptually, the compiler processes the resulting translation unit, followed by assembly/object generation and linking.

13.31 Practical Lab

Task 1

Create a macro:

```
#define PI 3.14159
```

and calculate the area of a circle.

Task 2

Create a function-like macro for finding the maximum of two values.

Task 3

Create a header file containing arithmetic function declarations.

Task 4

Create `.c` and `.h` files for a calculator.

Task 5

Use `#ifdef DEBUG` to create debug output.

Task 6

Create an include guard for your own header.

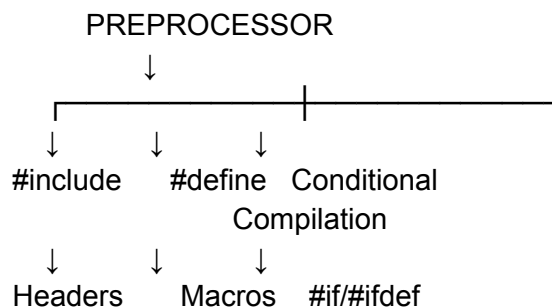
Task 7

Experiment with `__FILE__` and `__LINE__`.

Task 8

Compile a multi-file C program and understand the preprocessing, compilation, and linking stages.

Module 13 Quick Revision



Must Remember

`#include` → Include header
`#define` → Define macro
`#undef` → Remove macro
`#ifdef` → If defined
`#ifndef` → If not defined
`#if` → Conditional compilation
`#else` → Alternative
`#elif` → Another condition
`#endif` → End condition

Module 14: Storage Classes, Scope, Lifetime & Linkage

This module explains **where variables can be accessed, how long they exist, and how they are connected across functions and source files.**

14.1 Learning Objectives

You will learn:

- Scope
 - Lifetime / storage duration
 - Linkage
 - `auto`
 - `static`
 - `extern`
 - `register`
 - Local and global variables
 - Static local variables
 - Static global variables
 - External variables
 - Multi-file programs
 - Common interview questions
-

14.2 What is a Storage Class?

A storage-class specifier helps describe properties of an identifier such as its **storage duration, scope, and linkage**.

Common storage-class specifiers include:

`auto`
`static`
`extern`
`register`

In modern C, the exact meaning depends on where the specifier is used. Don't think of these merely as "where the variable is stored."

14.3 Three Important Concepts

Before learning storage classes, understand these three terms.

1. Scope

Where can I access this name?

2. Storage Duration

How long does the object exist?

3. Linkage

Can the same identifier refer to the same entity across different scopes or source files?

Remember:

Scope → Where?

Duration → How long?

Linkage → Connected to what?

14.4 Scope

Scope determines the region of source code where an identifier can be referred to.

Common scopes include:

Block scope

Function scope

Function prototype scope

File scope

14.5 Block Scope

A variable declared inside a block has block scope.

Example:

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int x = 10;
```

```
    {
```

```
        int y = 20;
```

```
        printf("%d %d\n", x, y);
```

```
    }
```

```
    printf("%d\n", x);

    return 0;
}
```

`y` cannot be referred to outside its block.

Conceptually:

```
main()
|
|— x
|
|— inner block
    |
    |— y
```

14.6 File Scope

An identifier declared outside all functions generally has file scope.

Example:

```
int count = 10;

void display(void)
{
    printf("%d", count);
}

int main(void)
{
    display();
}
```

`count` has file scope.

14.7 Function Scope

Function scope applies specifically to **labels** used with `goto`.

Example:

```
void test(void)
```

```
{
    goto end;

end:
    printf("Done");
}
```

The label `end` has function scope.

This is a more advanced distinction but is useful for understanding C terminology.

14.8 Function Prototype Scope

Parameter names appearing only in a function prototype have prototype scope.

Example:

```
int add(int a, int b);
```

The names `a` and `b` are meaningful only within that prototype.

You can also write:

```
int add(int, int);
```

14.9 Storage Duration

Storage duration describes how long an object exists.

The important categories are:

- Automatic
- Static
- Allocated

Automatic

Usually local variables whose lifetime begins when execution enters their block and ends when the block is left.

Static

Objects with static storage duration exist for the entire execution of the program.

Allocated

Objects created dynamically using functions such as `malloc()` exist until explicitly released with `free()`.

14.10 `auto`

`auto` specifies **automatic storage duration** for a block-scope variable.

Example:

```
void test(void)
{
    auto int x = 10;

    printf("%d", x);
}
```

In ordinary C programming, `auto` is rarely written explicitly because block-scope variables are automatic by default unless another storage-class rule applies.

So:

```
int x = 10;
```

inside a function is normally sufficient.

14.11 `static` Local Variable

A local variable declared `static` has:

- Block scope
- Static storage duration
- Its value persists between function calls

Example:

```
#include <stdio.h>
```

```
void counter(void)
{
```

```

    static int count = 0;

    count++;

    printf("%d\n", count);
}

int main(void)
{
    counter();
    counter();
    counter();

    return 0;
}

```

Output:

```

1
2
3

```

14.12 Normal Local vs Static Local

Normal local:

```

void test(void)
{
    int x = 0;
    x++;
    printf("%d\n", x);
}

```

Calling repeatedly:

```

1
1
1

```

Static local:

```

void test(void)
{
    static int x = 0;
    x++;
    printf("%d\n", x);
}

```

}

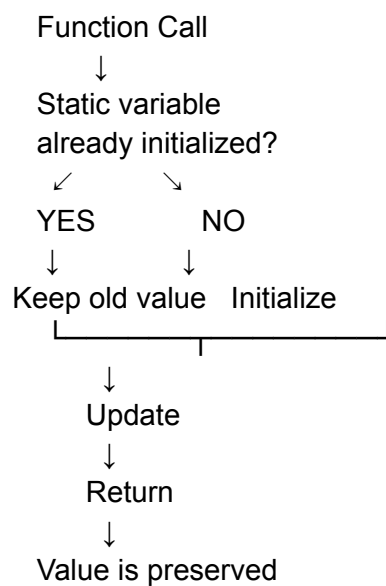
Calling repeatedly:

1

2

3

14.13 Static Local Flowchart



14.14 Why Use Static Local Variables?

They are useful when a function needs to **remember state between calls** without exposing that variable to other functions.

Examples:

- Counters
 - Caching state
 - Function-call tracking
 - Persistent internal state
-

14.15 Static at File Scope

static at file scope gives an identifier **internal linkage**.

Example:

```
static int count = 10;
```

This identifier can be referred to within that source file, but it is not available for external linkage from another translation unit.

This is useful for hiding implementation details.

14.16 Example: File-Private Function

```
static void helper(void)
{
    printf("Internal helper");
}
```

The function has internal linkage.

Another source file cannot use that function through external linkage.

14.17 Internal Linkage

Think of:

static at file scope



Internal linkage



Accessible within that translation unit

This is an important concept in professional multi-file C programming.

14.18 **extern**

extern declares an identifier whose definition is provided elsewhere.

Example:

main.c

```
#include <stdio.h>
```

```
extern int count;
```

```
int main(void)
{
    printf("%d", count);

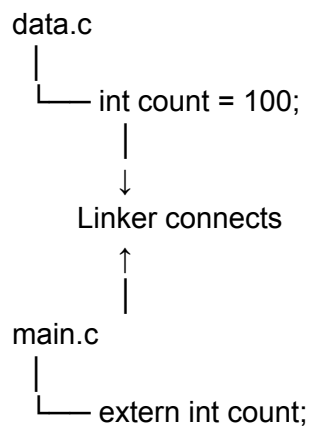
    return 0;
}
```

data.c

```
int count = 100;
```

The declaration in `main.c` refers to the definition in another translation unit.

14.19 extern Flow



14.20 Definition vs Declaration

This is an important concept.

Definition

Creates an object or function.

```
int count = 100;
```

Declaration

Tells the compiler about an existing entity.

```
extern int count;
```

For functions:

```
int add(int, int);
```

is a declaration/prototype, while the function body is the definition.

14.21 `register`

`register` requests that an object be stored in a processor register if possible.

Example:

```
register int i;
```

However, the compiler is free to ignore the request.

Modern optimizing compilers generally make their own register-allocation decisions.

So `register` is rarely useful in modern C.

14.22 Important Restriction with `register`

Historically, the language places restrictions around taking the address of an object declared with `register`.

For example:

```
register int x = 10;
```

You should not write:

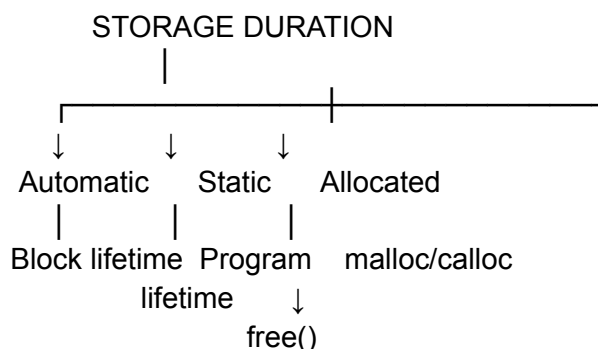
```
&x
```

because the language does not permit taking the address of a `register` object.

14.23 Storage Class Comparison

Specifier	Typical Context	Key Idea
<code>auto</code>	Block scope	Automatic storage duration
<code>static</code>	Block/file scope	Persistent lifetime or internal linkage
<code>extern</code>	File/block declarations	Refers to entity defined elsewhere
<code>register</code>	Block/function parameters	Request register storage; usually ignored

14.24 Storage Duration Comparison



14.25 Scope vs Lifetime

These are **not the same thing**.

Consider:

```
void counter(void)
{
    static int count = 0;
    count++;
}
```

`count` has:

Scope:
Block scope

Lifetime:
Entire program execution

So even though you can only **name** it inside `counter()`, the object itself persists between calls.

14.26 Scope vs Linkage

Consider:

`static int value;`

At file scope:

Scope → File scope

Linkage → Internal linkage

Duration → Static

These describe different properties.

14.27 Global Variable

A variable defined outside functions has file scope.

Example:

```
int total = 0;
```

```
void add(void)
{
    total++;
}
```

Such an object has static storage duration.

If defined without `static`, it normally has external linkage unless another rule applies.

14.28 Static Global Variable

```
static int total = 0;
```

This has:

File scope

Static storage duration

Internal linkage

It can be used by functions in that translation unit, but not referenced from another translation unit through external linkage.

14.29 Initialization of Static Variables

Objects with static storage duration are initialized before program startup.

If you do not provide an initializer, they are initialized appropriately to zero/null values.

Example:

```
static int count;
```

is initialized to:

0

Similarly:

```
static char text[10];
```

has all elements initialized to zero.

14.30 Automatic Variable Initialization

Automatic variables are **not automatically initialized to zero**.

Example:

```
void test(void)
{
    int x;
```

```
    printf("%d", x);  
}
```

Using `x` before assigning it a valid value is erroneous because its value is indeterminate.

Correct:

```
int x = 0;
```

14.31 Important Comparison

Automatic variable



No automatic zero initialization



Initialize before reading

Static-storage object



Initialized before program execution



No explicit initializer → zero/null initialization

14.32 Multi-File C Program

Professional C applications are usually divided into multiple files.

Example:

```
project/  
├──  
├── main.c  
├── calculator.c  
└── calculator.h
```

calculator.h

```
#ifndef CALCULATOR_H  
#define CALCULATOR_H
```

```
int add(int a, int b);
```

```
#endif
```

calculator.c

```
#include "calculator.h"
```

```
int add(int a, int b)
{
    return a + b;
}
```

main.c

```
#include <stdio.h>
#include "calculator.h"

int main(void)
{
    printf("%d", add(10, 20));

    return 0;
}
```

14.33 Why **static** Matters in Multi-File Programs

Suppose:

```
static void helper(void)
{
    ...
}
```

in **calculator.c**.

That helper is intended to be private to that translation unit.

This provides **encapsulation at the source-file level**.

Conceptually:

```
calculator.c
|
├── public_function()
|
└── static helper()
    ↓
    private implementation
```

14.34 `extern` vs `static`

Very important:

`extern`

↓

External linkage / refer to externally defined entity

while:

`static` at file scope

↓

Internal linkage

So:

`extern` → Can refer across translation units

`static` → Restricted to current translation unit

14.35 Example

`data.c`

```
int public_value = 100;
```

```
static int private_value = 200;
```

`main.c`

```
extern int public_value;
```

`public_value` can be referred to from `main.c`.

But:

```
extern int private_value;
```

cannot successfully refer to the file-scope `static` object from another translation unit because `private_value` has internal linkage.

14.36 Common Mistakes

Mistake 1: Thinking **static** always means "global"

Wrong.

static can be used at block scope as well:

```
void test(void)
{
    static int count;
}
```

Here it is a local name with persistent storage duration.

Mistake 2: Thinking **extern** creates the variable

Usually:

```
extern int count;
```

is a declaration, not a definition.

The actual definition might be:

```
int count = 100;
```

in another source file.

Mistake 3: Reading an Uninitialized Automatic Variable

```
int x;
```

```
printf("%d", x);
```

Don't do this.

Initialize it:

```
int x = 0;
```

Mistake 4: Assuming **register** Guarantees a CPU Register

It doesn't.

The compiler decides how to allocate storage.

14.37 Interview Questions

1. What is a storage class?

A storage-class specifier helps specify properties such as storage duration, scope, and linkage.

2. What does **static** mean for a local variable?

It gives the object static storage duration while retaining block scope.

3. What does **static** mean at file scope?

It gives the identifier internal linkage.

4. What is **extern**?

It declares an identifier that can refer to a definition with external linkage, often in another source file.

5. What is **register**?

It is a storage-class specifier that requests register storage, but the compiler may ignore the request.

6. What is scope?

The region of source code where an identifier can be referred to.

7. What is storage duration?

The period during program execution for which an object exists.

8. What is linkage?

It determines whether declarations of the same identifier can refer to the same entity across scopes or translation units.

9. Difference between local and static local variables?

A normal local automatic variable gets a new lifetime on each block entry, while a static local object persists for the entire program execution.

10. Why is `static` useful in multi-file programs?

It allows implementation details to remain private to a translation unit.

14.38 Practical Lab

Task 1

Write a program demonstrating a normal local variable.

Task 2

Create a static local counter.

Task 3

Create a global variable and access it from multiple functions.

Task 4

Create a `static` file-scope variable.

Task 5

Create two `.c` files and use `extern` to share a variable.

Task 6

Create a private `static` helper function in one source file.

Task 7

Experiment with automatic and static initialization.

14.39 Quick Revision

SCOPE

↓

Where can I use the name?

STORAGE DURATION

↓
How long does the object exist?

LINKAGE

↓
Can declarations refer to the same entity?

Storage Classes

auto

↓
Automatic storage duration

static

↓
Persistent lifetime
OR
Internal linkage at file scope

extern

↓
Refers to externally defined entity

register

↓
Request register storage

Module 14 Key Takeaway

Remember this table:

Concept	Remember
auto	Automatic storage duration
Local static	Block scope + program lifetime
File-scope static	Internal linkage
extern	Declaration referring to external entity
register	Register-storage request
Scope	Where name is visible

Duration	How long object exists
Linkage	Whether declarations can refer to same entity

Module 15: Advanced C Concepts & Final Revision

15.1 Learning Objectives

By the end of this module, you will understand:

- Command-line arguments
 - Function pointers
 - `void *` pointers
 - Bitwise operators
 - `const`
 - `volatile`
 - `assert()`
 - `sizeof`
 - `typedef`
 - Common undefined-behavior cases
 - C programming best practices
 - Final interview revision
-

15.2 Command-Line Arguments

Command-line arguments allow values to be passed to a program when it starts.

The standard form of `main()` is:

```
int main(int argc, char *argv[])
```

`argc`

`argc` means **argument count**.

`argv`

argv means **argument vector**.

It is an array of pointers to strings containing the arguments.

15.3 Command-Line Example

```
#include <stdio.h>
```

```
int main(int argc, char *argv[])
{
    printf("Argument count = %d\n", argc);

    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i, argv[i]);
    }

    return 0;
}
```

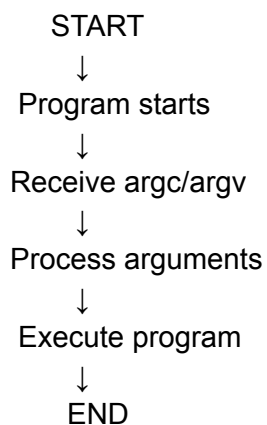
If executed conceptually as:

program.exe Hello 123

You might see:

```
Argument count = 3
Argument 0: program.exe
Argument 1: Hello
Argument 2: 123
```

15.4 Command-Line Flowchart



15.5 Function Pointers

A **function pointer** is a pointer that stores the address of a function.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Function pointer:

```
int (*operation)(int, int);
```

Assign:

```
operation = add;
```

Call:

```
printf("%d", operation(10, 20));
```

Output:

30

15.6 Function Pointer Structure

Function



Address of Function



Function Pointer



Call Function

Example:

```
int (*operation)(int, int);
```

Breakdown:

int → return type

(*operation) → pointer name
(int,int) → parameter types

15.7 Function Pointer Example

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int main(void)
{
    int (*fp)(int, int) = add;

    printf("%d", fp(10, 20));

    return 0;
}
```

Output:

30

15.8 Why Function Pointers Are Important

They are used for:

- Callbacks
- Sorting functions
- Event handlers
- Menu systems
- State machines
- Generic algorithms
- Function dispatch tables

A famous example is the standard library's `qsort()` function, which accepts a comparison function.

15.9 void * Pointer

A `void *` is a pointer to an object type without a specific pointed-to type.

Example:

```
int x = 10;
```

```
void *p = &x;
```

To access the integer through `p`, convert it to an appropriate pointer type:

```
printf("%d", *(int *)p);
```

15.10 Why void * Is Useful

`void *` is useful when writing generic code.

For example:

```
void *data;
```

can point to objects of different types.

The programmer must know the actual object type before dereferencing it.

15.11 Important Restriction

You cannot directly dereference a `void *` in standard C.

Incorrect:

```
*p
```

when `p` has type `void *`.

Instead:

```
*(int *)p
```

if `p` actually points to an `int`.

15.12 Bitwise Operators

Bitwise operators work on individual bits of integer types.

Important operators:

& AND
| OR
^ XOR
~ NOT
<< Left shift
>> Right shift

15.13 Bitwise AND &

Example:

5 = 0101
3 = 0011

& = 0001

Therefore:

5 & 3 = 1

15.14 Bitwise OR |

5 = 0101
3 = 0011

| = 0111

Therefore:

5 | 3 = 7

15.15 Bitwise XOR ^

XOR gives 1 when the corresponding bits are different.

5 = 0101

3 = 0011

^ = 0110

Therefore:

$5 \wedge 3 = 6$

15.16 Bitwise NOT ~

~ flips each bit of its operand.

Example:

unsigned int x = 5;

unsigned int result = ~x;

The exact decimal result depends on the width of unsigned int.

15.17 Left Shift <<

Example:

unsigned int x = 5;

$x \ll 1$

Conceptually:

0101 → 1010

For unsigned values, left shifting by a valid amount corresponds to multiplication by a power of two when the result remains representable.

15.18 Right Shift >>

Example:

```
unsigned int x = 8;
```

```
x >> 1
```

Conceptually:

1000 → 0100

Result:

4

For signed negative values, right-shift behavior is implementation-defined, so use unsigned integers when you need predictable bit-level behavior.

15.19 Bitwise Operator Table

Operator	Operation
&	AND
	OR
^	XOR
~	NOT
<<	Left shift
>>	Right shift

15.20 Bit Masking

Bitwise operations are often used to test or modify individual bits.

Example:

```
unsigned int flags = 5;
```

Binary:

0101

To test the lowest bit:

```
if (flags & 1)
{
    printf("Bit is set");
}
```

This technique is common in:

- Embedded systems
 - Operating systems
 - Device drivers
 - Permissions/flags
 - Networking
-

15.21 Setting a Bit

Suppose:

```
unsigned int flags = 0;
```

To set bit 2:

```
flags |= (1u << 2);
```

Conceptually:

0000

↓

0100

15.22 Clearing a Bit

To clear bit 2:

```
flags &= ~(1u << 2);
```

15.23 Toggling a Bit

To toggle bit 2:

```
flags ^= (1u << 2);
```

15.24 Checking a Bit

```
if (flags & (1u << 2))  
{  
    printf("Set");  
}
```

This is an important practical bit-manipulation technique.

15.25 **const**

const means that an object should not be modified through a particular identifier/expression.

Example:

```
const int x = 10;
```

You should not do:

```
x = 20;
```

15.26 Pointer and **const**

Pointer to constant data

```
const int *p;
```

You cannot modify the pointed-to **int** through **p**.

Constant pointer

```
int *const p = &x;
```

The pointer itself cannot be changed after initialization.

Both constant

```
const int *const p = &x;
```

Neither the pointer nor the pointed-to object can be modified through `p`.

15.27 `const` Example

```
int x = 10;
```

```
const int *p = &x;
```

```
/* *p = 20; */ // Not allowed through p
```

```
x = 20; // x itself is not const
```

This distinction is important.

15.28 `volatile`

`volatile` tells the compiler that the value of an object may change for reasons outside the normal flow of the program.

Example:

```
volatile int flag;
```

It is relevant in areas such as:

- Hardware registers
- Embedded systems
- Memory-mapped I/O
- Certain signal-handling scenarios

`volatile` does **not** make operations atomic or provide thread synchronization.

15.29 Example of `volatile`

Conceptually:

```
volatile int status;
```

A compiler should not assume that `status` remains unchanged between accesses merely because the current thread's code did not modify it.

15.30 `assert()`

The `assert()` macro is used to check assumptions during development.

Include:

```
#include <assert.h>
```

Example:

```
int age = 20;
```

```
assert(age >= 0);
```

If the condition is false, the assertion typically reports diagnostic information and terminates the program.

15.31 Example

```
#include <stdio.h>
```

```
#include <assert.h>
```

```
int divide(int a, int b)
```

```
{  
    assert(b != 0);
```

```
    return a / b;  
}
```

```
int main(void)
```

```
{  
    printf("%d", divide(10, 2));
```

```
    return 0;  
}
```


Output:

5

Assertions are primarily for detecting programming errors and violated assumptions during development, not for normal user-input validation.

15.32 NDEBUG

Assertions can be disabled by defining `NDEBUG` before including `<assert.h>`.

Example:

```
#define NDEBUG  
#include <assert.h>
```

Then:

```
assert(condition);
```

is disabled.

15.33 sizeof

`sizeof` determines the size in bytes of a type or object.

Example:

```
int x;
```

```
printf("%zu", sizeof(x));
```

The result is implementation-dependent.

15.34 sizeof with Arrays

For an actual array object:

```
int a[] = {10, 20, 30, 40};
```

you can calculate the number of elements:

```
size_t n = sizeof(a) / sizeof(a[0]);
```

Result:

4

But this does not work after the array has decayed to a pointer parameter.

15.35 **sizeof** with Pointers

```
int *p;
```

Then:

```
sizeof(p)
```

gives the size of the pointer, **not** the size of the memory block it points to.

This is a common interview question.

15.36 Undefined Behavior

Undefined behavior (UB) means the C standard imposes no requirements on what happens.

Examples include:

Out-of-bounds access

```
int a[5];
```

```
a[5] = 10;
```

Use-after-free

```
free(p);
```

```
*p = 10;
```

Dereferencing a null pointer

```
int *p = NULL;  
*p = 10;
```

Signed integer overflow

```
int x = INT_MAX;  
x++;
```

assuming the operation actually overflows the signed `int`.

Undefined behavior must be taken seriously in professional C programming.

15.37 Undefined Behavior Flow

Incorrect Operation



Undefined Behavior



No guaranteed result



Could crash

Could appear to work

Could corrupt data

Could behave differently after optimization

15.38 Common C Programming Best Practices

1. Initialize variables

```
int count = 0;
```

2. Check pointers

```
if (p != NULL)
```

3. Check allocation results

```
if (p == NULL)
```

4. Free dynamic memory

```
free(p);
```

5. Close files

`fclose(fp);`

6. Avoid buffer overflows

Use bounded input functions such as:

`fgets()`

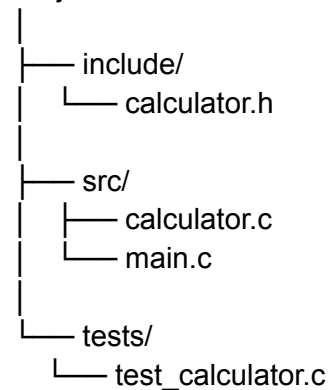
7. Keep functions focused

One function should ideally have one clear responsibility.

15.39 C Programming Architecture

A professional C project can look like:

Project



Conceptually:

Header



Declarations



Source files



Definitions



Compiler



Linker



Executable

15.40 Complete C Learning Roadmap

You have now covered:

Module 1 → C Fundamentals
Module 2 → Variables & Data Types
Module 3 → Operators
Module 4 → Input & Output
Module 5 → Control Statements
Module 6 → Functions
Module 7 → Arrays
Module 8 → Strings
Module 9 → Pointers
Module 10 → Structures, Unions & Enums
Module 11 → Dynamic Memory
Module 12 → File Handling
Module 13 → Preprocessor & Macros
Module 14 → Storage Classes
Module 15 → Advanced C

15.41 Most Important C Topics

For interviews and technical rounds, prioritize:

★★★★★ Pointers
★★★★★ Arrays
★★★★★ Strings
★★★★★ Functions
★★★★★ Structures
★★★★★ Dynamic Memory
★★★★☆ File Handling
★★★★☆ Bitwise Operations
★★★★☆ Preprocessor
★★★★☆ Storage Classes

15.42 Most Important Relationships

Array + Pointer

a[i]

↓

*(a + i)

Pointer + Function

function(&x)



pointer receives address



*p modifies x

Structure + Pointer

p->member

Dynamic Memory + Pointer

malloc()



pointer



heap allocation



free()

File Handling

fopen()



Read/Write



fclose()

15.43 Top Interview Questions

Q1. Difference between array and pointer?

An array is an object containing elements, while a pointer is an object that stores an address. An array expression often converts to a pointer to its first element, but an array and pointer are not the same type.

Q2. Why does `scanf()` use `&`?

For ordinary scalar variables, `scanf()` needs an address where it can store the input.

```
scanf("%d", &x);
```

Q3. What is a dangling pointer?

A pointer that refers to an object or allocated storage whose lifetime has ended.

Q4. What is a memory leak?

Allocated memory that is no longer reachable and therefore cannot be released normally.

Q5. Difference between `malloc()` and `calloc()`?

`malloc()` allocates uninitialized storage, while `calloc()` allocates storage for multiple elements and initializes the allocated bytes to zero.

Q6. What is `realloc()`?

It attempts to resize an existing dynamically allocated memory block.

Q7. What is a segmentation fault?

It is a common manifestation of an invalid memory access, but the exact behavior depends on the operating system and implementation. It is not itself the C language definition of the underlying error.

Q8. What is a structure?

A user-defined type that groups related members, potentially of different types.

Q9. What is a union?

A user-defined type whose members share storage.

Q10. What is recursion?

A technique in which a function calls itself, normally with a base case that terminates the recursion.

Q11. What is a function pointer?

A pointer that stores the address of a function and can be used to call that function.

Q12. What is **static**?

Its meaning depends on context. At block scope, it gives static storage duration; at file scope, it gives internal linkage.

Q13. What is **extern**?

A declaration that can refer to an entity with external linkage defined elsewhere.

Q14. What is **volatile**?

It tells the compiler that an object's value may change for reasons not captured by ordinary program flow.

Q15. What is **const**?

It restricts modification of an object through a particular identifier/expression.

15.44 Final C Cheat Sheet

VARIABLE



Stores Data

ARRAY



Collection of Same-Type Elements

STRING



Character Array + '\0'

POINTER



Stores Address

FUNCTION



Reusable Block of Code

STRUCTURE



Groups Different Data Types

UNION



Shares Storage

ENUM



Named Integer Constants

MALLOC



Dynamic Allocation

CALLOC



Dynamic Allocation + Zeroed Bytes

REALLOC



Resize Allocation

FREE



Release Allocation

FILE



Persistent Data

MACRO



Preprocessor Substitution

STATIC



Persistent Storage / Internal Linkage

EXTERN



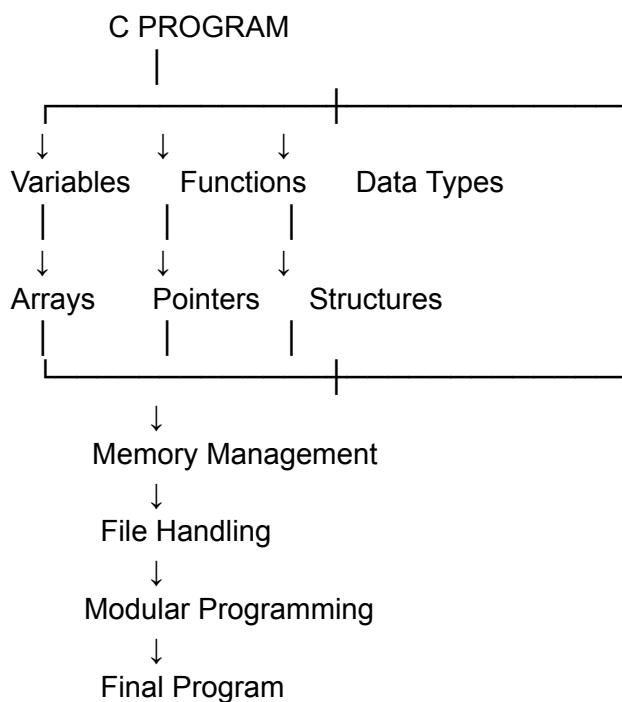
External Linkage Declaration

FUNCTION POINTER



Stores Function Address

15.45 Final C Programming Flow



Module 15 Final Takeaway

If you want to become **strong in C for technical interviews**, don't just memorize syntax. Understand this chain:

C Syntax



Logic



Memory



Pointers



Data Structures



Functions



Dynamic Memory



File Handling



Problem Solving